# 领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】


**1、本教程介绍**

1.1、学了linux可以做什么

1.2、本教程适合哪些人学习?

1.3、本教程特色

**2、Linux入门**

2.1、Linux概述

2.2、Linux的发行版

2.3、Linux VS Windows

**3、VMware和Centos7的安装**

3.1、VMware的安装

（1）下载VMware

（2）双击VMware安装文件

（3）欢迎界面

（4）同意许可证

（5）选择安装路径

（6）用户体验计划

（7）快捷方式

（8）开始安装

（9）等待安装完成

（10）输入许可证

（11）VMware安装完成

3.2、Centos7的安装

3.2.1、配置电脑

（1）进入VMware

（2）新建一个虚拟机

（3）自定义新的虚拟机

（4）解决虚拟机的兼容性

（5）选择当前虚拟机的操作系统

（6）选择虚拟机将来需要安装的系统

（7）配置电脑

（8）查看本机CPU个数

（9）选择CPU的个数

（10）设置虚拟机的内存

（11）选择虚拟机上网方式

（12）选择对应的文件系统的IO方式

（13）选择磁盘类型

（14）选择磁盘的种类

（15）选择虚拟机磁盘大小

（16）电脑配置完成

3.2.2、安装系统

（1）事前检查 bios 的虚拟化设置是否打开

（2）选择CD/DVD的方式安装系统

（3）指定镜像文件

（4）开启虚拟机开始安装系统

（5）系统安装引导界面

（6）选择语言

（7）设置日期和时间

（8）软件选择，选择GNOME桌面

（9）配置磁盘分区

（10）手动添加分区

（11）添加 boot 区（1G容量）

（12）添加swap交换分区

（13）剩下的磁盘空间给根目录
（14）完成分区配置
（15）关闭 kdump
（16）设置网络和主机名
（17）开始安装
（18）设置 root 用户密码， 一定要设置
（19）安装时间比较长 大概需要 10 几分钟
（20）安装完成，重启虚拟机
（21）进入引导界面(以下内容就按照图片走， 就不做过多解释)
（22）完成安装
**4、Linux文件与目录结构**
4.1 Linux文件
4.2 Linux目录结构
**5、VI/VIM编辑器**
5.1、vi/vim是什么
5.2、一般模式
（1）常用语法表
（2）vi/vim键盘图
5.3、编辑模式
（1）进入编辑模式
（2）退出编辑模式
5.4、指令模式
5.5、模式转换
**6、网络配置和系统管理操作**
6.1、VMware提供3种网络设置
1）桥接模式
2）NAT模式
3）仅主机模式
6.2、虚拟机网段和网关
1）查看虚拟网络编辑器
2）虚拟机网络设置
3）DHCP设置
4）DHCP设置
5）查看网关
6.3、配置网络ip地址
6.3.1、ifconfig查看网络信息
6.3.2、ping测试主机之间的网络
1）基本语法
2）案例
6.3.3、修改IP地址为固定的
1）修改IP配置文件
2）重启网络
5.3.4、修改IP地址后可能遇到的问题
6.4、配置主机名
6.4.1、hostname查案当前服务器主机名称
6.4.2、修改主机名称
方式1：修改/etc/hostname文件，需重启
方式2：hostnamectl命令修改，无需重启
6.4.3、修改linux的主机映射文件（hosts文件）
**7、远程登录（Xshell）**
7.1、远程连接
7.2、Xshell的使用
1）下载
2）安装
3）打开xshell
4）创建连接
（1）打开创建连接的窗口
（2）新建会话连接

(3) 输入机器名称（随意）、目标机器ip、端口
(4) 输入目标机器的用户名和密码
(5) 双击即可连接目标机器
(6) 连接成功
(7) 调整字体大小

**8、系统管理**

8.1、Linux中的进程和服务
8.2、service 服务管理（CentOS 6 版本-了解）
1）基本语法
2）经验技巧
3）案例实操
(1) 查看网络服务状态
(2) 停止网络服务
(3) 启动网络服务
(4) 重启网络服务
8.3、chkconfig 设置后台服务的自启配置（CentOS 6 版本）
1） 基本语法
2） 案例实操
(1) 开启/关闭 network(网络)服务的自动启动
(2) 开启/关闭 network 服务指定级别的自动启动
8.4、systemctl（CentOS 7 版本 重点掌握）
1）基本语法
2）经验技巧
3）案例实操
(1) 查看防火墙服务的状态
(2) 停止防火墙服务
(3) 启动防火墙服务
(4) 重启防火墙服务
8.5、systemctl设置后台服务的自启动配置
1）基本语法
2）案例实操
(1) 查看服务开机启动状态
(2) 关闭防火墙服务的自动启动
(3) 开启防火墙服务的自动启动
8.6、系统运行级别
1）Linux运行级别【CentOS 6中】
2）CentOS7 运行级别简化为
multi-user.target：多用户有网，无图形界面
graphical.target：多用户有网，有图形界面
3）systemctl get-default：查看当前运行级别
4）systemctl set-default：修改当前运行级别
8.7、关闭防火墙
1）临时关闭防火墙
(1) 查看防火墙状态
(2) 临时关闭防火墙
2）开机启动时关闭防火墙
(1) 设置开机自动启动防火墙
(2) 设置开机不启动防火墙
8.8、关机重启命令
1）基本语法
sync：将数据由内存同步到硬盘中
half：停机，关闭系统，但不断电
poweroff：关机，断电
reboot：重启，等同于shutdown -r now
shutdown [选项] 时间
2）经验技巧
3）案例实操
(1) sync：将数据由内存同步到硬盘中

(2) reboot：重启
(3) halt：停机（不断电）
(4) 10分钟后关机，并广播消息
(5) shutdown 时间点：指定时间点关机
(6) shutdown -c：取消关机，并广播消息
(7) shutdown -h now：立马关机（等同于reboot）
(8) shutdow -r now：立即重启（等同于reboot）

**9、帮助命令**

9.1、man：获得命令帮助信息
9.2、type：查看命令的类型【内部命令|外部命令】
9.3、help：获取shell内置命令帮助信息
9.4、命令 --help：查看外部命令的帮助信息
9.5、clear：清屏
9.6、reset：彻底清屏
9.7、常用快捷键

**10、文件目录类命令**

10.1、pwd：显示当前工作目录的绝对路径
10.2、ls：列出目录的内容
10.3、cd：切换目录
10.4、mkdir：创建一个新的目录
10.5、rmdir：删除一个空目录
10.6、touch：创建空文件
10.7、cp：复制文件或目录
10.8、rm：删除文件或目录
10.9、mv：移动文件与目录或重命名
10.10、cat：查看文件内容
10.11、more：文件内容分屏查看器
10.12、less：分屏显示文件内容
10.13、echo：输出内容到控制台
10.14、head：显示文件头部内容
10.15、tail：输出文件尾部内容
10.16、>输出重定向 和 >>追加
10.17、ln：创建软连接
10.18、history：查看已执行过的历史命令
10.19、tree：树形显示目录所有文件

**11、时间日期类命令**

11.1、date：日期命令
11.2、date：显示当前时间
11.3、date：显示非当前时间
11.4、date -s：设置系统时间
11.5、cal：查看日历

**12、用户管理命令**

12.1、useradd：添加新用户
12.2、passwd：设置用户密码
12.3、id：查看用户是否存在
12.4、cat /etc/passwd：查看创建了哪些用户
12.5、su：切换用户
12.6、userdel：删除用户
12.7、who：查看登录用户信息
12.8、sudo：设置普通用户具有root权限
12.9、usermod：修改用户

**13、用户组管理命令**

13.1、groupadd：创建用户组
13.2、groupdel：删除用户组
13.3、groupmod：修改用户组组名

**14、文件权限类命令**

14.1、文件属性
14.2、chmod：改变文件或目录权限

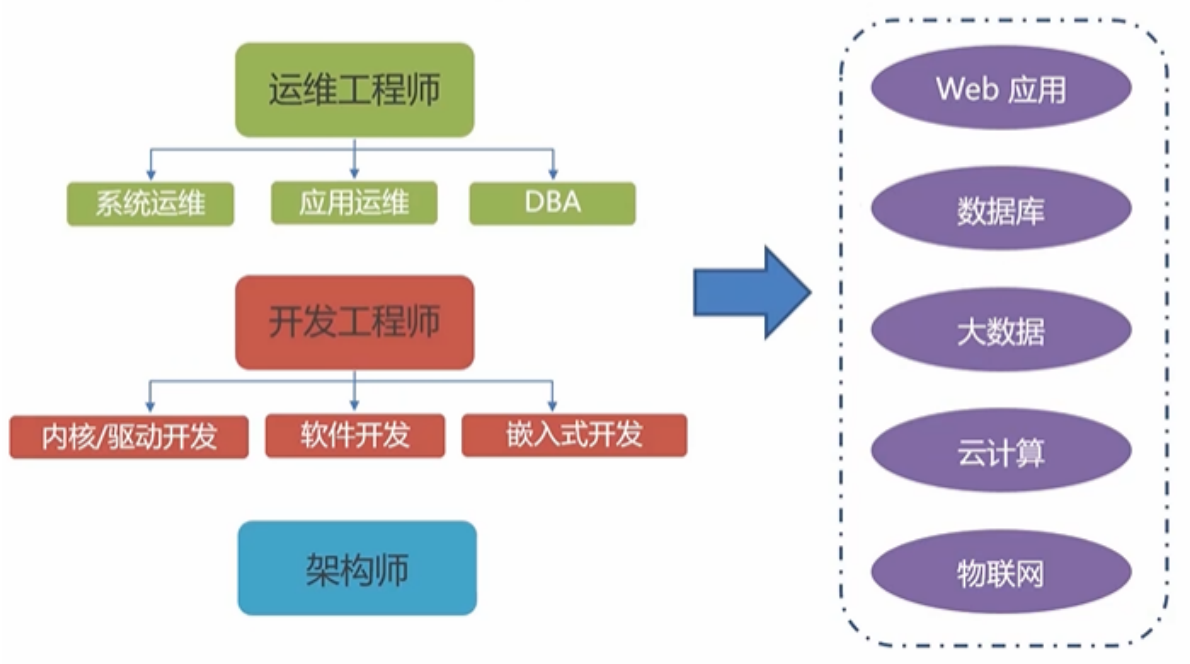
14.3、chown：改变所有者
14.4、chgrp：改变文件或目录所属组
**15、搜索查找类命令**
15.1、find：查找文件或者目录
15.2、locate：快速定位文件路径
15.3、grep：过滤查找即“|”管道符
15.4、which：在PATH下查找命令位置
15.5、whereis：查找可执行文件路径
**16、压缩和解压类命令**
16.1、gzip/gunzip：压缩、解压
16.2、zip/unzip：压缩、解压
16.3、tar：打包、解压（常用）
**17、磁盘管理类命令**
17.1、磁盘查看和分区类
17.1.1、du：查看文件和目录占用的磁盘空间
17.1.2、df：查看磁盘空间使用情况
17.1.3、lsblk：查看设备挂载情况
17.1.4、mount：挂载
17.1.5、umount：卸载
17.2、磁盘管理(添加磁盘、分区、格式化、挂载)
17.2.1、linux如何使用一块硬盘?
1）硬盘
2）分区
3）格式化
4）挂载
17.2.2、硬盘说明
17.3、案例：演示添加硬盘的过程（含分区、格式化、挂载整个过程）
1）需求
2）添加一块20G的硬盘
3）重启机器，即可看到新的硬盘
4）创建第1个主分区：sdb1，大小为4G
5）创建第2个主分区：sdb2，大小为5G
6）创建第3个主分区：sdb3，大小为6G
7）创建扩展分区：sdb4，大小为剩下的空间
8）创建逻辑分区：sdb5，大小为2G
9）创建逻辑分区：sdb6，大小为3G
10）输入p查看分区列表，6个分区都ok了
11）输入w保存并退出（别忘啦）
12）查看/dev/sdb磁盘分区挂载情况
13）格式化第一个分区sdb1
14）格式化其他几个分区（不包含扩展分区）
15）挂载
16）使用mount命令进行挂载
17）设置自动挂载
**18、进程管理类命令（ps、kill、pstree、top、netstat）**
18.1、ps：查看当前系统进程状态
1）基本语法
2）选项说明
3）ps aux：查看进程CPU、内存占用率
4）ps -ef：显示父子进程信息
5） ps -Lf pid：查看某个进程的所有内核线程
18.2、kill：终止进程
1）基本语法
2）选项
3）案例实操
（1）关闭火狐浏览器
（2）通过进程名称杀死进程
18.3、pstree：查看树进程

1）基本语法
2）选项说明
3）案例实操
（1）显示进程pid
（2）查看一个特定的进程
18.4、top：实时监控系统状态
1）语法
2）选项说明
3）操作说明
4）查询结果字段解释
（1）第1行信息为任务队列信息
（2）第2行为进程信息
（3）第3行为CPU信息
（4）第4行为物理内存信息
（5）第5行为交换分区信息
（6）进程信息区
5）2个案例
18.4、netstat：显示网络状态和端口占用信息
1） 基本语法
2）选项说明
3）连接状态详解
4） 案例实操
（1） 通过进程号查看sshd进程的网络信息
（2） 查看22端口号是否被占用
**19、系统定时任务（crontab）**
19.1、需开启crond服务
19.2、crontab：定时任务设置
1）基础语法
2）选项说明
3）添加任务
（1）编辑任务
（2）任务格式
（3）特殊符号说明
（4） 特定时间执行命令
19.3、案例
（1）当前用户添加一个任务
（2）参看当前用户任务列表
（3）删除当前用户所有任务
（4）如果想删除某个任务怎么办呢?
**20、软件包管理**
20.1、RPM
20.1.1、RPM 概述
20.1.2、RPM 查询命令（rpm -qa）
1） 基本语法
2）经验技巧
3） 案例实操：查询firefox软件安装情况
20.1.3、RPM 卸载命令（rpm -e）
1） 基本语法
2）选项说明
3） 案例实操：卸载firefox软件
20.1.2 RPM 查询命令（rpm -ivh）
1） 基本语法
2）选项说明
3） 案例实操：安装firefox软件
20.2 YUM 仓库配置
20.2.1 YUM 概述
20.2.2 YUM 的常用命令
1） 基本语法

2） 选项说明
3） 参数说明
4）常用命令
5）案例实操实操（java1.8 查找、安装、卸载）
20.2.3、修改网络 YUM 源
1）安装 wget
2）在/etc/yum.repos.d/目录下， 备份默认的 repos 文件
3）下载网易163或者是aliyun的repos文件，任选其一
4）使用下载好的repos文件替换默认的repos文件
5）清理旧缓存数据， 缓存新数据
6）测试，安装的时候会提示阿里云的域名

# 1、本教程介绍

## 1.1、学了linux可以做什么




## 1.2、本教程适合哪些人学习?

- 所有程序员
- 大数据工程师
- 运维工程师

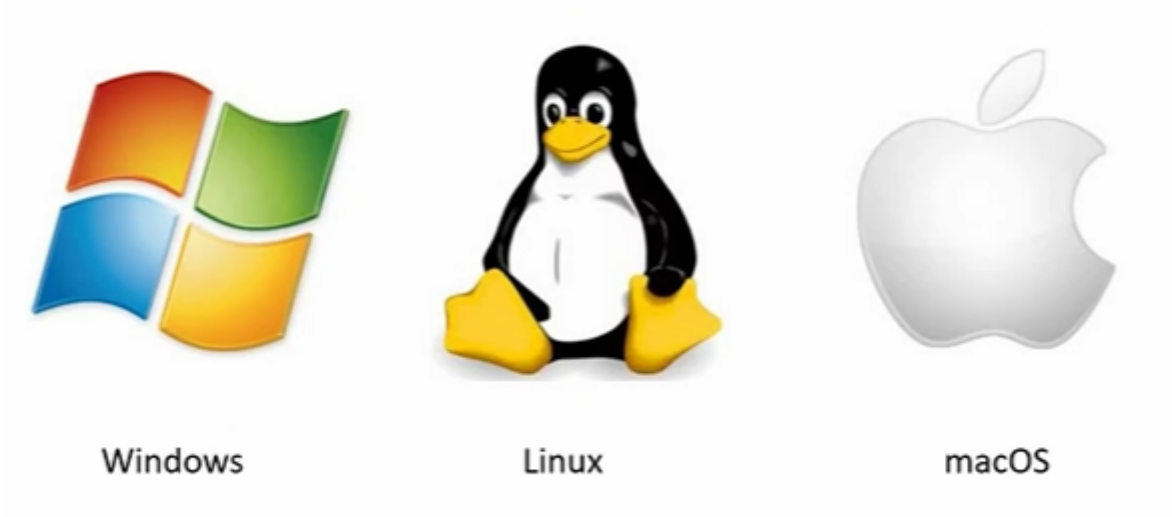
## 1.3、本教程特色

- 基于CentOS 7
- 简单易上手，即使是技术小白也适合学习
- 详细的常用命令讲解
- 学完之后，基本上就掌握了Linux所有的知识
- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

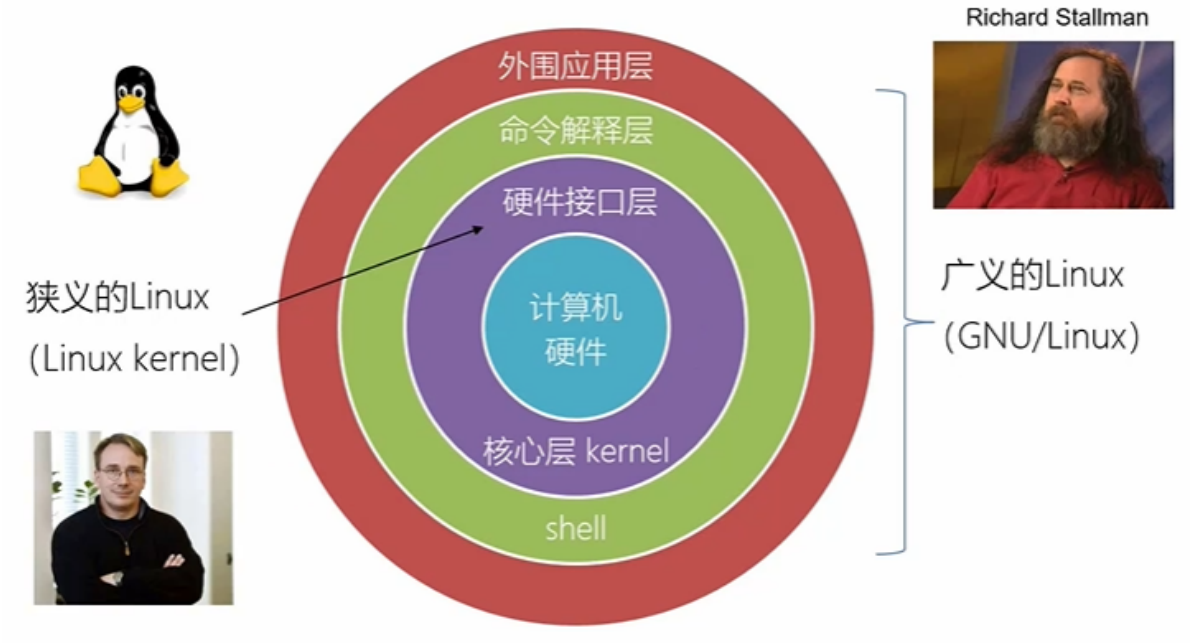
# 2、Linux入门

## 2.1、Linux概述

Linux是一个操作系统（OS），我们比较熟悉的操作系统有Windows、苹果电脑上的macOS，而Linux也是和他们并驾齐驱的操作系统。



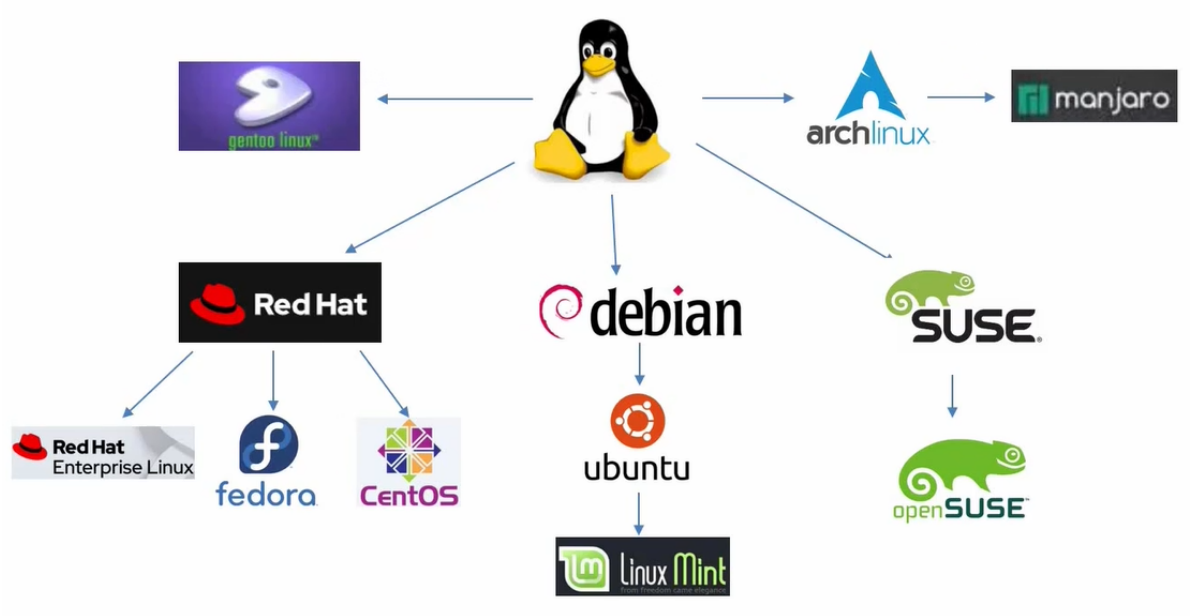

Linux内核最初只是由芬兰人林纳斯·托瓦兹(Linus Torvalds)在赫尔辛基大学上学时出于个人爱好而编写的。Liux是一套免费使用和自由传播的类Unix操作系统，是一个基于POSX和UNTX的多用户、多任务、支持多线程和多CPU的操作系统。Liux能运行主要的UNX工具软件、应用程序和网络协议。它支持32位和64位硬件。Linux继承了Unix以网络为核心的设计思想，是一个性能稳定的多用户网络操作系统。 目前市面上较知名的发行版有：Ubuntu、RedHat、.CentOS、Debain、Fedora、SuSE、OpenSUSE。




## 2.2、Linux的发行版

目前用的最多的是CentOS和ubuntu。

- CentOS：更稳定一些，适合做服务器使用
- ubuntu：主打界面漂亮，更新比较激进，比较适合个人学习、测试使用。




## 2.3、Linux VS Windows

| 特点比较 | Windows | Linux |
| --- | --- | --- |
| 费用 | 收费且很贵 | 免费或少许收费 |
| 软件与支持 | 数量和质量都有优势，不过大部分为收费软件；由微软官方提供支持和服务 | 开源自由软件，用户可以修改定制和再发布，由于基本免费没有资金支持，部分软件质量和体验欠缺；由全球所有的Linux开发者和自由软件社区提供支持 |
| 安全性 | 三天两头打补丁安装系统更新，还是会中病毒木马 | 要说Linux没有安全问题，那当然是不可能的；相对来说，肯定比Windows平台更加安全 |
| 使用习惯 | 基本都是纯图形界面下操作使用，依靠鼠标和键盘完成一切操作，用户上手容易，入门简单 | 兼容图形界面操作和命令行操作，可以只用键盘完成一切操作；新手入门较困难，需要一些学习和指导，一旦熟练之后效率极高 |
| 可定制性 | 封闭的，系统可定制性很差 | 开源，可定制性非常高 |
| 应用场景 | 个人电脑桌面操作系统主要是Windows | 服务器操作系统主要是Linux；世界上大部分软件和服务都是运行在Linux之上的。 |


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

# 3、VMware和Centos7的安装

学习Linux需要一个环境，我们需要创建一个虚拟机，然后在虚拟机上安装一个Centos系统来学习。

虚拟机（Virtual Machine）指通过软件模拟的具有完整硬件系统功能的、运行在一个完全隔离环境中的完整计算机系统，在实体计算机中能够完成的工作在虚拟机中都能够实现，可以通过虚拟机在我们的电脑上虚拟出很多计算机，然后在这些计算机上可以安装不同的操作系统。

一台电脑本身是可以装多个操作系统的， 但是做不到多个操作系统切换自如， 所以我们需要一款软件帮助我们达到这个目的，推荐的软件为 VMware， VMware 可以使用户在一台计算机上同时运行多个操作系统，还可以像 Windows 应用程序一样来回切换。 用户可以如同操作真实安装的系统一样操作虚拟机系统， 甚至可以在一台计算机上将几个虚拟机系统连接为一个局域网或者连接到互联网。

## 3.1、VMware的安装

### （1）下载VMware

```
链接：https://pan.baidu.com/s/1_N830MgeYLt_VF4bTFWj2w
提取码：yyds
```

| 文件名 | 修改时间 | 类型 | 大小 |
| --- | --- | --- | --- |
| CentOS-7-x86_64-DVD-2009.iso | 2022-05-15 11:15 | iso文件 | 4.39GB |
| VMware-workstation-full-16.2.2-19200509.exe | 2022-05-15 11:15 | exe文件 | 615.42MB |
| Xshell-7.0.0090.exe | 2022-05-15 11:15 | exe文件 | 44.64MB |

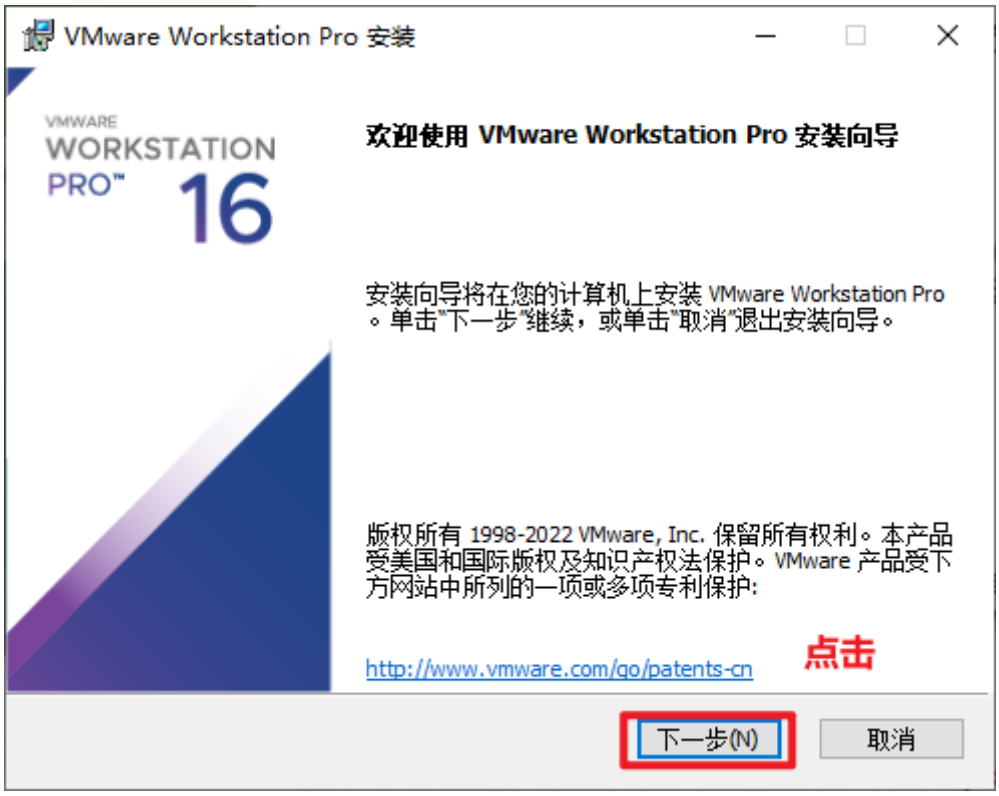
### （2）双击VMware安装文件

| 名称 | 修改日期 | 类型 | 大小 |
| --- | --- | --- | --- |
| CentOS-7-x86_64-DVD-2009.iso | 2022/4/1 10:46 | 光盘映像文件 | 4,601,856... |
| VMware-workstation-full-16.2.2-19200509.exe | 2022/4/1 10:35 | 应用程序 | 630,195 KB |
| Xshell-7.0.0090.exe | 2022/4/1 10:35 | 应用程序 | 45,714 KB |

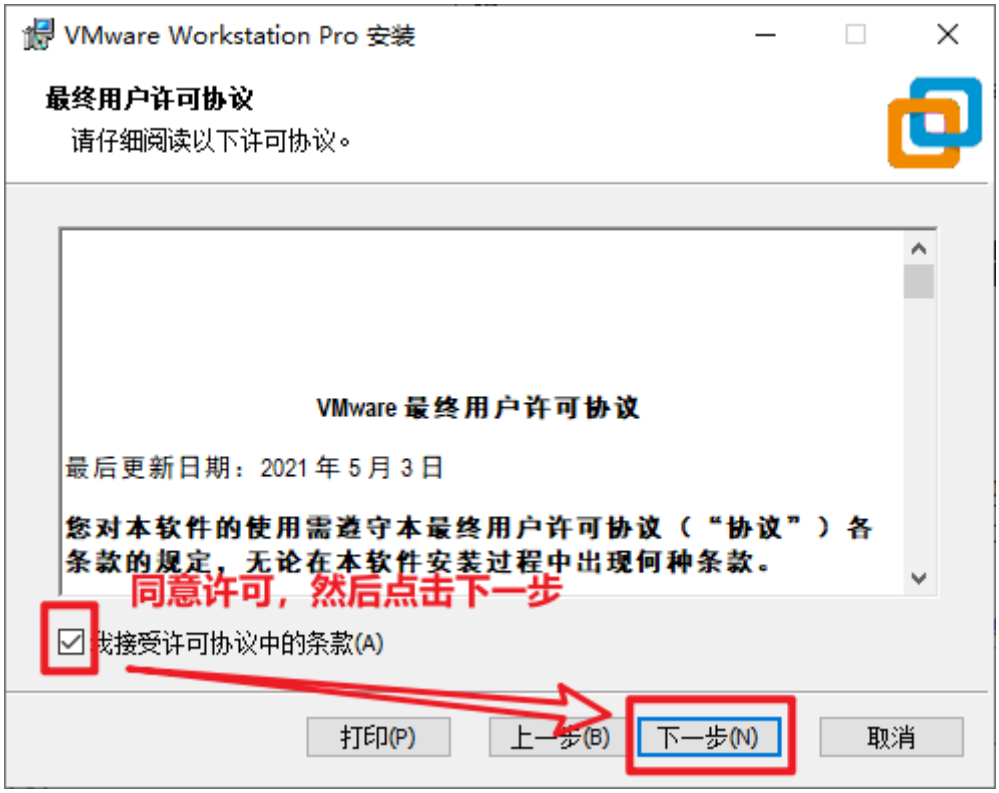
双击即可安装



## (3) 欢迎界面




## (4) 同意许可证

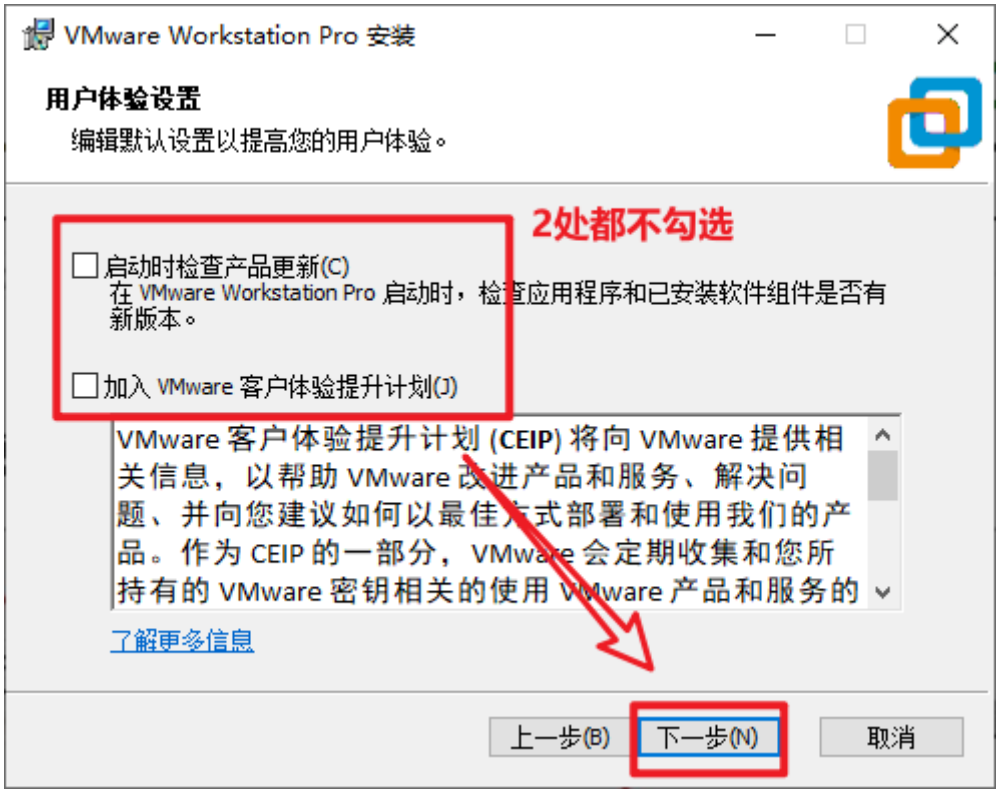



## (5) 选择安装路径



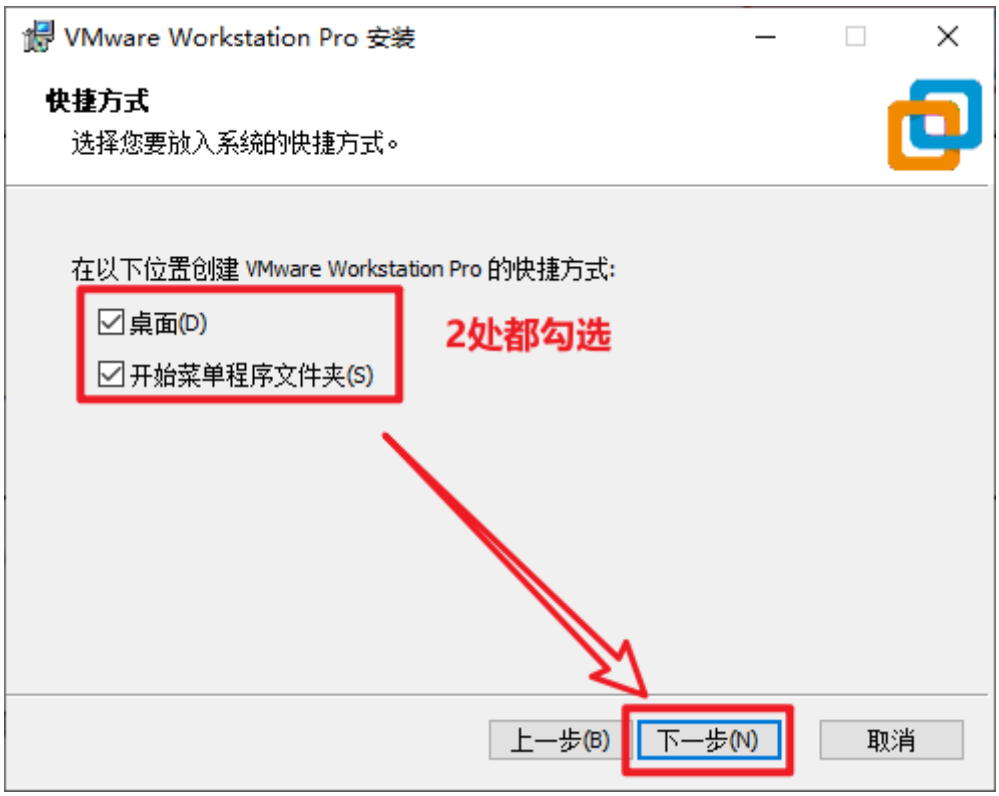

## （6）用户体验计划

每次开启虚拟机时， 会提醒让你更新版本， 建议两个都不勾。

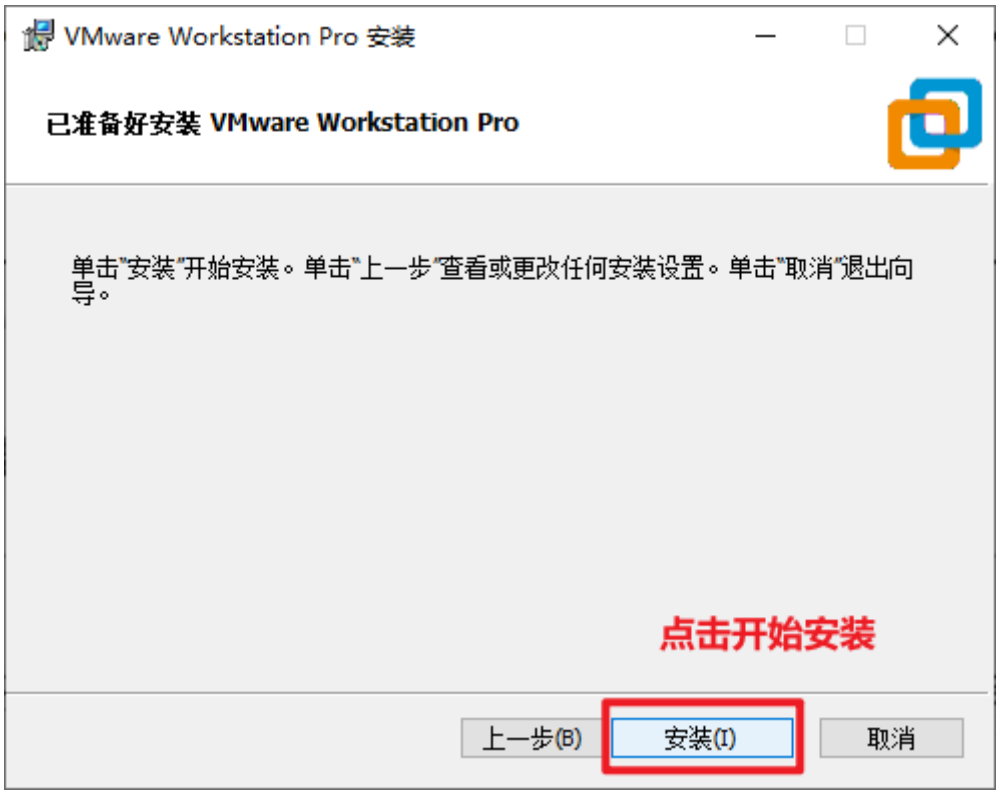



## （7）快捷方式

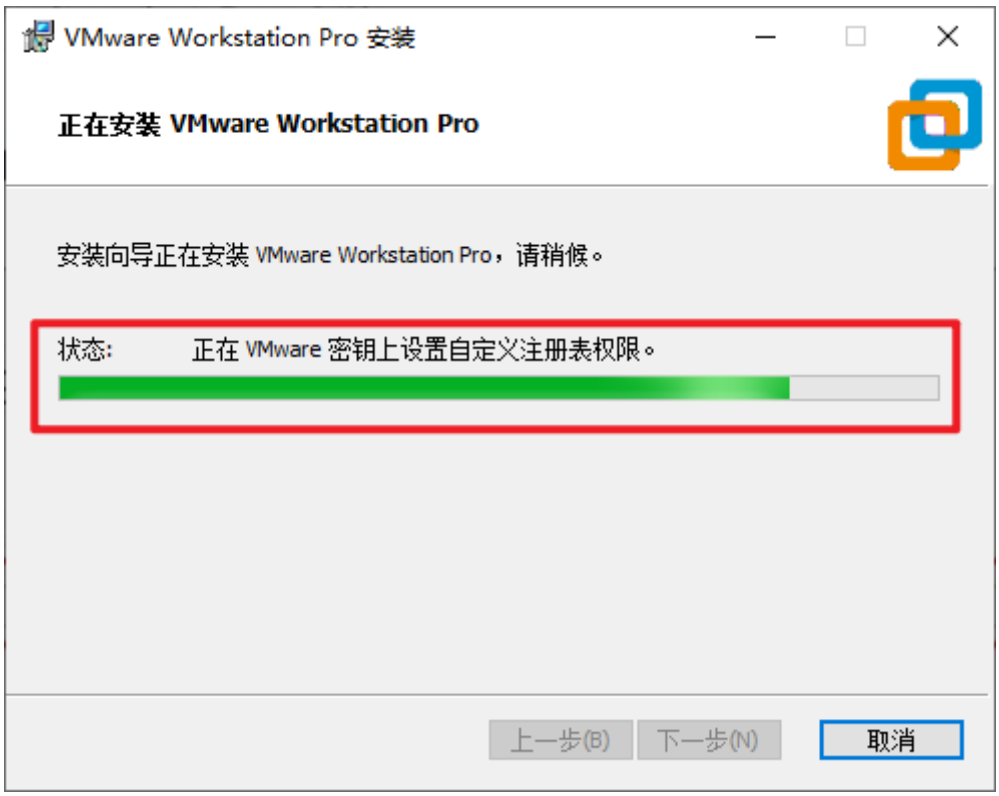
建议勾选， 安装后使用起来会比较方便。




## (8) 开始安装

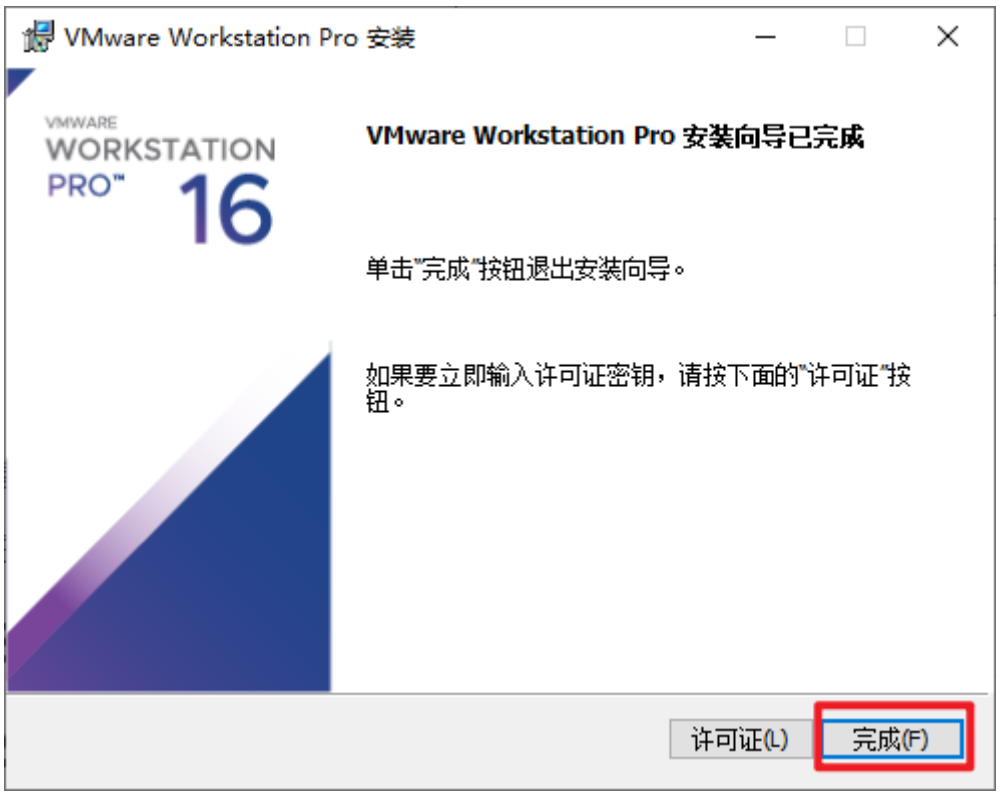



## (9) 等待安装完成



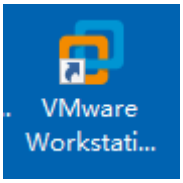

## （10）输入许可证

> 输入许可证， 激活 VMware。
>
> 注： 如果点击的是完成也没关系， 后面还是输入许可证的。




## （11）VMware安装完成

> 桌面上的快捷方式， 双击它就可以使用啦。




- 领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】
- 更多学习资料

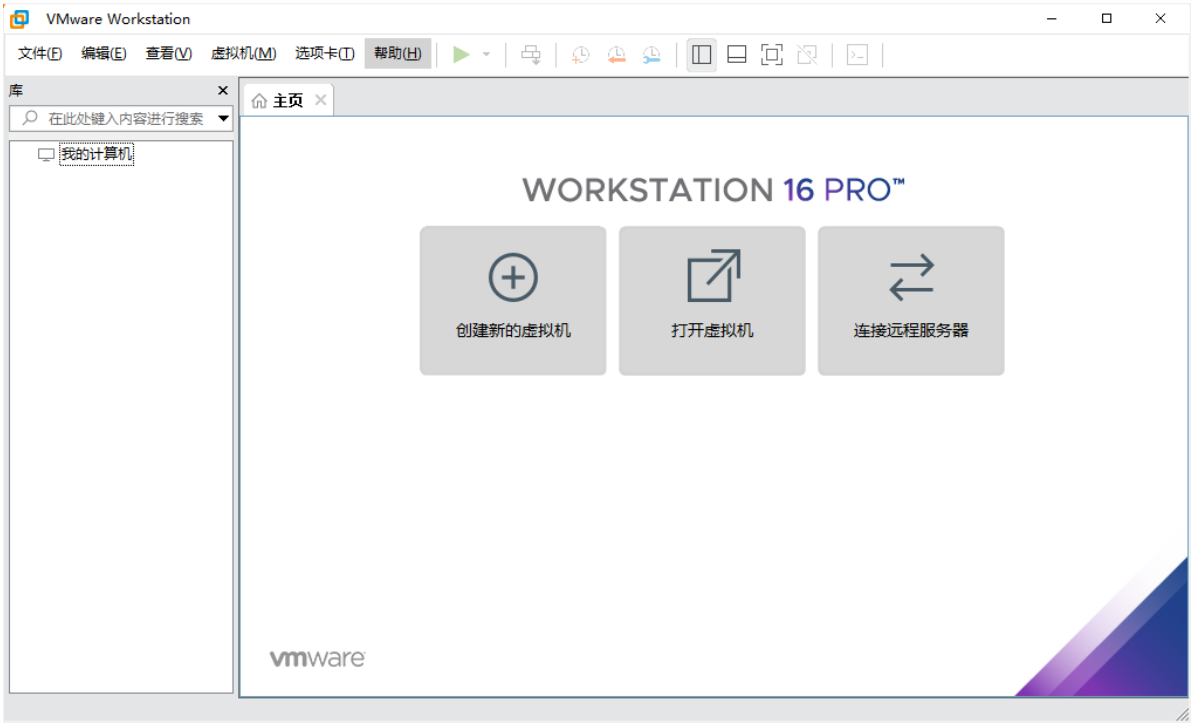
# 3.2、Centos7的安装

系统的安装得分两个步骤， 第一步得配置一台电脑， 选配 cpu， 内存， 磁盘， 网卡等硬件。 第二步才是安装系统。

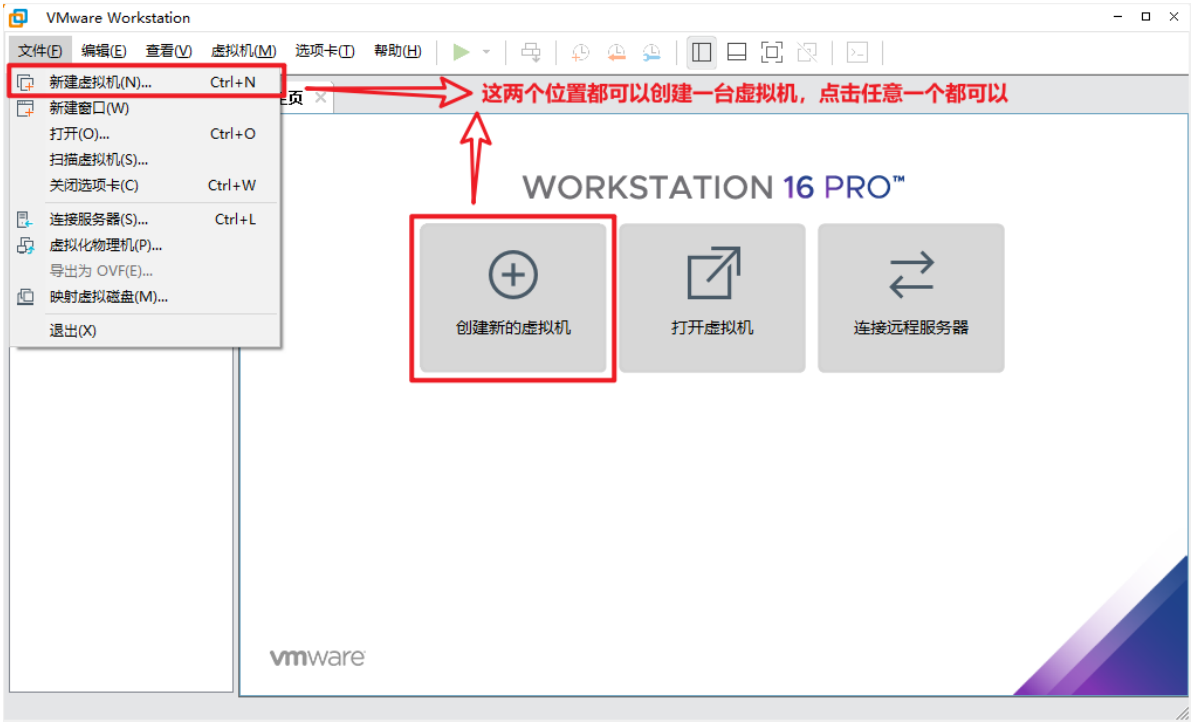
## 3.2.1、配置电脑

### （1）进入VMware

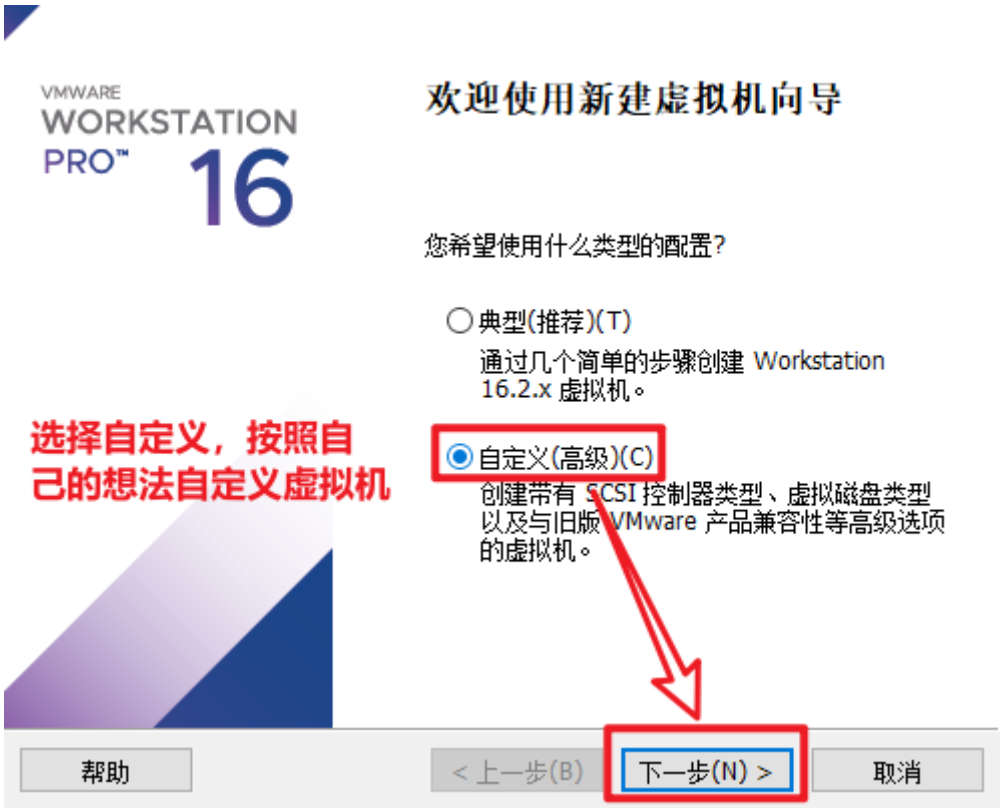
> 双击桌面上已安装好的VMware图标，看到如下界面



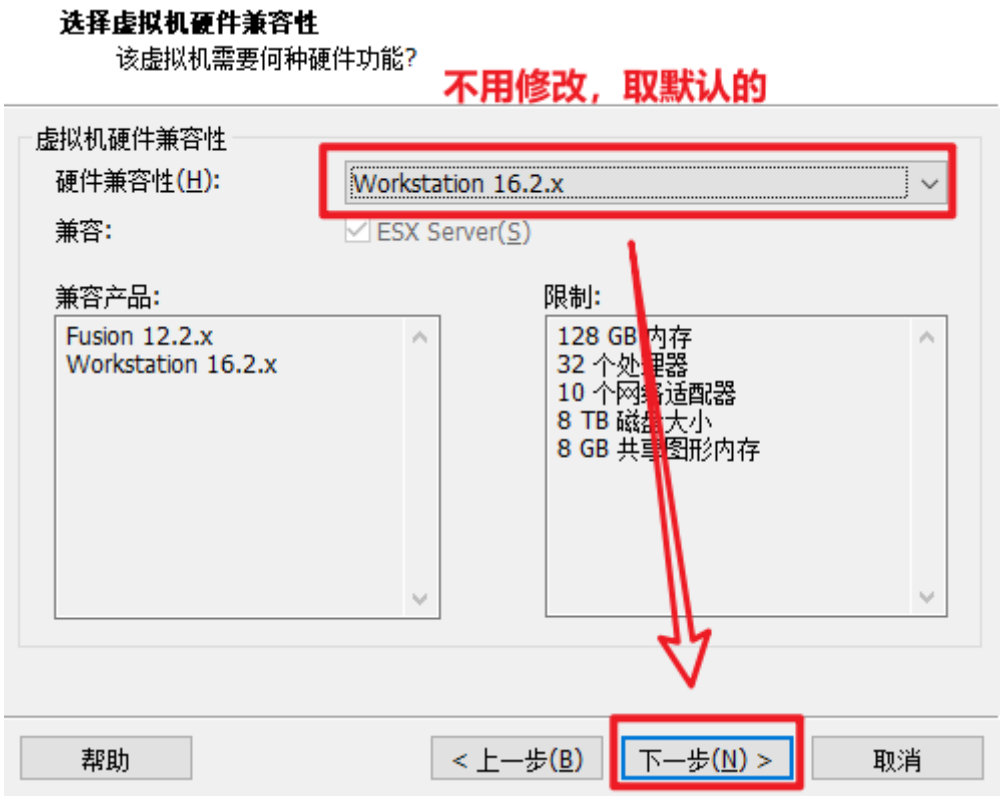

### （2）新建一个虚拟机




## (3) 自定义新的虚拟机

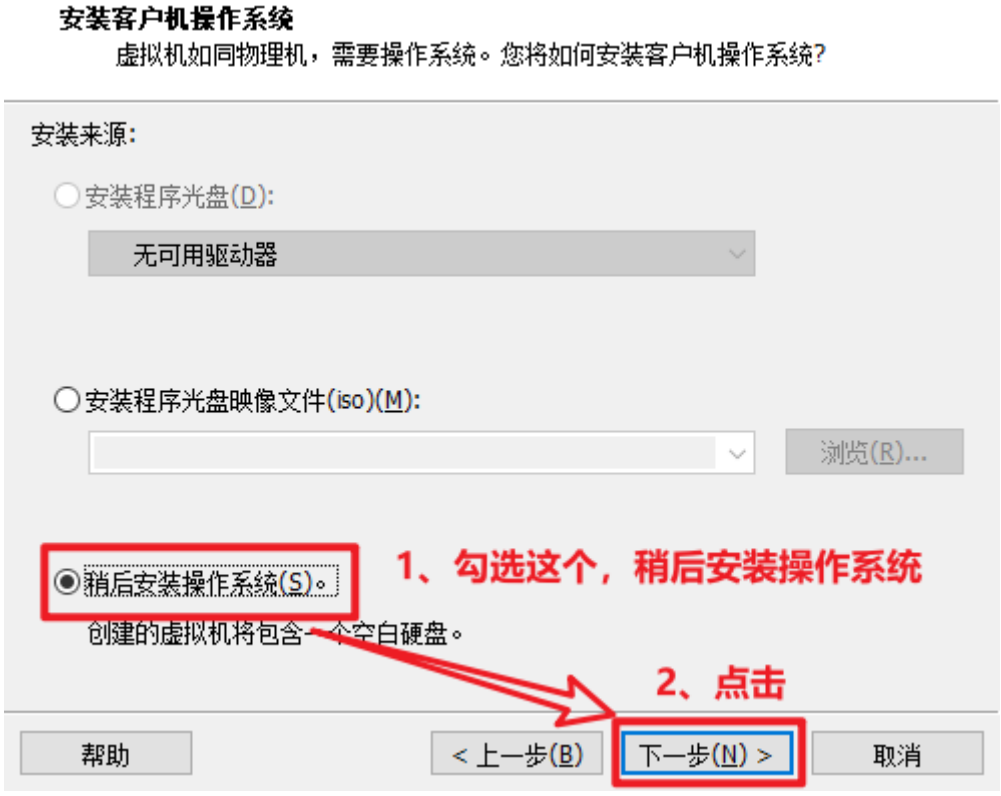



## (4) 解决虚拟机的兼容性



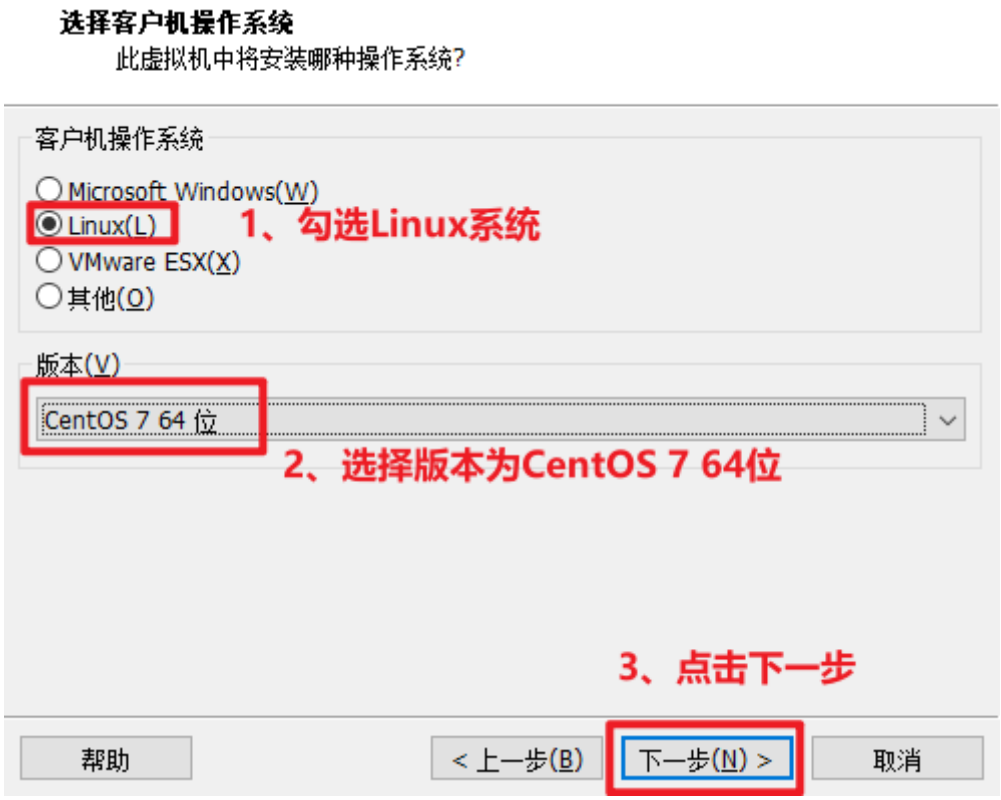

### (5) 选择当前虚拟机的操作系统

> 我们先配置电脑，再安装系统。



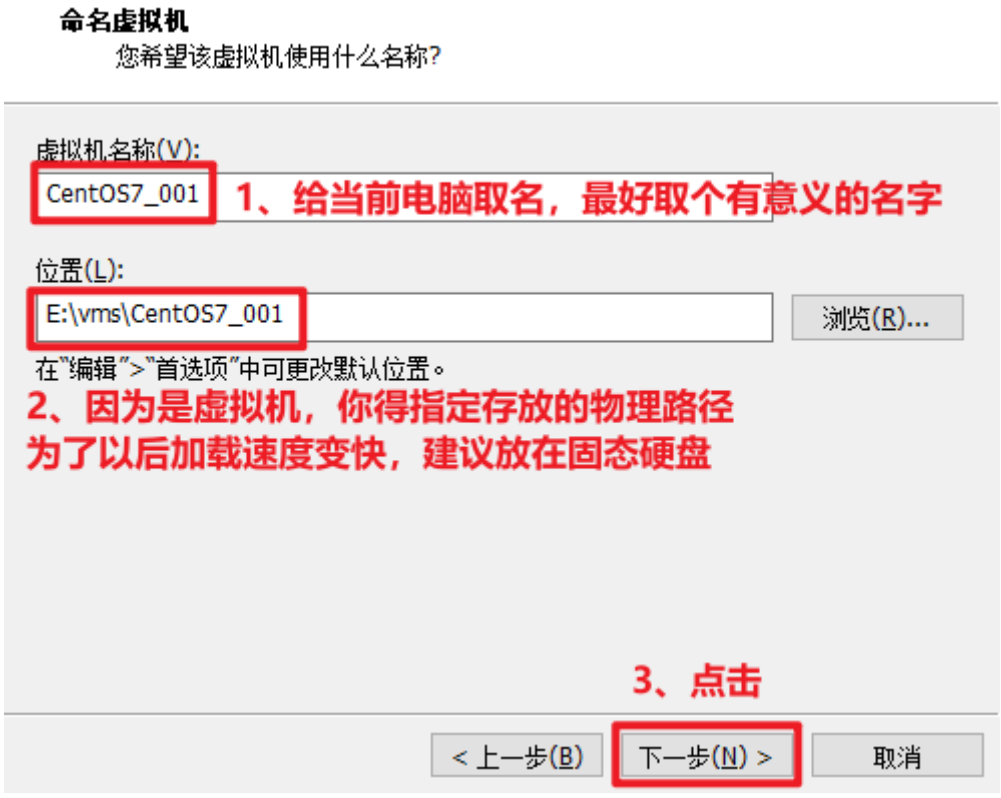

### (6) 选择虚拟机将来需要安装的系统

> 因为不同的操作系统需要解决不同的兼容性问题， 所以需要选择将来用什么系统， 提前做适配。



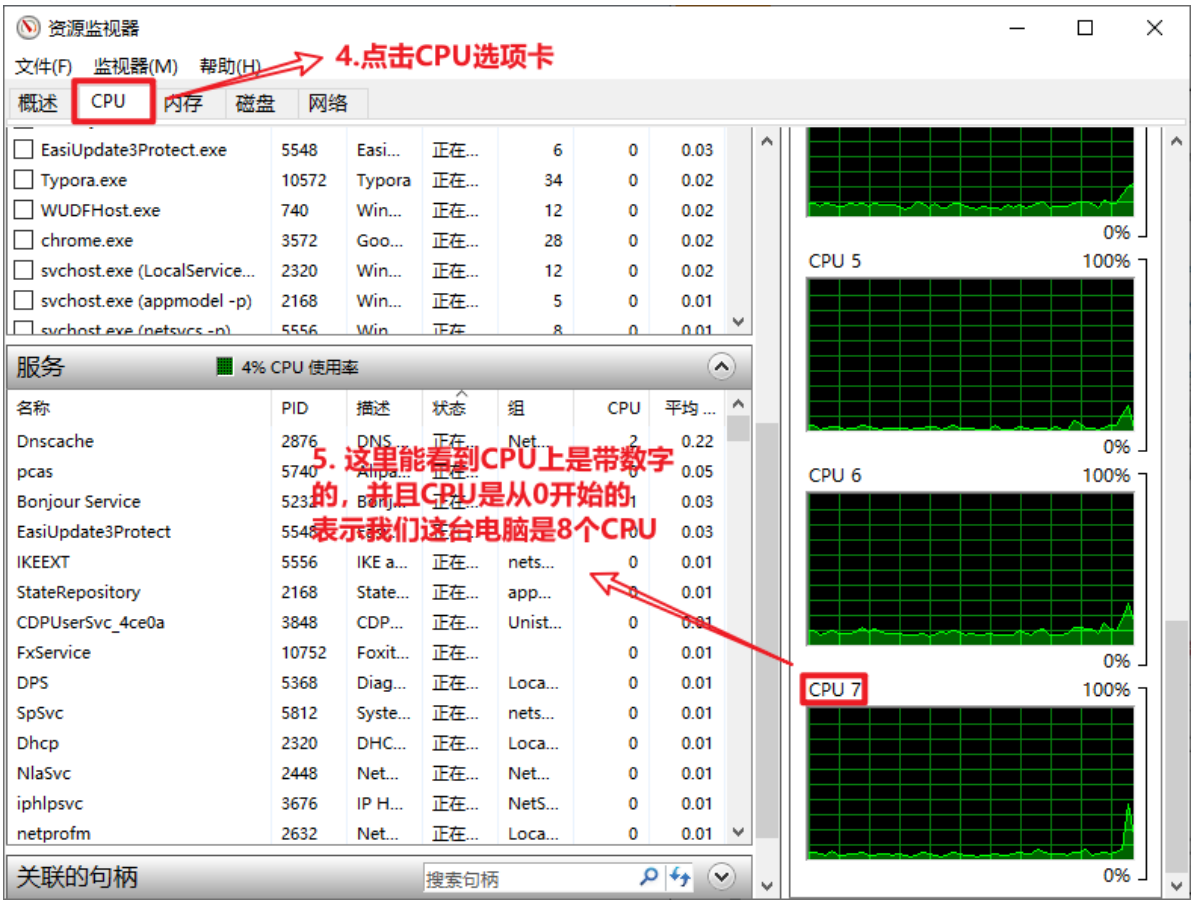

### (7) 配置电脑

> 给自己配置电脑取个名字， 并存放在物理机的位置在哪。



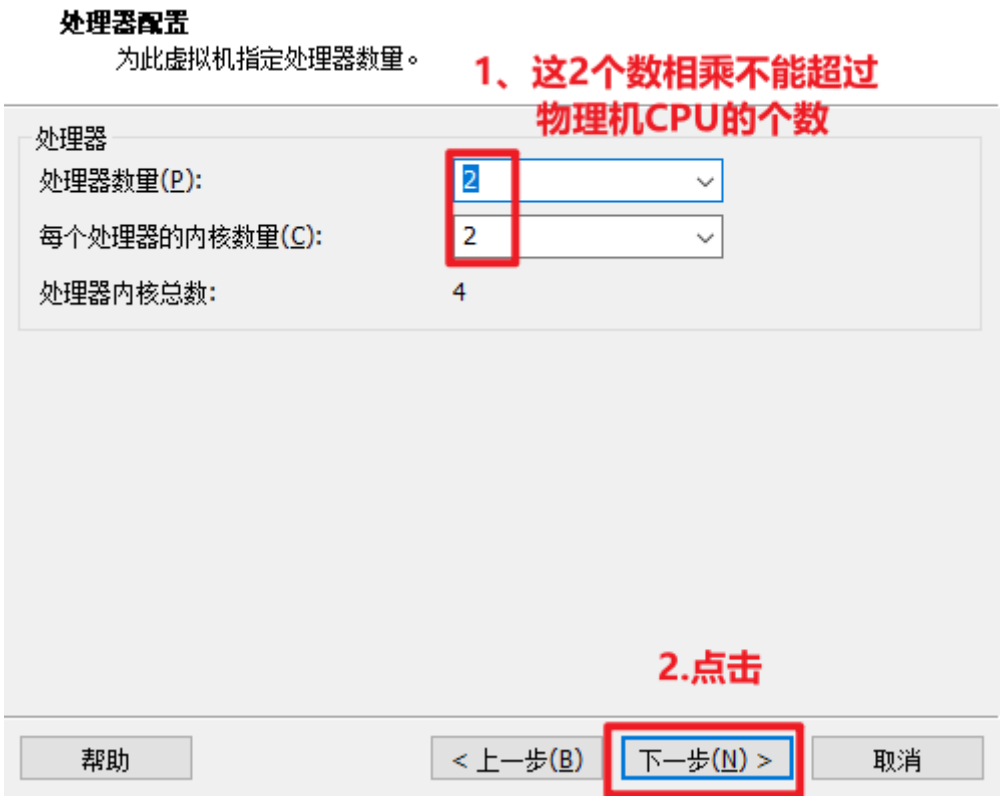

### (8) 查看本机CPU个数




## （9）选择CPU的个数

> 有个原则就是选满（跟物理机的 CPU 个数相同， 但是不能超过）。

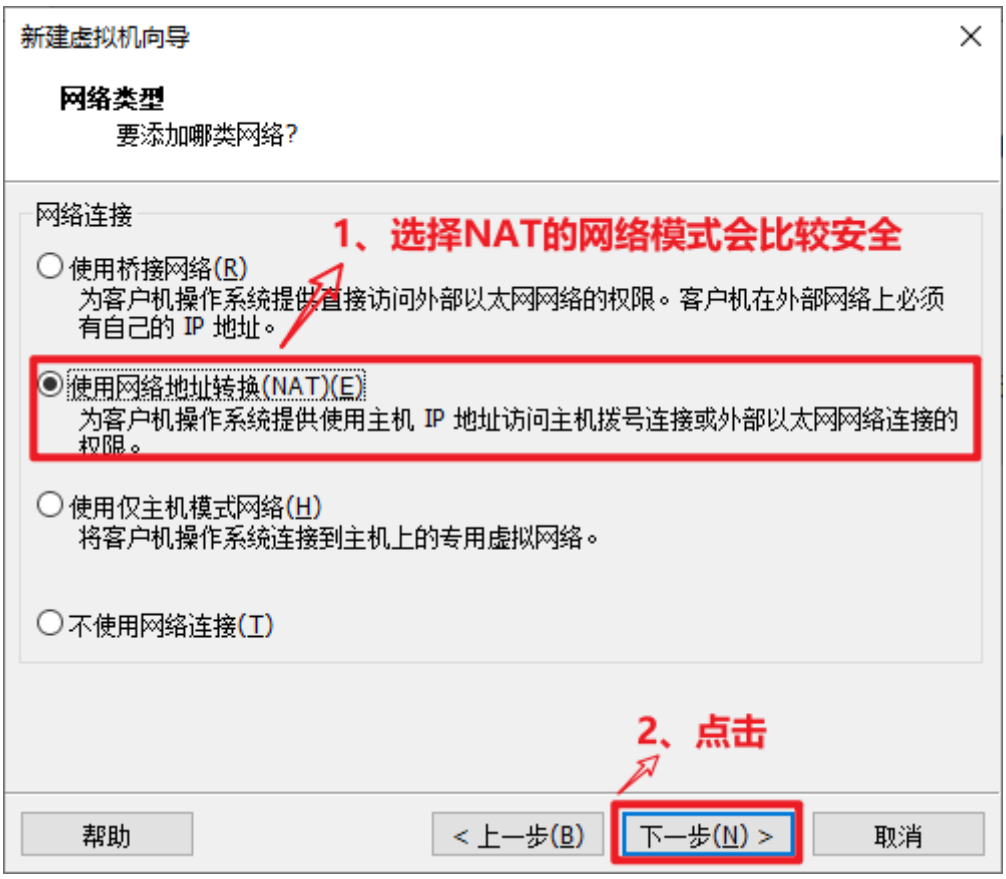



## （10）设置虚拟机的内存

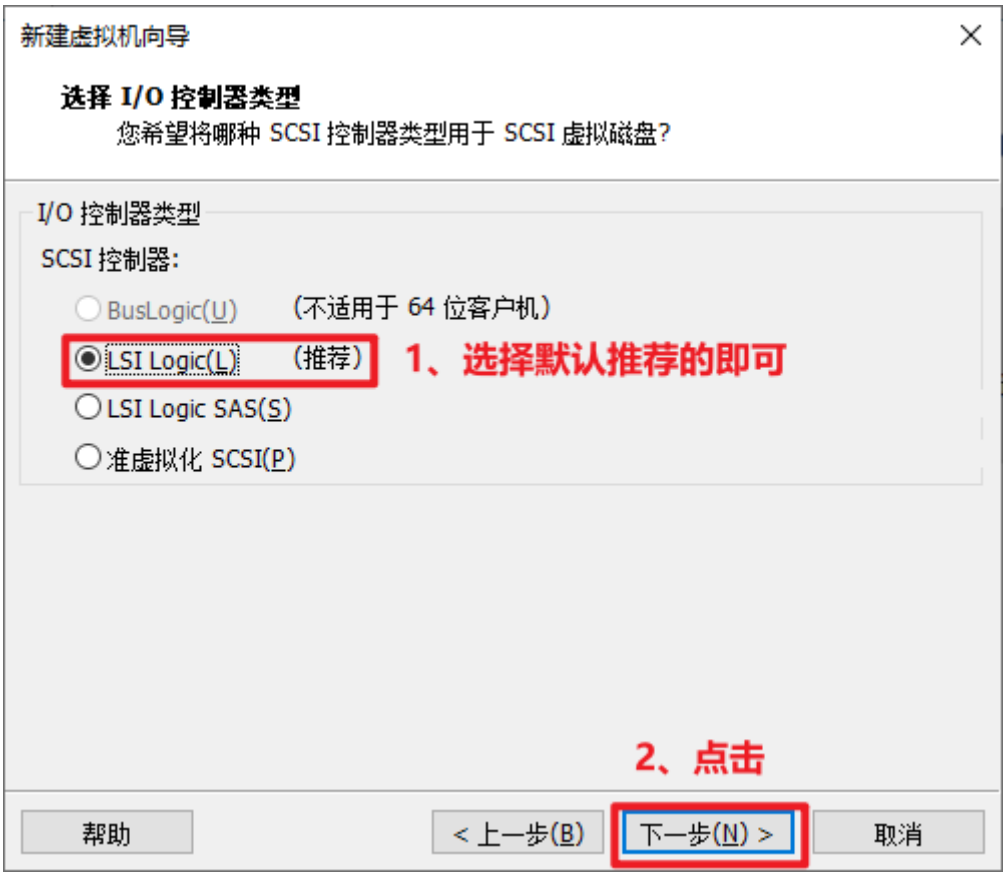
> 内存大小有一定要求， 建议 4g， 不能给太多， 后期会有多台虚拟机同时启动。




## （11）选择虚拟机上网方式



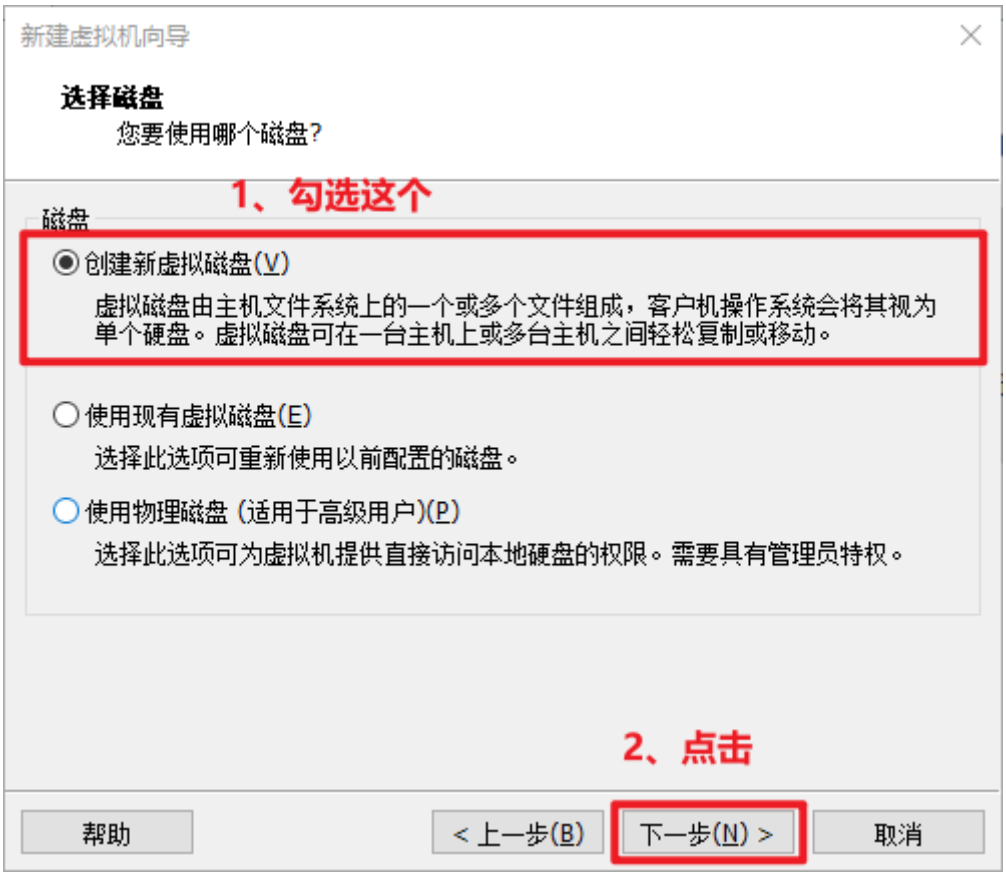

**（12）选择对应的文件系统的IO方式**



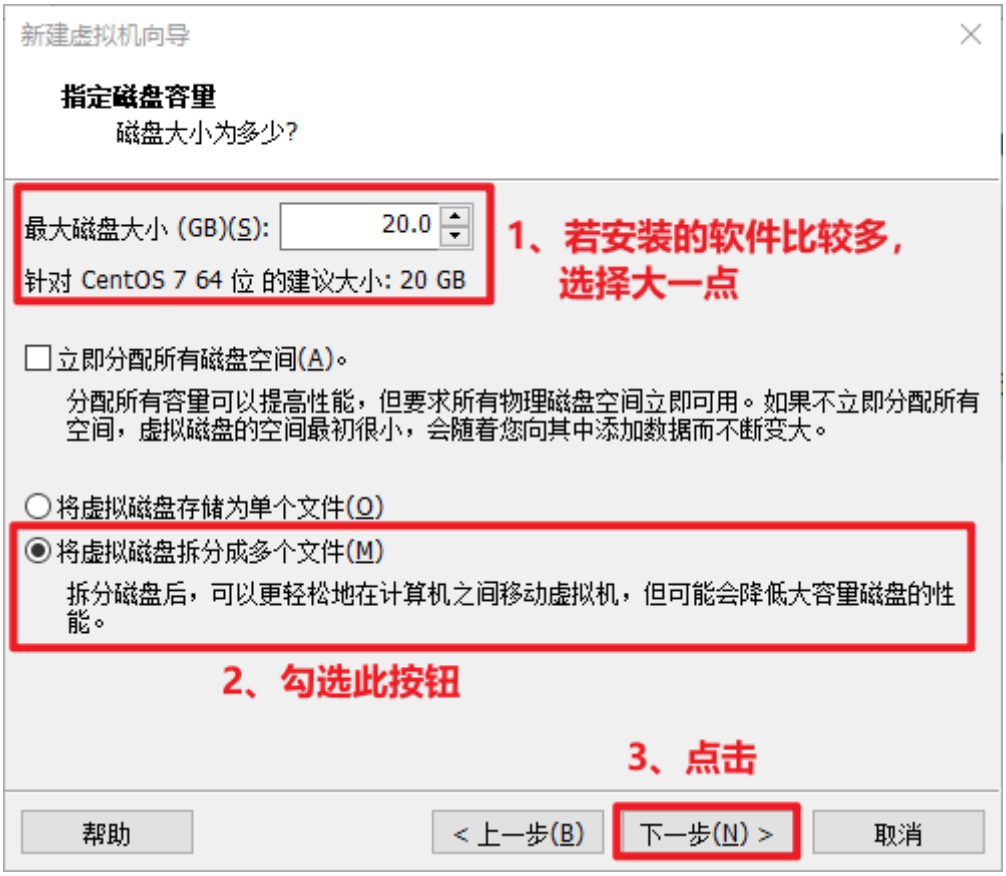
## (13) 选择磁盘类型

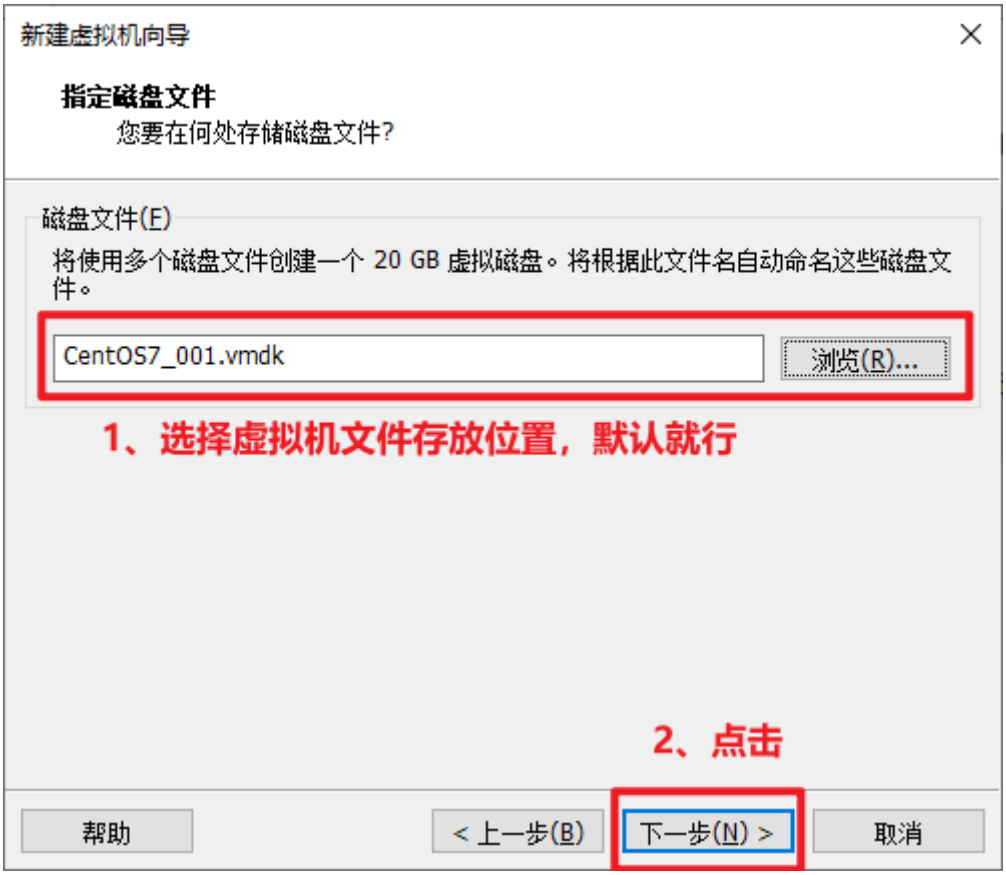



## (14) 选择磁盘的种类

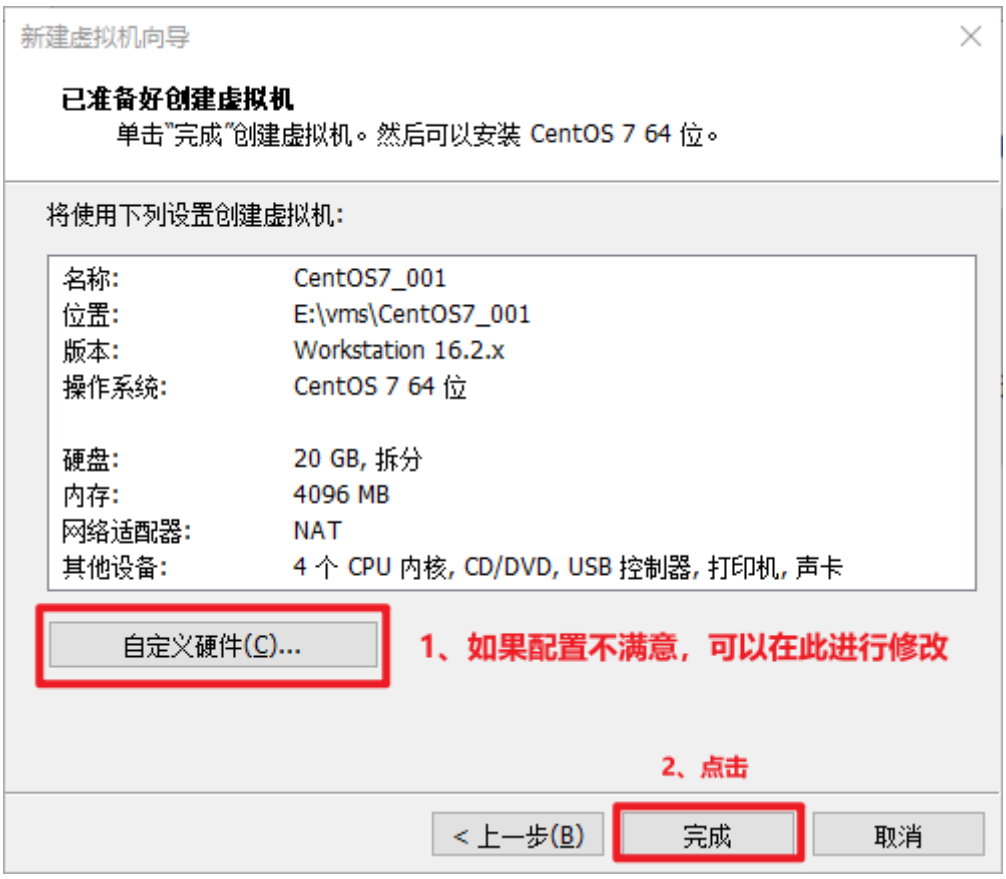


## （15）选择虚拟机磁盘大小



## （16）电脑配置完成







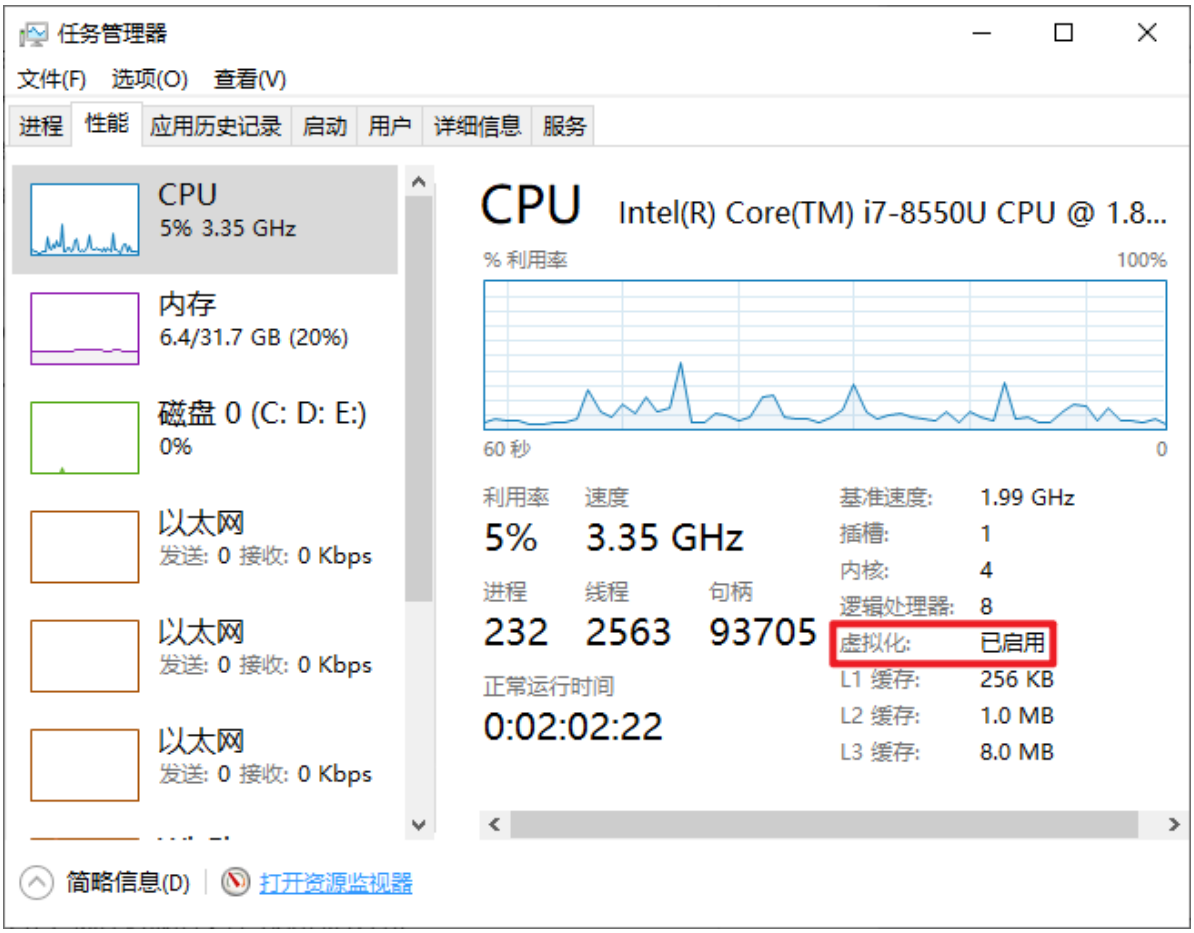

- 领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】
- 更多学习资料

## 3.2.2、安装系统

### （1）事前检查 bios 的虚拟化设置是否打开

注： 在安装系统之前需要检查自己 bios 的虚拟化设置是否打开（大部分的电脑都是打开的， 大家可以先尝试直接安装， 如果出现错误再去调试， 没有出错就不用管了）以下是查看虚拟机 bios 是否开启的方式。

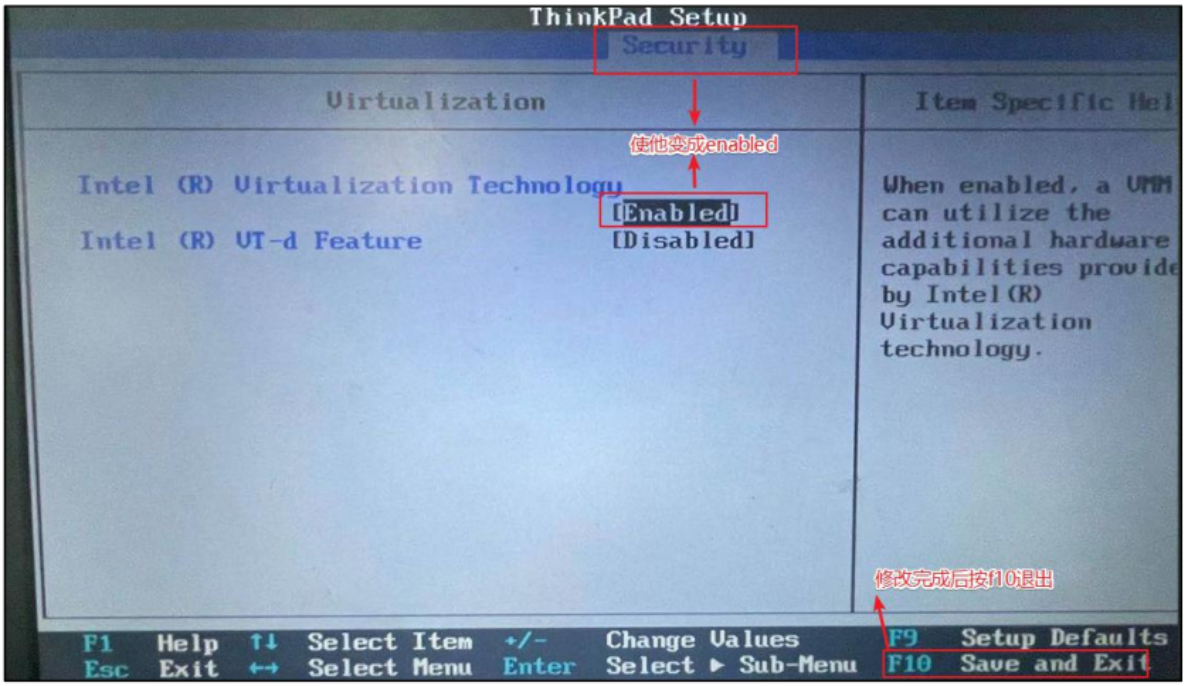
（1）window10



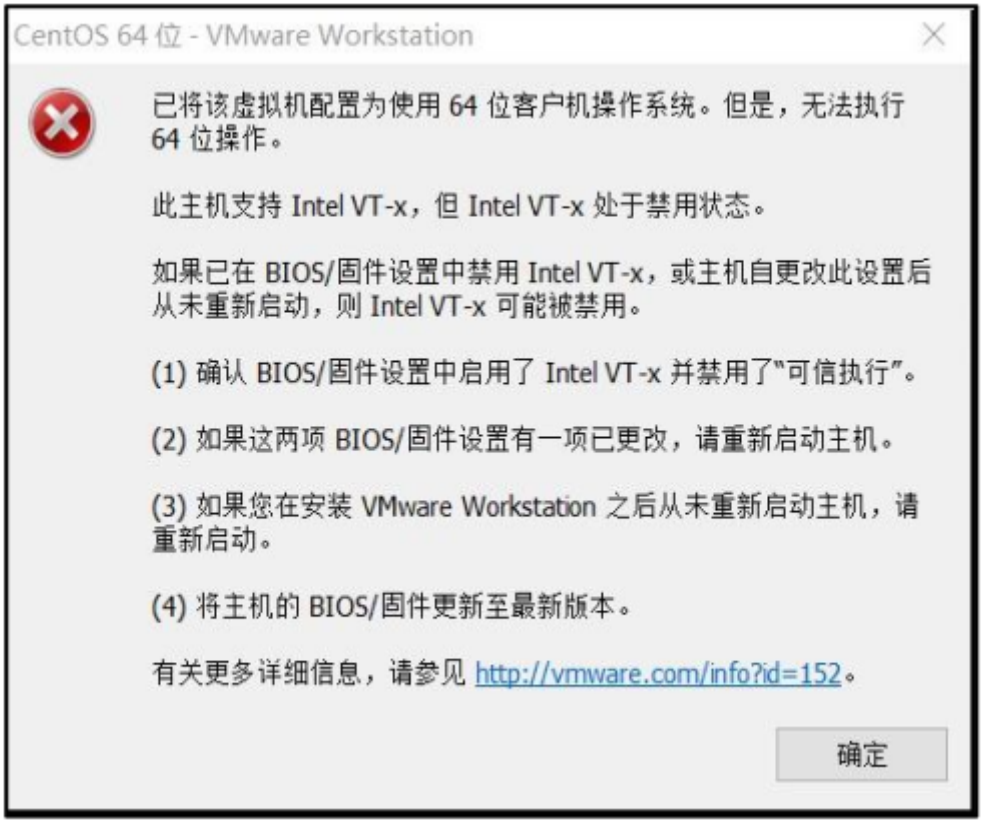

（2） window7 里面看不到（得去 bios 里面看）

（3） 如果发现 bios 虚拟化没有开启怎么办， 重启电脑， 在加载界面时按 f1-f10，或者电脑旁边一个小洞， 具体怎么进入得去查一下（按照自己电脑的型号去查）

（4） 修改虚拟化为开启（thinkpad 为例） 找到 security 里面的 VT 并改成 enabled



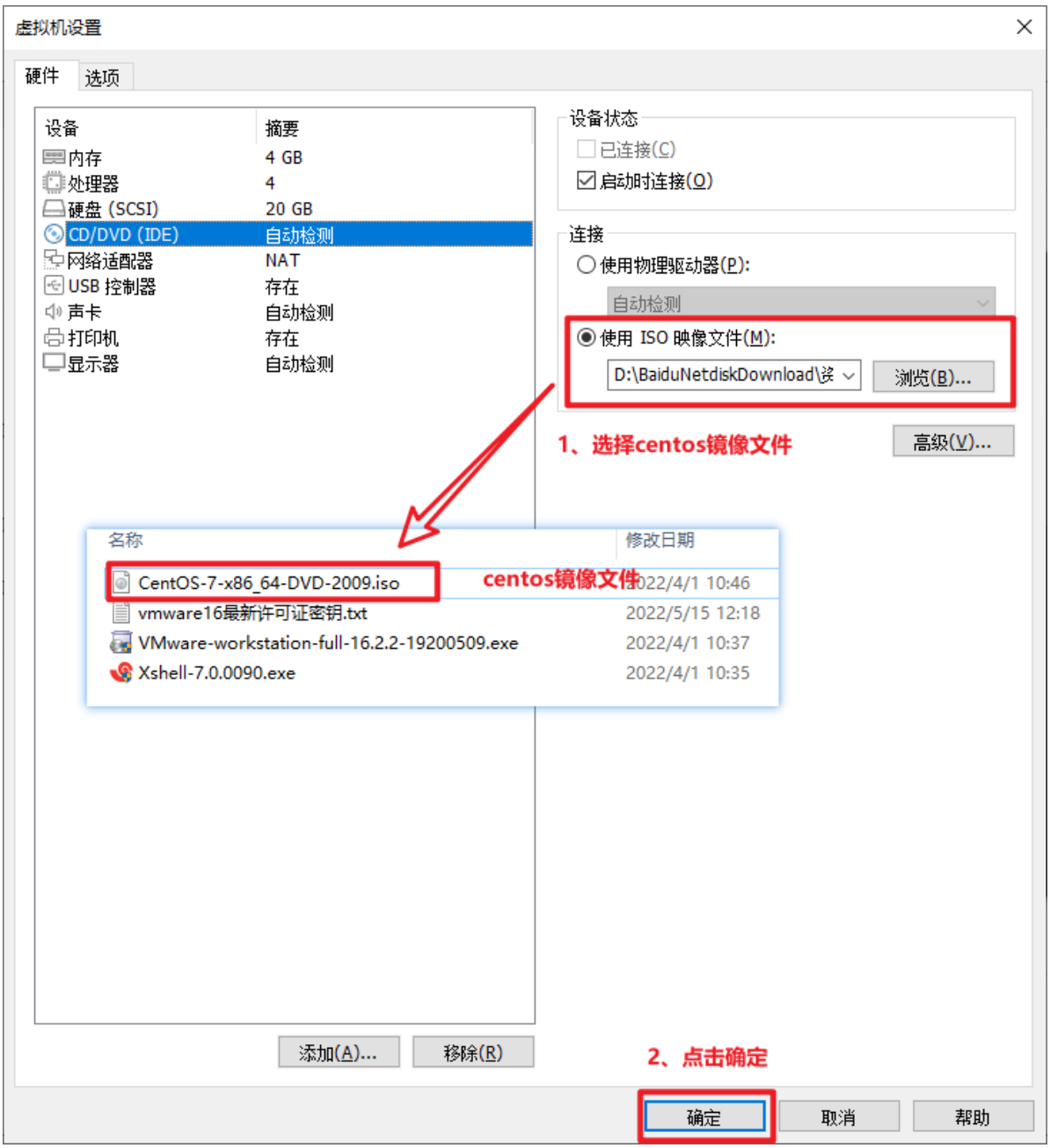

注： 如果虚拟化没有开启报的是以下错误。




**（2）选择CD/DVD的方式安装系统**


CentOS7_001 - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
库
在此处键入内容进行搜索
我的计算机
CentOS7_001
1、选择机器
主页
CentOS7_001
CentOS7_001
开启此虚拟机
编辑虚拟机设置
设备
内存 4 GB
处理器 4
硬盘 (SCSI) 20 GB
CD/DVD (IDE) 自动检测
网络适配器 NAT
USB 控制器 存在
声卡 自动检测
打印机 存在
显示器 自动检测
2、点击这里
描述
在此处键入对该虚拟机的描述。

**（3）指定镜像文件**

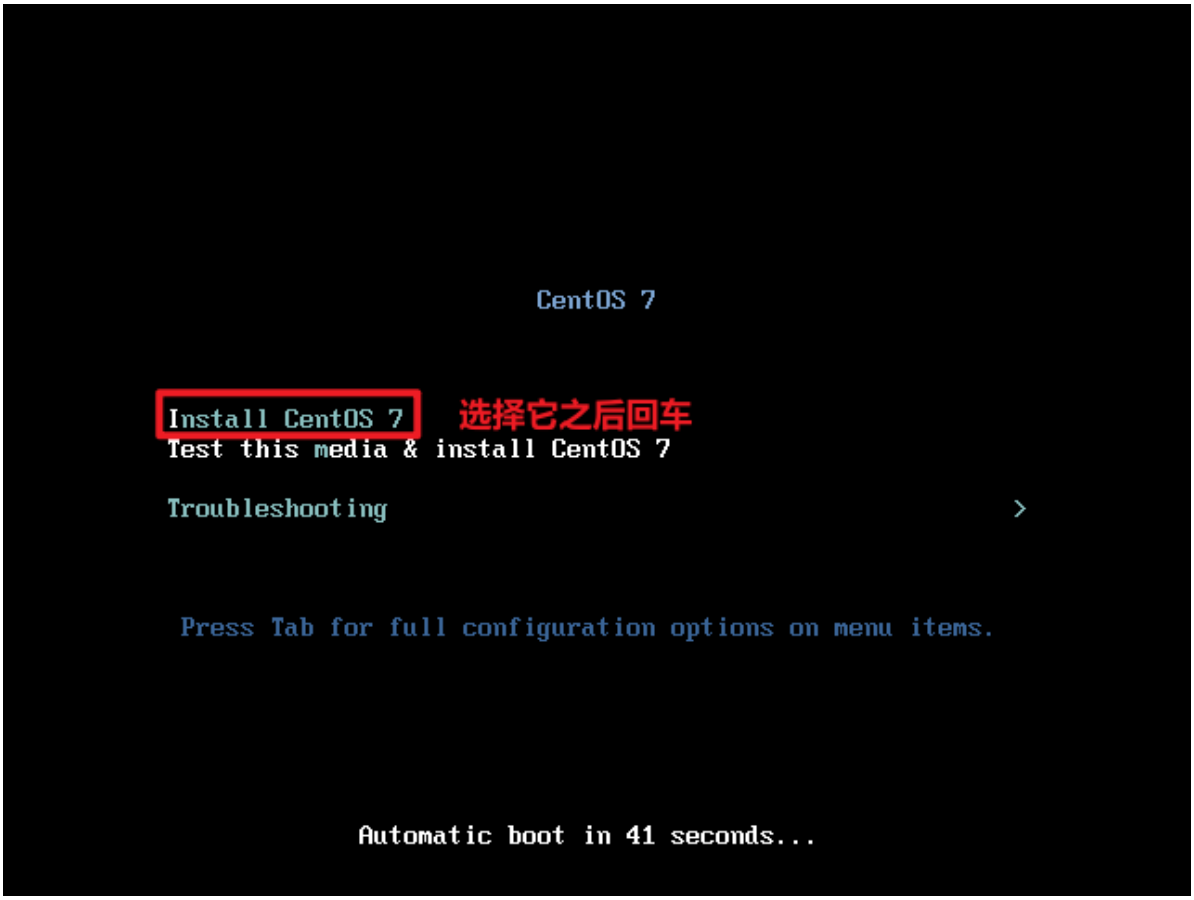



**（4）开启虚拟机开始安装系统**




### (5) 系统安装引导界面

> 你得需要将鼠标点击进入界面中， 但是鼠标会消失， 你此刻得用键盘的上下键来控制选项， 图标变白了表示当前选中的是哪个选项， 然后敲回车， 表示执行所选选项。
>
> **注： 如果这个时候你需要鼠标可以使用 ctrl+alt 一起按呼出鼠标。**




耐心等待它的安装，会跳转到下面的界面。

### （6）选择语言



## （7）设置日期和时间







## （8）软件选择，选择GNOME桌面


安装信息摘要
CENTOS 7 安装
cn
帮助！ (F1)
CentOS
本地化
日期和时间(T)
亚洲/上海 时区
键盘(K)
汉语
语言支持(L)
简体中文 (中国)
软件
1、点击这里
安装源(I)
本地介质
软件选择(S)
GNOME 桌面
系统
安装位置(D)
KDUMP
退出(Q)
开始安装(B)
在点击"开始安装"按钮前我们并不会操作您的磁盘。
请先完成带有此图标标记的内容再进行下一步。


软件选择
完成(D)
2、点击完成
CENTOS 7 安装
cn
帮助！
基本环境
最小安装
基本功能。
计算节点
执行计算及处理的安装。
基础设施服务器
用于操作网络基础设施服务的服务器。
文件及打印服务器
用于企业的文件、打印及存储服务器。
基本网页服务器
提供静态及动态互联网内容的服务器。
虚拟化主机
最小虚拟化主机。
带 GUI 的服务器
带有用于操作网络基础设施服务 GUI 的服务器。
GNOME 桌面
GNOME 是一个非常直观且用户友好的桌面环境。
KDE Plasma Workspaces
KDE Plasma Workspaces 是一个高度可配置图形用户界面，其中包括面板、桌面、系统图标以及桌面向导和很多功能强大的 KDE 应用程序。
开发及生成工作站
1、勾选GNOME桌面，其他不用动
已选环境的附加选项
备份客户端
用来连接到备份服务器并进行备份的客户端工具。
GNOME 应用程序
一组经常使用的 GNOME 应用程序。
互联网应用程序
电子邮件、聊天和视频会议软件。
传统 X Windows 系统的兼容性
用于从继承 X Windows 环境中迁移或者可用于该环境的兼容程序。
办公套件和生产率
全套办公套件以及其它产品工具。
智能卡支持
支持使用智能卡验证。
兼容性程序库
用于在 CentOS Linux 的之前版本中构建的应用程序的兼容程序库。
开发工具
基本开发环境。
安全性工具
用于完整性和可信验证的安全性工具。
系统管理工具

## （9）配置磁盘分区


安装信息摘要
CENTOS 7 安装
cn
帮助！ (F1)
CentOS
语言支持(L)
简体中文 (中国)
软件
安装源(I)
本地介质
软件选择(S)
GNOME 桌面
系统
1、点击这里
安装位置(D)
已选择自动分区
KDUMP
已启用 Kdump
网络和主机名(N)
未连接
SECURITY POLICY
No content found
退出(Q)
开始安装(B)
在点击"开始安装"按钮前我们并不会操作您的磁盘。
请先完成带有此图标标记的内容再进行下一步。


安装目标位置
完成(D)
2、点击完成
CENTOS 7 安装
cn
帮助！
选择要在其中安装系统的设备。点击主菜单中的"开始安装"按钮前不会对该设备进行任何操作。
本地标准磁盘
20 GiB
VMware, VMware Virtual S
sda / 20 GiB 空闲
不会对未在此处选择的磁盘进行任何操作。
专用磁盘 & 网络磁盘
添加磁盘(A)...
不会对未在此处选择的磁盘进行任何操作。
其它存储选项
分区
自动配置分区(U)。
我要配置分区(I)。
1、选中这个，按照自己需求配置分区
我想让额外空间可用(M)。
加密
加密我的数据(E)。
然后设置密码。
完整磁盘摘要以及引导程序(F)...
已选择 1 个磁盘；容量 20 GiB；20 GiB 空闲
刷新(R)

## （10）手动添加分区




## （11）添加 boot 区（1G容量）






## （12）添加swap交换分区






### （13）剩下的磁盘空间给根目录

> 根目录作为存储使用， 将剩下的空间都给他（20 - 4 - 1） = 15




手动分区
完成(D)
CENTOS 7 安装
cn
帮助！
新 CentOS 7 安装
系统
/boot
sda1
1024 MiB
swap
sda2
sda2
挂载点(P)：
设备：
VMware, VMware Virtual S (sda)
修改...(M)
添加新挂载点
在下方创建挂载点后
有更多自定义选项可用。
挂载点(P)：
/
1、这里选择/
期望容量(D)：
15G
2、大小为剩下的空间，即15G
取消(C)
添加挂载点(A)
3、点击按钮
标签(L)：
名称(N)：
sda2
可用空间
15 GiB
总空间
20 GiB
已选择 1 个存储设备(S)
全部重设(R)


手动分区
完成(D)
CENTOS 7 安装
cn
帮助！
新 CentOS 7 安装
系统
/boot
sda1
1024 MiB
/
sda3
15 GiB
swap
sda2
4096 MiB
sda3
挂载点(P)：
/
设备：
VMware, VMware Virtual S (sda)
期望容量(D)：
15 GiB
修改...(M)
设备类型(T)：
标准分区
加密(E)
文件系统(Y)：
xfs
重新格式化(O)
标签(L)：
名称(N)：
sda3
可用空间
992.5 KiB
总空间
20 GiB
已选择 1 个存储设备(S)
全部重设(R)

**（14）完成分区配置**




## （15）关闭 kdump

> 本身虚拟机内存就不够， 他会吃掉一部分内存， 我们尽量省一点




KDUMP
完成(D)
2、点击完成
CENTOS 7 安装
cn
帮助！
Kdump 是一个内核崩溃转储机制。在系统崩溃的时候，kdump 将捕获系统信息，这对于诊断崩溃的原因非常有用。注意，kdump 需要预留一部分系统内存，且这部分内存对于其他用户是不可用的。
启用 kdump（E）
1、不要勾选
为 Kdump 保留的内存：
自动（A）
手动（M）
要保留的内存（MB）：
160
总系统内存（MB）：
3931
可用系统内存（MB）：
3771

## （16）设置网络和主机名


安装信息摘要
CENTOS 7 安装
cn
帮助！（F1）
CentOS
亚洲/上海 时区
汉语
语言支持(L)
简体中文 (中国)
软件
安装源(I)
本地介质
软件选择(S)
GNOME 桌面
系统
安装位置(D)
已选择自定义分区
KDUMP
已禁用 Kdump
网络和主机名(N)
未连接
SECURITY POLICY
No content found
点击这里修改网络和主机名
退出(Q)
开始安装(B)
在点击"开始安装"按钮前我们并不会操作您的磁盘。




（17）开始安装



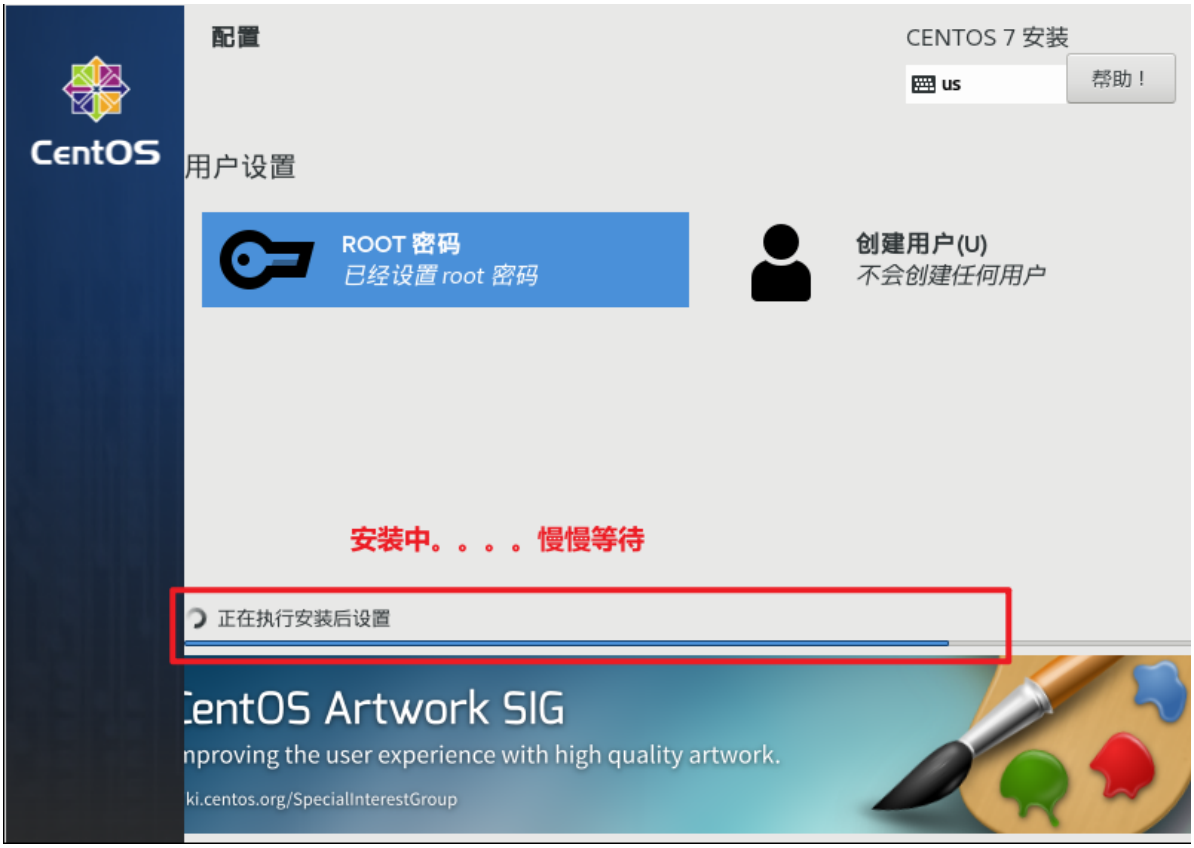

（18）设置 root 用户密码， 一定要设置




密码设置成什么自己决定， 但是不要忘， 建议使用（123456），但是生产环境千万不要这么设置，一定要设置复杂密码。




**（19）安装时间比较长 大概需要 10 几分钟**

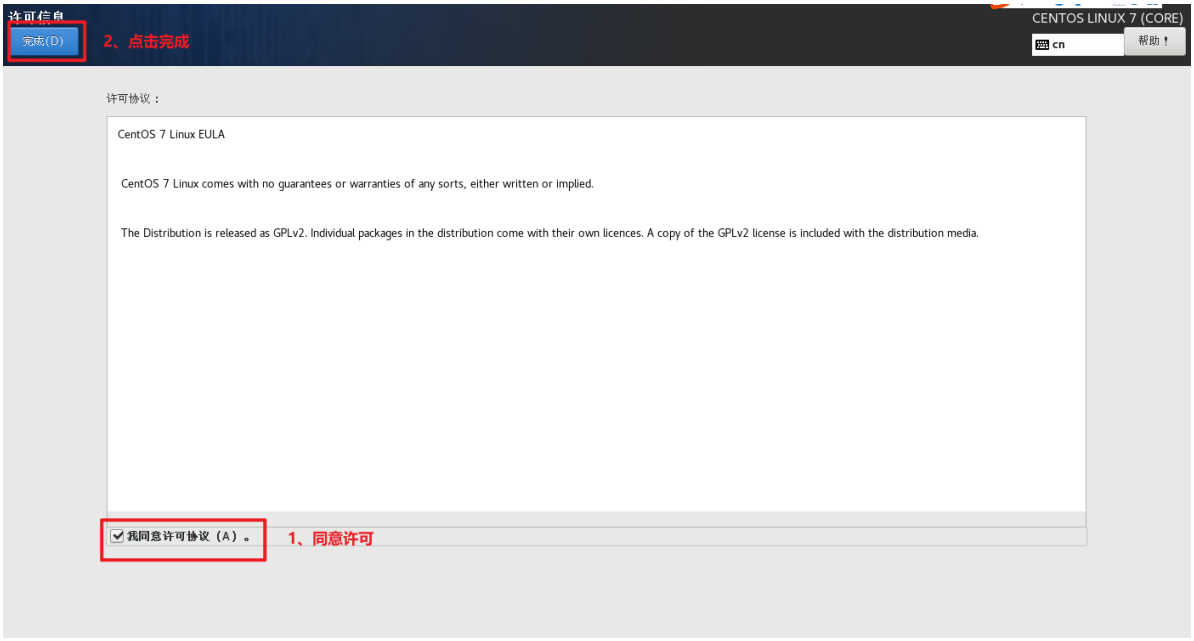



**（20）安装完成，重启虚拟机**

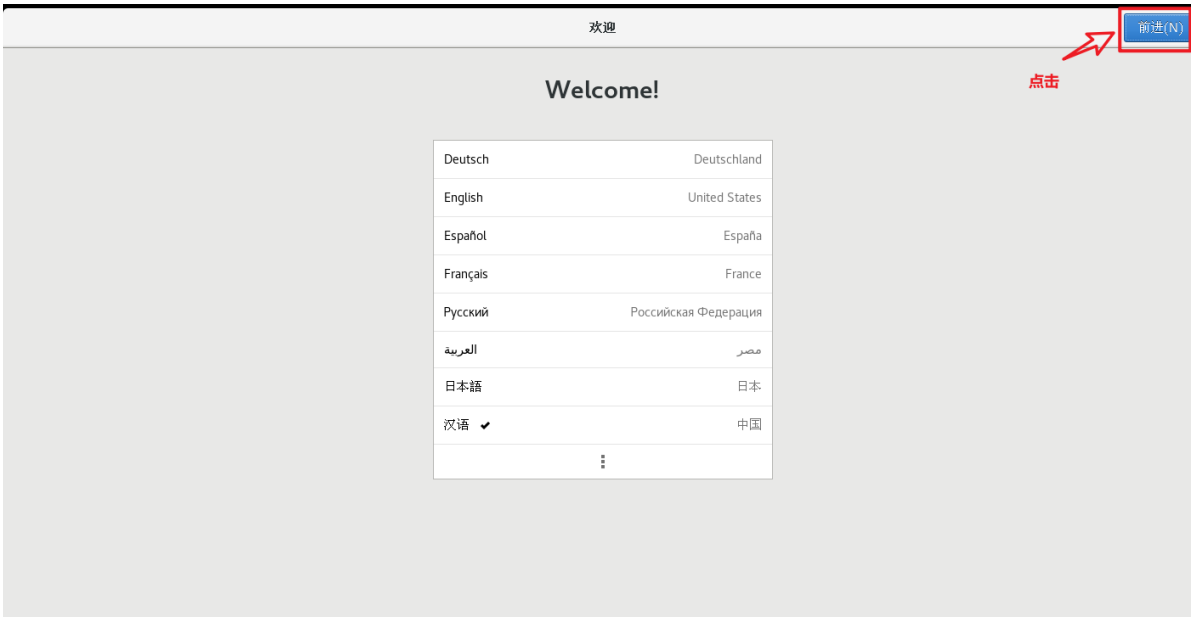



**（21）进入引导界面(以下内容就按照图片走， 就不做过多解释)**


初始设置
CENTOS LINUX 7 (CORE)
cn
帮助！
CentOS
LICENSING
LICENSE INFORMATION
未接受许可证
1、点击进去，同意许可，然后右下角才能亮
系统
网络和主机名(N)
有线（ens33）已连接
用户设置
创建用户(U)
不会创建任何用户
退出（Q）
完成配置（F）
请先完成带有此图标标记的内容再进行下一步。


许可信息
完成(D)
2、点击完成
CENTOS LINUX 7 (CORE)
cn
帮助！
许可协议：
CentOS 7 Linux EULA
CentOS 7 Linux comes with no guarantees or warranties of any sorts, either written or implied.
The Distribution is released as GPLv2. Individual packages in the distribution come with their own licences. A copy of the GPLv2 license is included with the distribution media.
我同意许可协议（A）。
1、同意许可

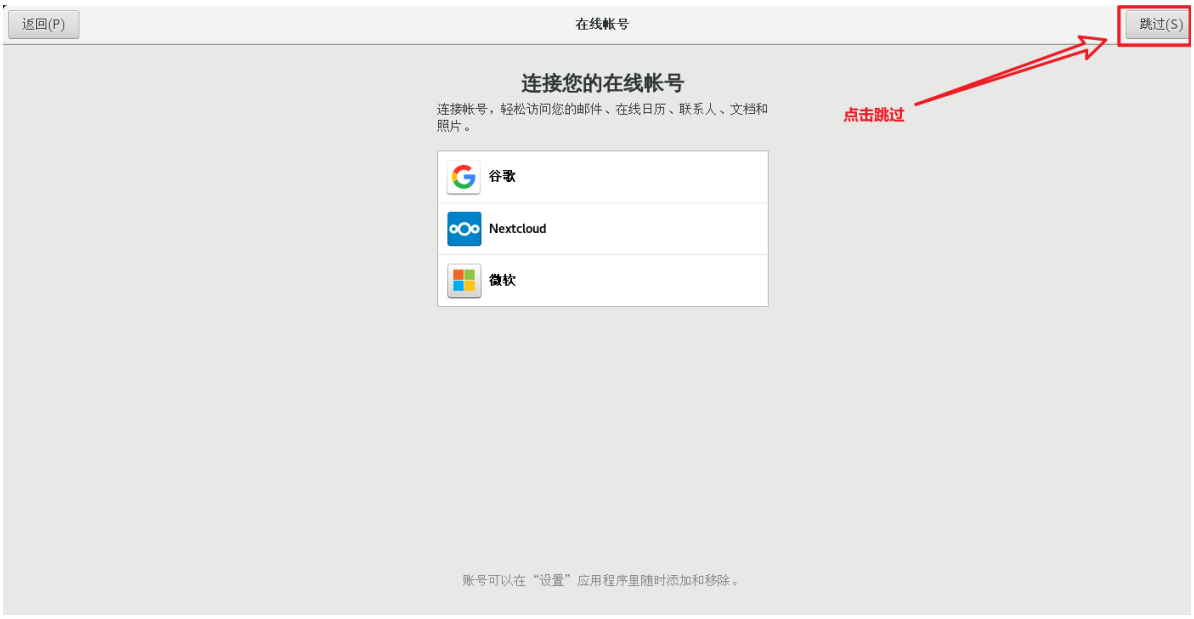

初始设置
CENTOS LINUX 7 (CORE)
cn
帮助！
CentOS
LICENSING
LICENSE INFORMATION
许可证已接受
系统
网络和主机名(N)
有线（ens33）已连接
用户设置
创建用户(U)
不会创建任何用户
点击完成配置
退出（Q）
完成配置（F）


欢迎
前进(N)
点击
Welcome!
Deutsch
Deutschland
English
United States
Español
España
Français
France
Русский
Российская Федерация
العربية
مصر
日本語
日本
汉语
中国


返回(P)
输入
前进(N)
点击前进
输入
选择您的键盘布局或者其他输入方式。
Hanyu Pinyin (altgr)
预览
汉语 (Intelligent Pinyin)
预览
汉语
预览


返回(P)
隐私
前进(N)
2、点击前进
隐私
位置服务
关闭
1、关闭
允许应用程序确定您的地理位置。定位服务使用时会有指示状态提示。
使用 Mozilla 位置服务： 隐私政策
隐私控制可以随时在“设置”应用程序里更改。


返回(P)
时区
前进(N)
时区
如果您的位置可被发现，时区将会自动设置。您也可以搜索城市来自己设置。
上海，上海，中国
1、搜索上海
2、点击前进
CST (UTC+08)
上海，中国
14:52

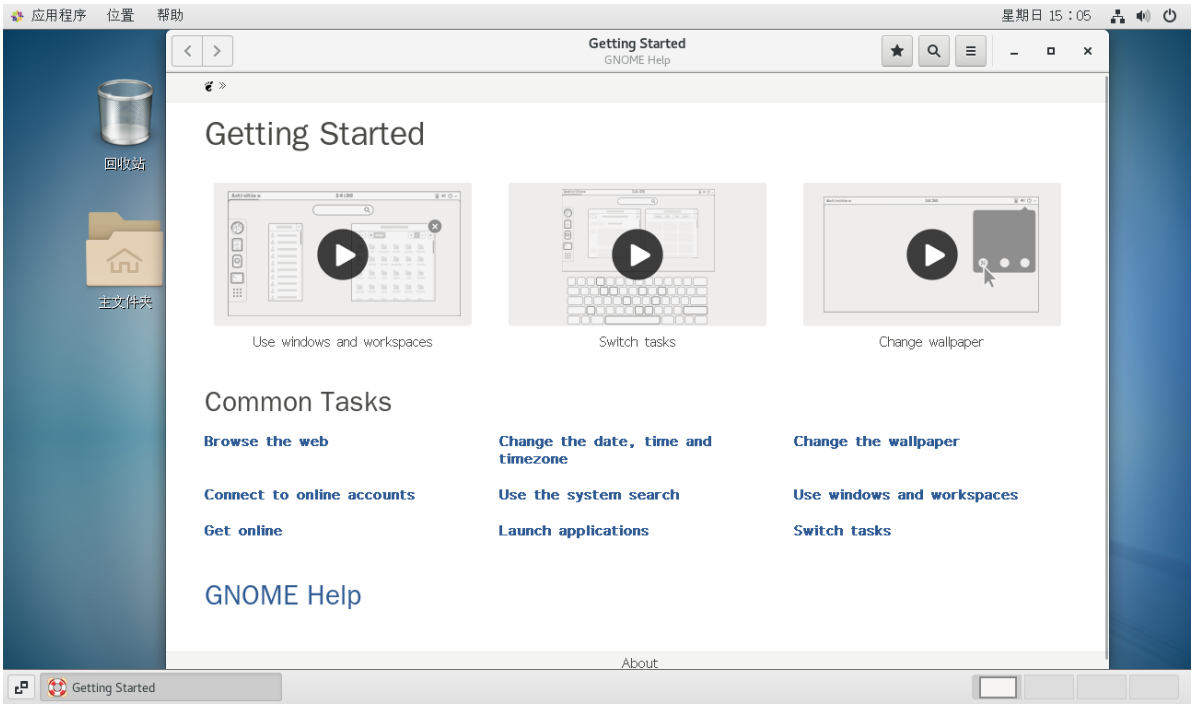

返回(P)
在线帐号
跳过(S)
连接您的在线帐号
连接帐号，轻松访问您的邮件、在线日历、联系人、文档和照片。
谷歌
Nextcloud
微软
点击跳过
账号可以在“设置”应用程序里随时添加和移除。


返回(P)
关于您
前进(N)
2、点击前进
关于您
我们需要一些详细信息来完成设置。
全名(F)
ready
用户名(U)
ready
1、需要创建一个账户，随便写
设置企业登录(E)


返回(P)
密码
前进(N)
2、点击前进
设置密码
请记好您的密码。
密码(P)
尝试在密码中混合使用字母，数字和标点符号。
确认(C)
1、设置刚才用户的密码，建议记好
若这里密码设置不合理，
则右上角"前进"按钮无法点击


准备好了
一切就续，开始用吧！
开始使用 CentOS Linux(S)
点击完成安装

## （22）完成安装


应用程序
位置
帮助
星期日 15：05
回收站
主文件夹
Getting Started
GNOME Help
Getting Started
Use windows and workspaces
Switch tasks
Change wallpaper
Common Tasks
Browse the web
Change the date, time and timezone
Change the wallpaper
Connect to online accounts
Use the system search
Use windows and workspaces
Get online
Launch applications
Switch tasks
GNOME Help
About
Getting Started

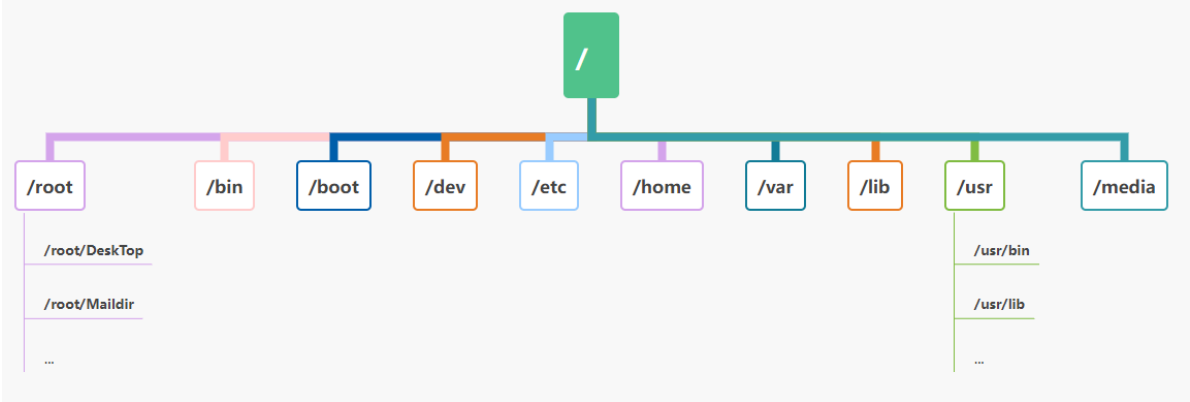
- 领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】
- 更多学习资料

# 4、Linux文件与目录结构

## 4.1 Linux文件

Linux系统中一切皆文件。

## 4.2 Linux目录结构



| 目录 | 说明 |
| --- | --- |
| /bin | 是Binary的缩写，这个目录存放着经常使用的命令 |
| /sbin | s就是Super User的意思，这里存放的是系统管理员使用的系统管理程序 |
| /home | 存放着普通用户的主目录，在Linux中每个用户都有一个自己的目录，一般该目录是以用户的账户号命名的 |
| /root | 该目录为系统管理员，也称作超级权限管理者的用户主目录 |
| /lib | 系统开机所需要最基本的动态链接共享库，其作用类似于Windows里面的DLL文件，几乎所有的应用程序都需要用到这些共享库 |
| /lost+found | 这个目录一般情况下是空的，当系统非法关机后，这里就存放一些文件 |
| /etc | 所有的系统管理员所需要的配置文件和子目录 |
| /usr | 这是一个非常重要的目录，用户的很多应用程序和文件都放在这个目录下，类似于windows下面的program files目录 |
| /boot | 这里存放的是启动Linux时使用的一些核心文件，包括一些链接文件以及镜像文件，自己安装的别放这里 |
| /proc | 这个目录是一个虚拟的目录，他是系统内存的映射，我们可以通过直接访问这个目录来获取系统信息 |
| /srv | service缩写，该目录存放一些服务启动之后需要提取的数据 |
| /sys | 这是linux2.6内核的一个很大的变化，该目录下安装了2.6内核中新出现的一个文件系统sysfs |
| /tmp | 这个目录是用来存放一些临时文件的 |
| /dev | 类似于windows的设备管理器，把所有的硬件用文件的形式存储 |
| /media | linux系统会自动识别一些设备，例如U盘，光驱等等，当时识别后，linux会把识别的设备挂载到这个目录下 |
| /mnt | 系统提供该目录是为了让用户临时挂载别的文件系统，我们可以将外部的存储挂载在/mnt/上，然后进入该类目就可以查看里面的内容了 |
| /opt | 这个是给主机额外安装软件所摆放的目录，比如你安装一个MySQl数据库就可以放在这个目录，默认是空的 |
| /var | 这个目录存放不但扩充着的东西，我们习惯将那些经常修改的目录放在这个目录，比如：各种日志文件 |


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 5、VI/VIM编辑器

## 5.1、vi/vim是什么

VI是Unix操作系统和类Unix操作系统中最强大的文本编辑器。

VIM编辑器是从VI编辑器发展出来的一个更强大的文本编辑器，可以主动以文字颜色辨别语法的正确性，方便程序设计，VIM和VI编辑器完全兼容。

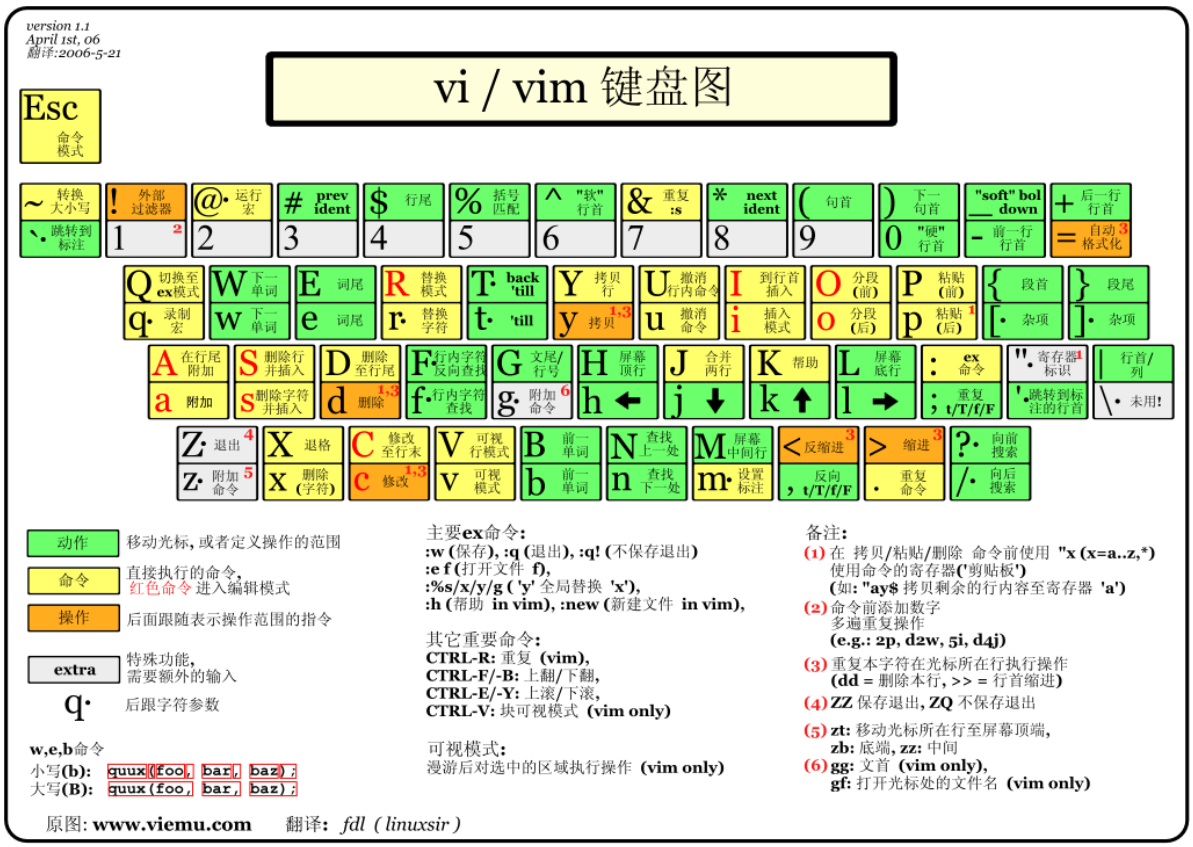
# 5.2、一般模式

以vi打开一个档案就直接进入一般模式了（这是默认的模式），在这个模式中，你可以使用【上线左右】按键来移动光标，你可以使用【删除字符】或【删除整行】来处理档案内容，也可以使用【复制、粘贴】来处理你的文件数据。

## （1）常用语法表

| 语法 | 功能描述 |
| --- | --- |
| yy | 复制光标当前一行 |
| y数字y | 复制一段（从第几行到第几行） |
| `y$` | 在当前行复制：行首到光标位置的内容 |
| `y^` | 在当前行复制：光标位置至行尾的内容 |
| p | 箭头移动到目的行粘贴，会粘贴在目的行的后面 |
| u | 撤销上一步 |
| dd | 删除光标当前行 |
| d数字d | 删除光标（含）后多少行 |
| `d$` | 在当前行删除：行首到光标位置的内容 |
| `d^` | 在当前行删除：光标位置至行尾的内容 |
| x | 剪切一个字母，相当于del |
| X | 剪切一个字母，相当于Backspace |
| yw | 复制一个词 |
| dw | 删除一个词 |
| shift+6（^） | 移动到行头 |
| shift+4（$） | 移动到行尾 |
| 1+G | 移动到页头 |
| G | 移动到页尾 |
| `数字+G` 或 `数字+gg` | 移动到目的行 |
| `gg` 或 `H` | 移动到第一行 |
| H | 移动到屏幕顶行 |
| L | 移动到屏幕底行 |

### （2）vi/vim键盘图




## 5.3、编辑模式

在一般模式中可以进行删除、复制、粘贴等动作，但是却无法编辑文件内容，要等你按下【i，I，o，O，a，A】等任意一个字母之后才会进入编辑模式。

注意了，通常在Linux中，按下这些按键时，在画面的左下方会出现【INSERT或REPLACE】的字样，此时才可以进行编辑，而如果要回到一般模式时，则必须要按下【Esc】这个按键即可退出编辑模式。

### （1）进入编辑模式

| 按键 | 功能 |
| --- | --- |
| i | 当前光标前 |
| I | 光标所在行最前 |
| a | 当前光标后 |
| A | 光标所在行最后 |
| o | 当前光标行的下一行 |
| O | 当前光标行的上一行 |

### （2）退出编辑模式

按【ESC】键，便可退出编辑模式，会进入一般模式。

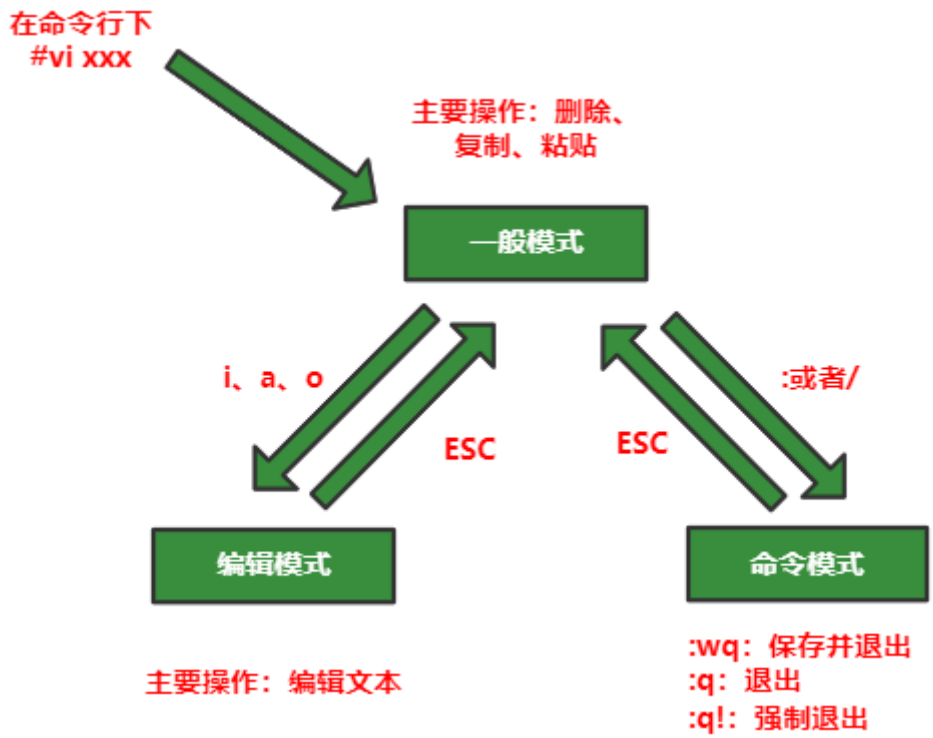
## 5.4、指令模式

在一般模式当中，输入【:/?】3个中的任何一个按钮，就可以将光标移动到最底下那一行。

在这个模式当中，可以提供你【搜索资料】的动作，而读取、存盘、大量取代支付、离开vi、显示行号等动作是在此模式中达成的！

| 命令 | 功能 |
| --- | --- |
| :w | 保存 |
| :q | 退出 |
| :! | 强制执行 |
| :wq! | 强制保存退出 |
| /要查找的单词 | n查找下一个，N往上查找 |
| :noh | 取消高亮显示 |
| :set nu | 显示行号 |
| :set nonu | 关闭行号 |
| :s/old/new | 替换当前行匹配到的第一个old为new |
| :s/old/new/g | 替换当前行匹配到的所有old为new |
| :%s/old/new | 替换文档汇总每一行匹配到的第一个old为new |
| :%s/old/new/g | 替换内容，将old替换为new，/g替换匹配到的所有内容 |

## 5.5、模式转换




- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

# 6、网络配置和系统管理操作

视频：https://www.bilibili.com/video/BV1WY4y1H7d3?p=19

## 6.1、VMware提供3种网络设置

### 1）桥接模式

虚拟机直接接连接外部物理网络的模式，主机起到了桥接网络的作用，这种模式下，虚拟机可以直接访问外部网络，并且对外部网络是可见的。

### 2）NAT模式

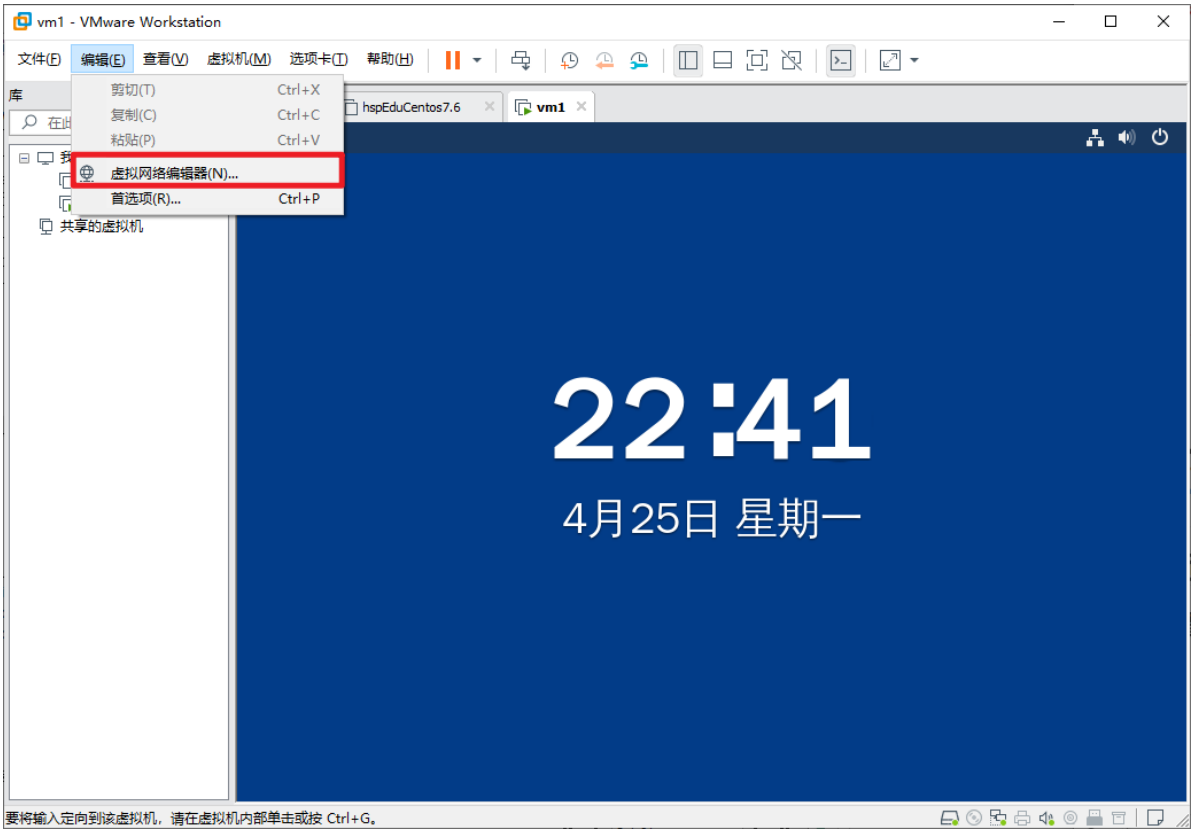
虚拟机和主机构建一个专用网络，并通过虚拟机网络地址转换（NAT）设备对 IP 进行转换，虚拟机通过共享主机 IP 可以访问外部网络，但外部网络无法访问虚拟机。

### 3）仅主机模式

虚拟机只与主机共享一个专用网络，与外部网络无法通信。

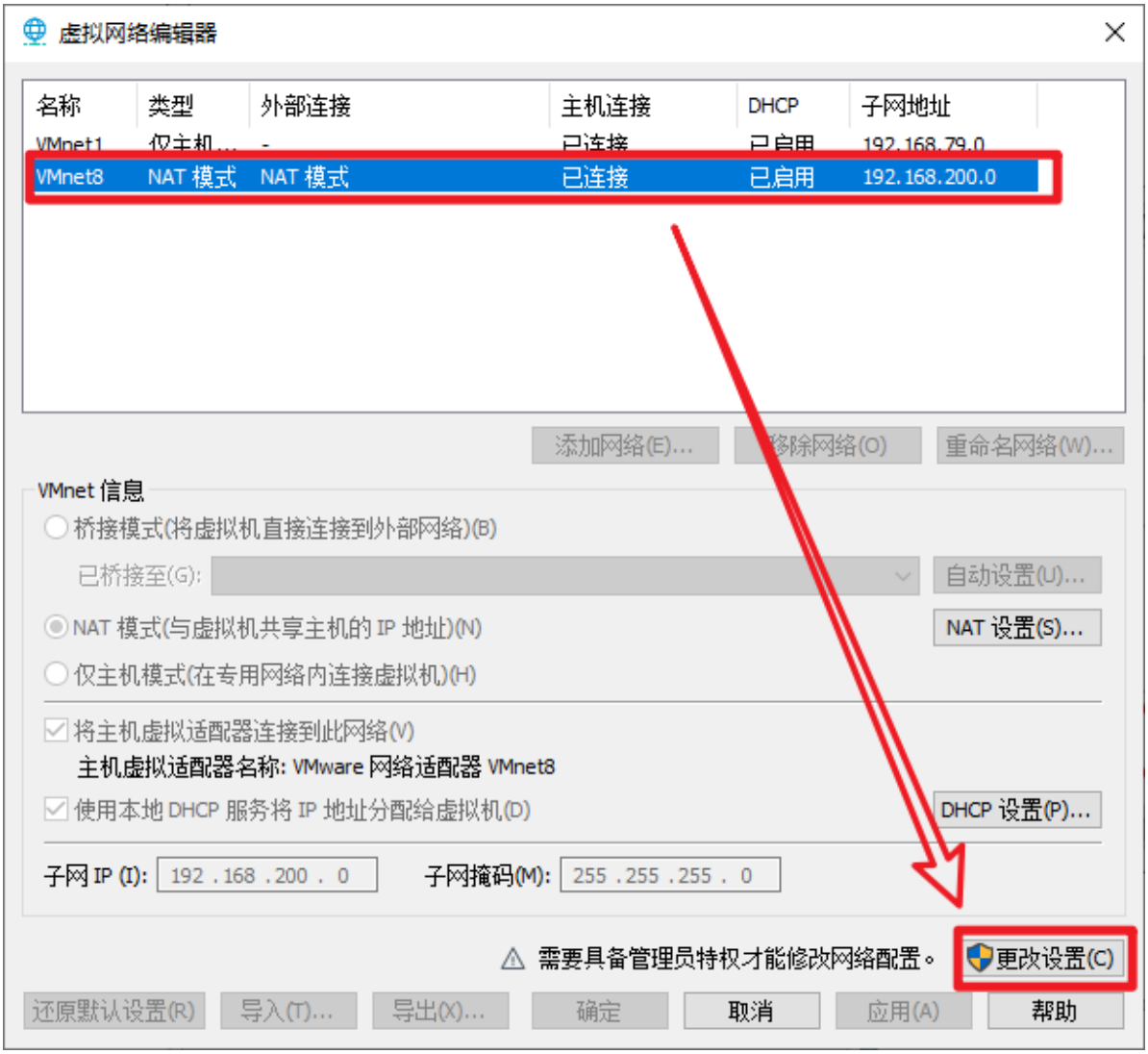
## 6.2、虚拟机网段和网关

### 1）查看虚拟网络编辑器



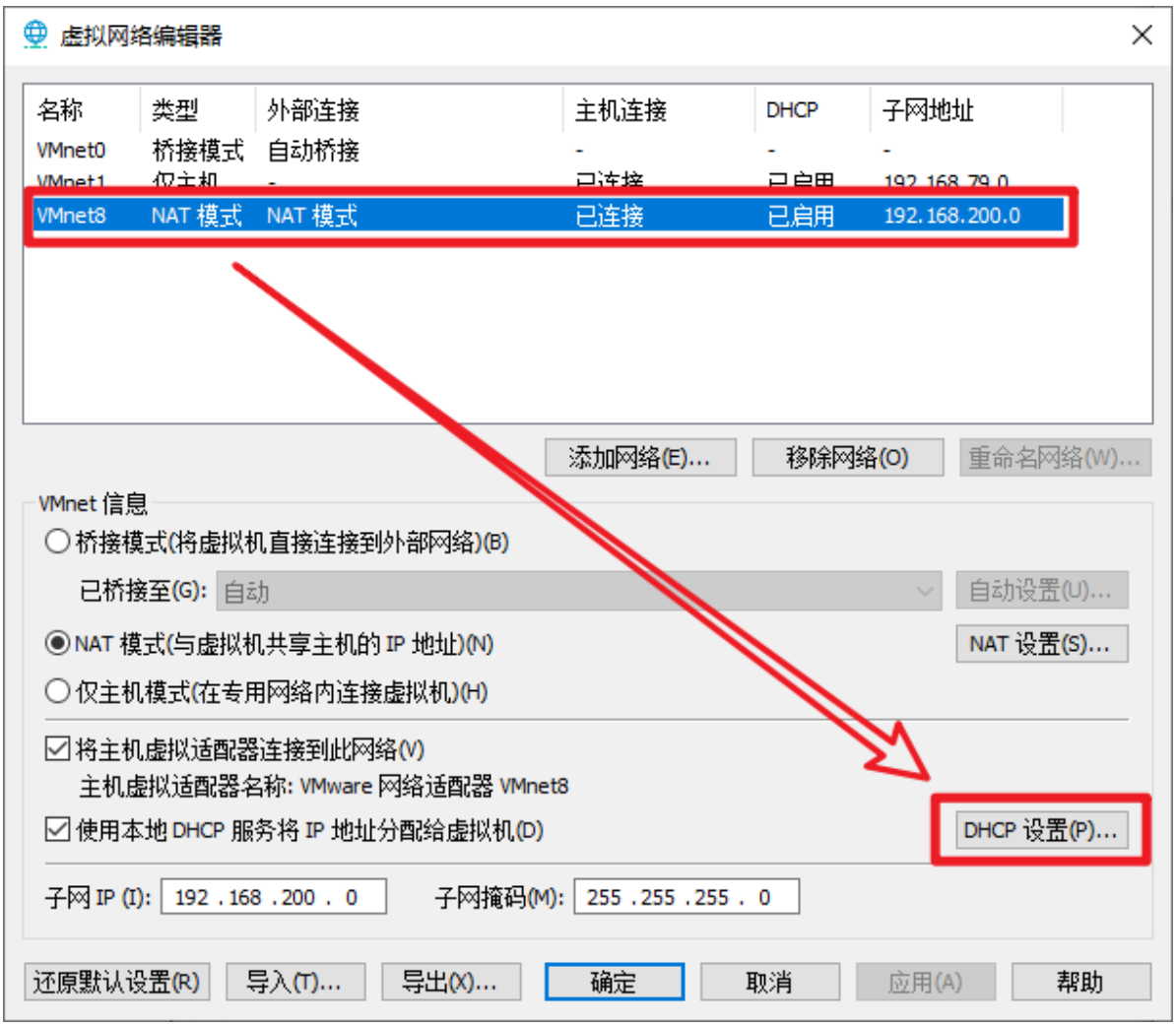
## 2）虚拟机网络设置

> 如下图，需要选中列表中的VMnet8




## 3）DHCP设置

> 如下图，需要选中列表中的VMnet8


虚拟网络编辑器
名称 类型 外部连接 主机连接 DHCP 子网地址
VMnet0 桥接模式 自动桥接 - - -
VMnet8 NAT 模式 NAT 模式 已连接 已启用 192.168.200.0
添加网络(E)...
移除网络(O)
重命名网络(W)...
VMnet 信息
桥接模式(将虚拟机直接连接到外部网络)(B)
已桥接至(G): 自动
自动设置(U)...
NAT 模式(与虚拟机共享主机的 IP 地址)(N)
NAT 设置(S)...
仅主机模式(在专用网络内连接虚拟机)(H)
将主机虚拟适配器连接到此网络(V)
主机虚拟适配器名称: VMware 网络适配器 VMnet8
使用本地 DHCP 服务将 IP 地址分配给虚拟机(D)
DHCP 设置(P)...
子网 IP (I): 192 . 168 . 200 . 0
子网掩码(M): 255 . 255 . 255 . 0
还原默认设置(R)
导入(T)...
导出(X)...
确定
取消
应用(A)
帮助

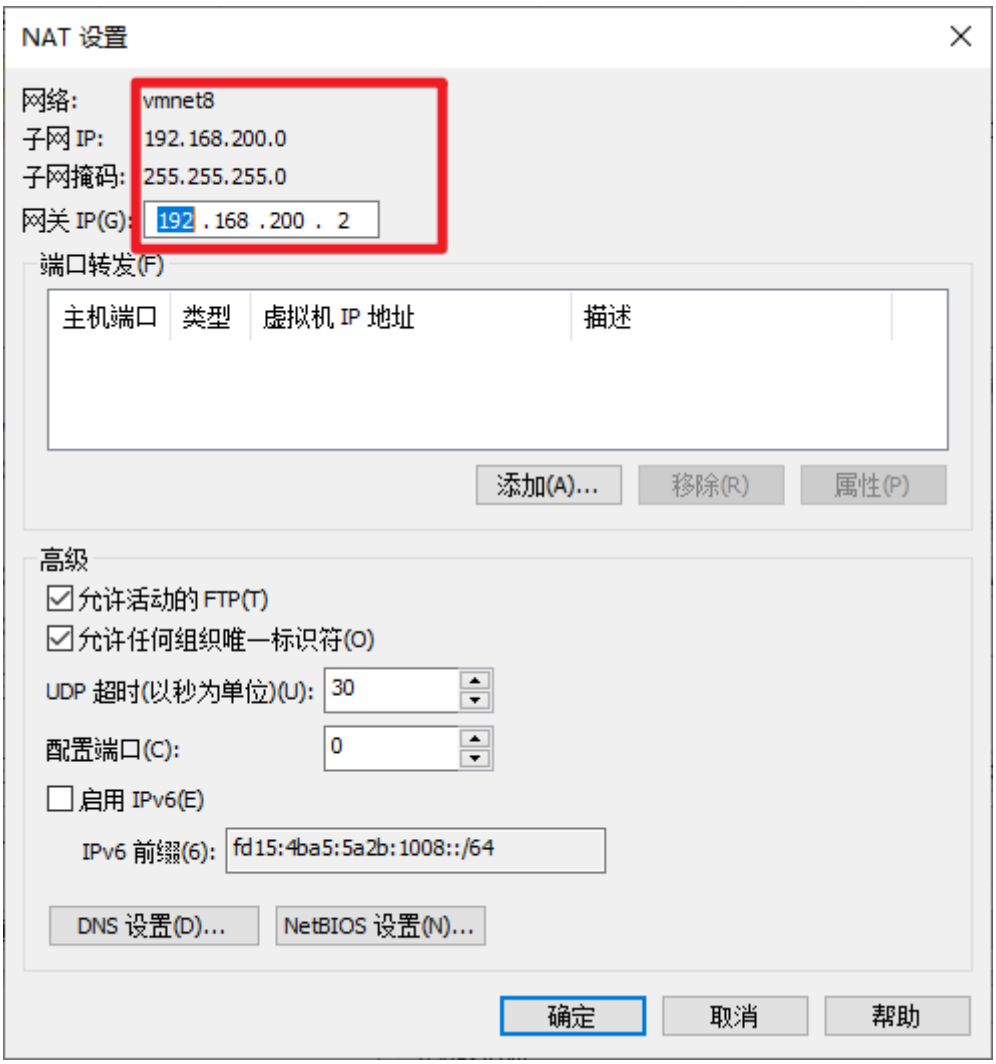
## 4）DHCP设置


DHCP 设置
网络: vmnet8
子网 IP: 192.168.200.0
子网掩码: 255.255.255.0
起始 IP 地址(S): 192 . 168 . 200 . 128
结束 IP 地址(E): 192 . 168 . 200 . 254
广播地址: 192.168.200.255
天: 小时: 分钟:
默认租用时间(D): 0 0 30
最长租用时间(M): 0 2 0
确定
取消
帮助


虚拟网络编辑器
名称 类型 外部连接 主机连接 DHCP 子网地址
VMnet0 桥接模式 自动桥接 - - -
VMnet1 仅主机... - 已连接 已启用 192.168.79.0
VMnet8 NAT 模式 NAT 模式 已连接 已启用 192.168.200.0
添加网络(E)...
移除网络(O)
重命名网络(W)...
VMnet 信息
桥接模式(将虚拟机直接连接到外部网络)(B)
已桥接至(G): 自动
自动设置(U)...
NAT 模式(与虚拟机共享主机的 IP 地址)(N)
NAT 设置(S)...
仅主机模式(在专用网络内连接虚拟机)(H)
将主机虚拟适配器连接到此网络(V)
主机虚拟适配器名称: VMware 网络适配器 VMnet8
使用本地 DHCP 服务将 IP 地址分配给虚拟机(D)
DHCP 设置(P)...
子网 IP (I): 192 . 168 . 200 . 0
子网掩码(M): 255 . 255 . 255 . 0
还原默认设置(R)
导入(T)...
导出(X)...
确定
取消
应用(A)
帮助

## 5）查看网关


虚拟网络编辑器
名称 类型 外部连接 主机连接 DHCP 子网地址
VMnet0 桥接模式 自动桥接 - - -
VMnet1 仅主机... - 已连接 已启用 192.168.79.0
VMnet8 NAT 模式 NAT 模式 已连接 已启用 192.168.200.0
添加网络(E)...
移除网络(O)
重命名网络(W)...
VMnet 信息
桥接模式(将虚拟机直接连接到外部网络)(B)
已桥接至(G): 自动
自动设置(U)...
NAT 模式(与虚拟机共享主机的 IP 地址)(N)
NAT 设置(S)...
仅主机模式(在专用网络内连接虚拟机)(H)
将主机虚拟适配器连接到此网络(V)
主机虚拟适配器名称: VMware 网络适配器 VMnet8
使用本地 DHCP 服务将 IP 地址分配给虚拟机(D)
DHCP 设置(P)...
子网 IP (I): 192 . 168 . 200 . 0
子网掩码(M): 255 . 255 . 255 . 0
还原默认设置(R)
导入(T)...
导出(X)...
确定
取消
应用(A)
帮助




# 6.3、配置网络ip地址

## 6.3.1、ifconfig查看网络信息

```
[root@test1 ~]# ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.200.135  netmask 255.255.255.0  broadcast 192.168.200.255
        inet6 fe80::b84b:79f6:abb4:afa9  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:32:65:be  txqueuelen 1000  (Ethernet)
        RX packets 462144  bytes 639516380 (609.8 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 175430  bytes 14334643 (13.6 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

## 6.3.2、ping测试主机之间的网络

### 1）基本语法

> 用来测试当前服务器是否可以连接目标主机

```
ping 目标主机
```

### 2）案例

> 测试当前服务器是否可以连接百度，效果如下，表示正常

```
[root@test1 ~]# ping baidu.com
PING baidu.com (220.181.38.148) 56(84) bytes of data.
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=1 ttl=128 time=32.4 ms
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=2 ttl=128 time=33.7 ms
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=3 ttl=128 time=31.2 ms
```

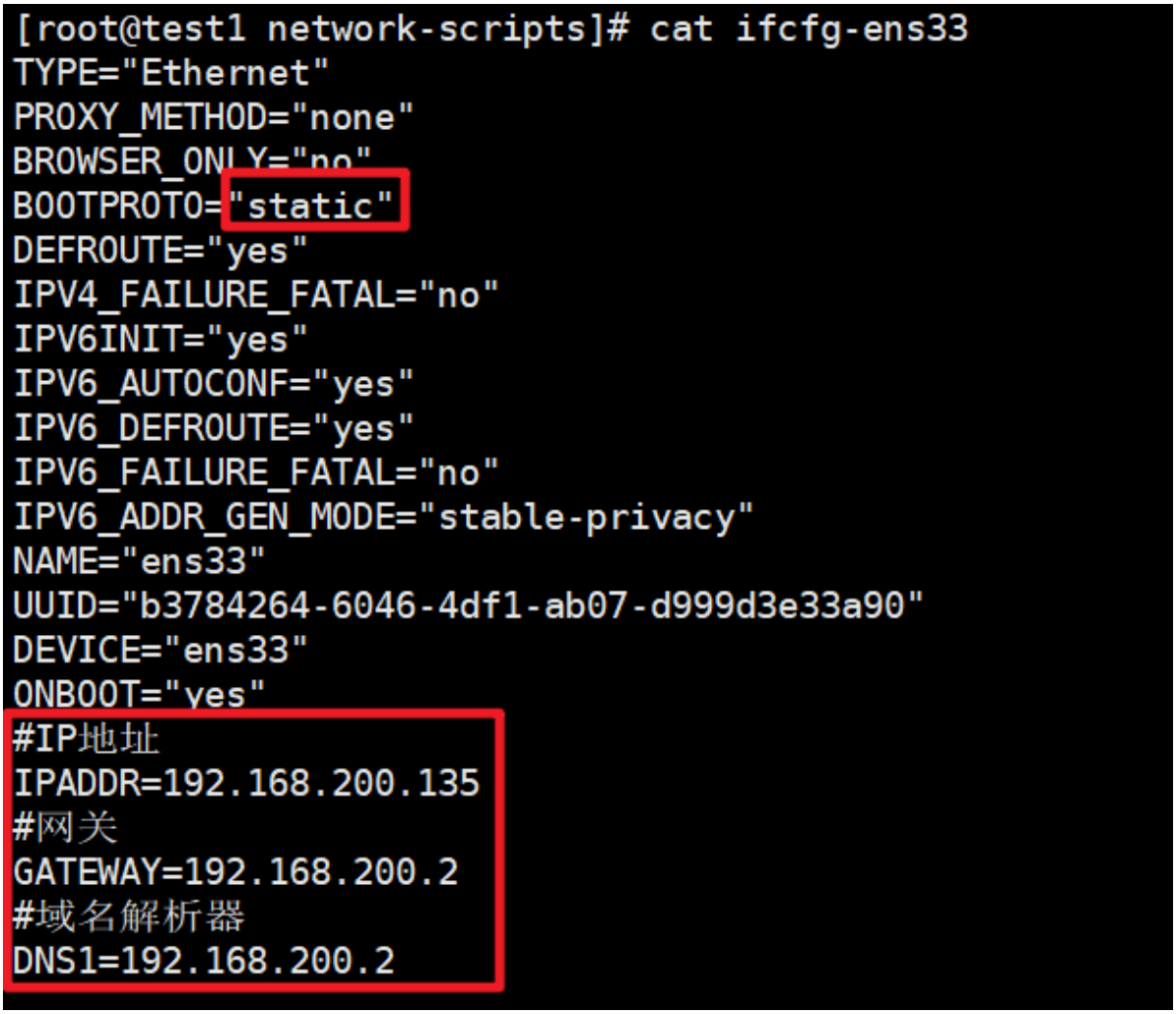
## 6.3.3、修改IP地址为固定的

### 1）修改IP配置文件

修改`/etc/sysconfig/network-scripts/ifcfg-ens33`文件内容，以下标红的项必须修改， 有值的按照下面的值修改， 没有该项的要增加

```
TYPE="Ethernet"     #网络类型（通常是 Ethemet）
PROXY_METHOD="none"
BROWSER_ONLY="no"
BOOTPROTO="static"    #IP 的配置方法[none|static|bootp|dhcp]（引导时不 使用协议|静态分配 IP|BOOTP 协议|DHCP 协议）
DEFROUTE="yes"
IPV4_FAILURE_FATAL="no"
IPV6INIT="yes"
IPV6_AUTOCONF="yes"
IPV6_DEFROUTE="yes"
IPV6_FAILURE_FATAL="no"
IPV6_ADDR_GEN_MODE="stable-privacy"
NAME="ens33"
UUID="e83804c1-3257-4584-81bb-660665ac22f6"    #随机 id
DEVICE="ens33"    #接口名（设备,网卡）
ONBOOT="yes"    #系统启动的时候网络接口是否有效（yes/no）
#IP 地址
IPADDR=192.168.1.100
#网关
GATEWAY=192.168.1.2
#域名解析器
DNS1=192.168.1.2
```

本机修改之后




### 2）重启网络

使用`service network restart`命令重启网络，效果如下

```
[root@test1 network-scripts]# service network restart
Restarting network (via systemctl):                        [  确定  ]
```

## 5.3.4、修改IP地址后可能遇到的问题

（1）物理机器能ping通虚拟机，但是虚拟机ping不通物理机，一般都是因为物理机的防火墙问题，把物理机防火墙关闭就行

（2）虚拟机能ping通物理机，但是虚拟机ping不通外网，一般都是因为DNS的设置有问题

（3）虚拟机`ping www.baidu.com`显示域名未知等信息，一般查看GATEWAY和DNS设置是否正确

（4）如果以上全部设置完还是不行，需要关闭NetworkManager服务

- 关闭：`systemctl stop NetworkManager`
- 禁用：`systemctl disable NetworkManager`

（5）如果检查发现systemctl status network 有问题，需要检查ifcgf-ens33

# 6.4、配置主机名

## 6.4.1、hostname查案当前服务器主机名称

```
[root@test1 ~]# hostname
test1
```

## 6.4.2、修改主机名称

### 方式1：修改/etc/hostname文件，需重启

如果感觉主机名不合适，我们可以进行修改，通过编辑`/etc/hostname`文件进行修改，修改完毕之后重启生效。

```
[root@test1 ~]# vi /etc/hostname
```

### 方式2：hostnamectl命令修改，无需重启

```
[root@testx ~]# cat /etc/hostname
testx
[root@testx ~]# hostnamectl set-hostname test1
[root@testx ~]# hostname
test1
[root@testx ~]# cat /etc/hostname
test1
```

## 6.4.3、修改linux的主机映射文件（hosts文件）

由于虚拟机比较多，配置时通常会采用主机名的方式配置，比较简单方便，不用刻意记ip地址。

编辑`/etc/hosts`

> 格式：ip 名称1 名称2 名称n

```
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
192.168.0.103 test103
```

这样我们就可以通过`test103`来访问`192.168.0.103`这台机器了，比如`ping test103`效果如下，自动根据名称在`/etc/hosts`文件中找到对应的ip

```
[root@testx ~]# ping test103
PING test103 (192.168.0.103) 56(84) bytes of data.
64 bytes from test103 (192.168.0.103): icmp_seq=1 ttl=128 time=0.646 ms
64 bytes from test103 (192.168.0.103): icmp_seq=2 ttl=128 time=0.499 ms
64 bytes from test103 (192.168.0.103): icmp_seq=3 ttl=128 time=1.89 ms
```

window上也也有这种文件，对应的位置如下

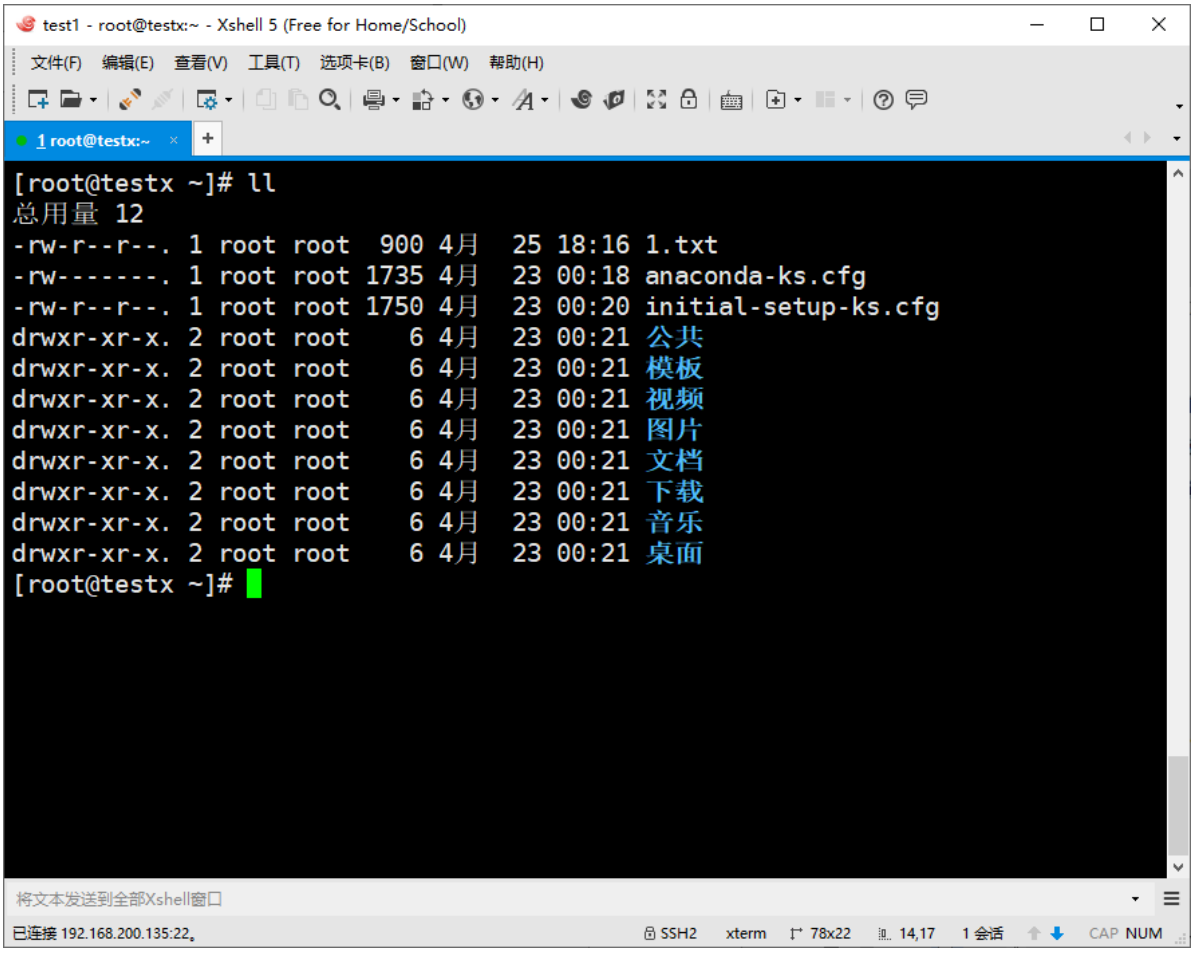
```
C:\Windows\System32\drivers\etc\hosts
```


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

# 7、远程登录（Xshell）

## 7.1、远程连接

通常在工作过程中，公司中使用的真实服务器或者是云服务器，都不允许除运维人员之外的员工直接接触，因此就需要通过远程登录的方式来操作。 所以，远程登录工具就是必不可缺的，目前，比较主流的有 Xshell、SSH Secure Shell、SecureCRT、FinalShell 等，大家可以根据自己的习惯自行选择，我用的是Xshell，效果如下图。




## 7.2、Xshell的使用

### 1）下载

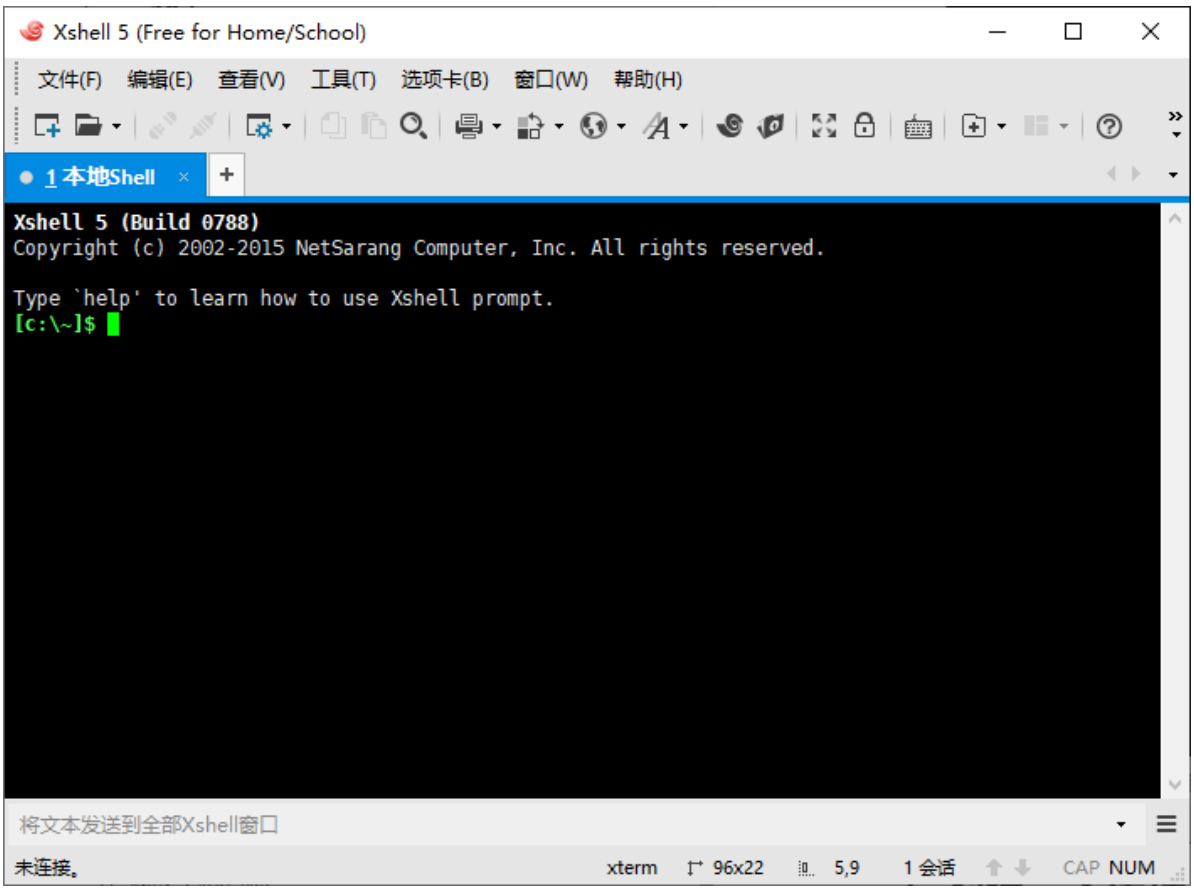
```
链接：https://pan.baidu.com/s/1mw_yS6RHyiy8gdgwzGSLKA
提取码：yyds
```

### 2）安装

安装就不介绍，exe文件下一步下一步就ok了。

### 3）打开xshell

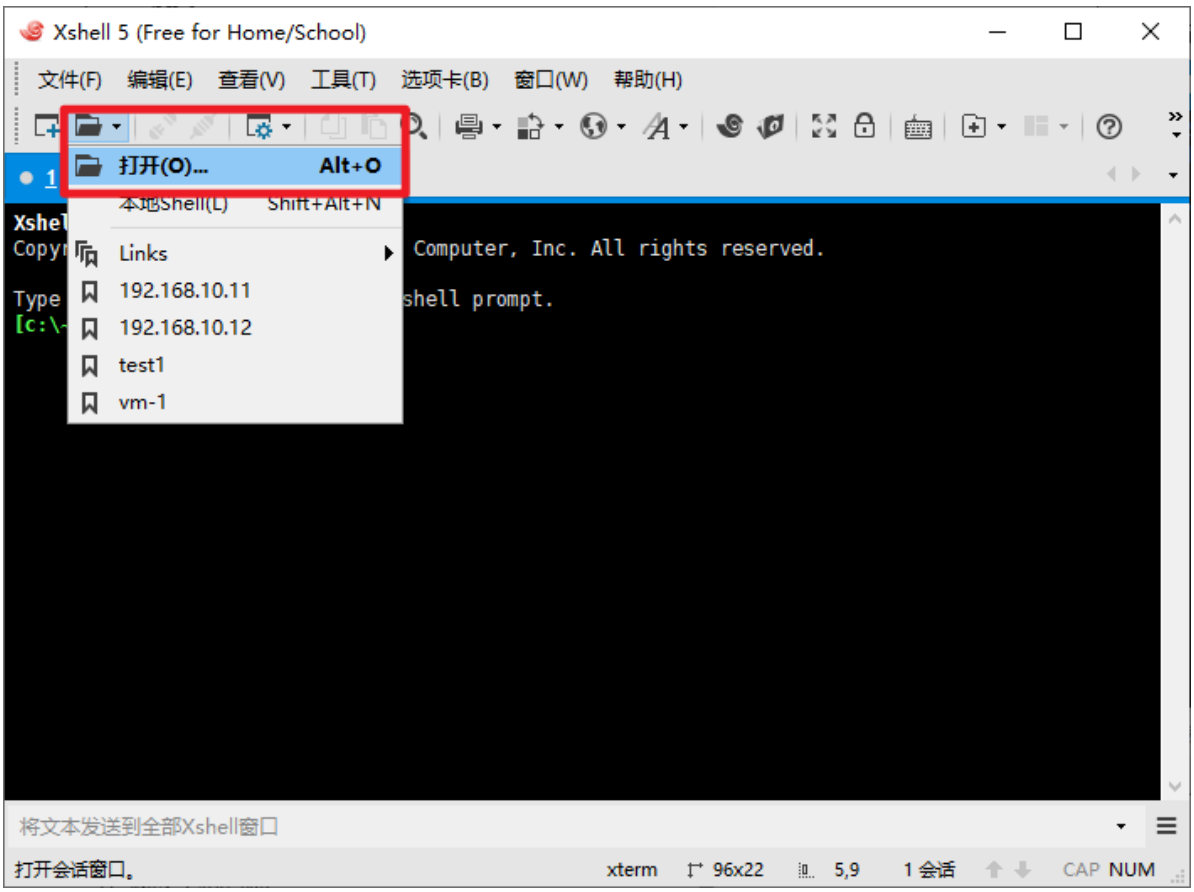
双击xshell软件即可打开，打开之后如下图



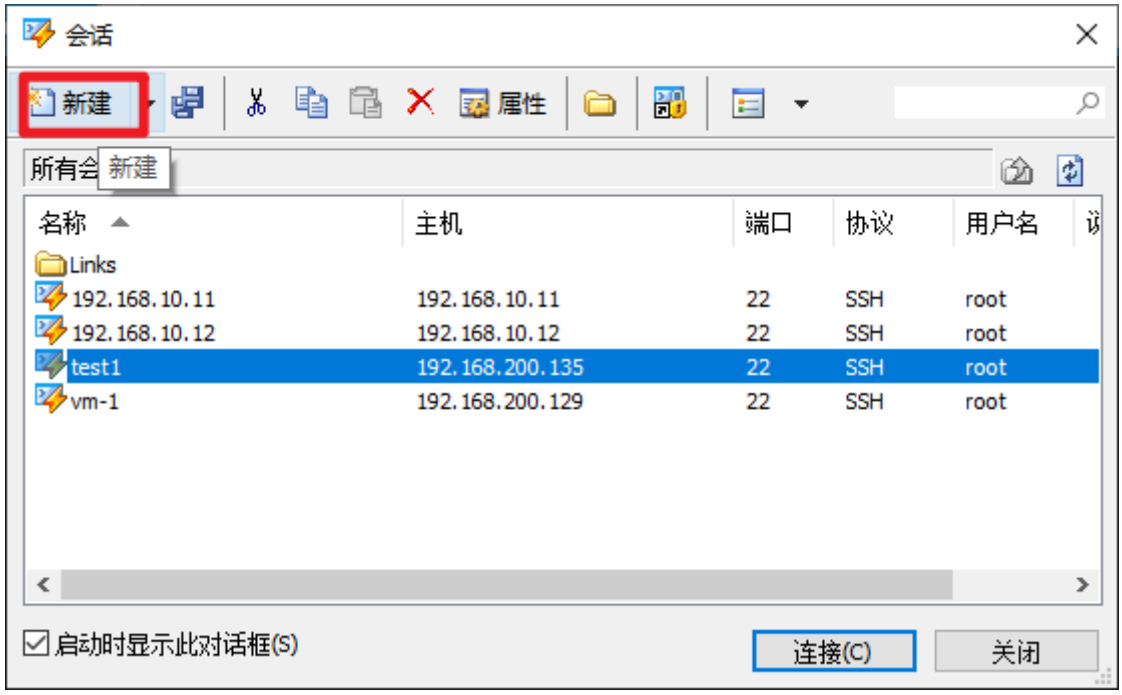

## 4）创建连接

比如我们想连接到`192.168.200.135`，

### （1）打开创建连接的窗口

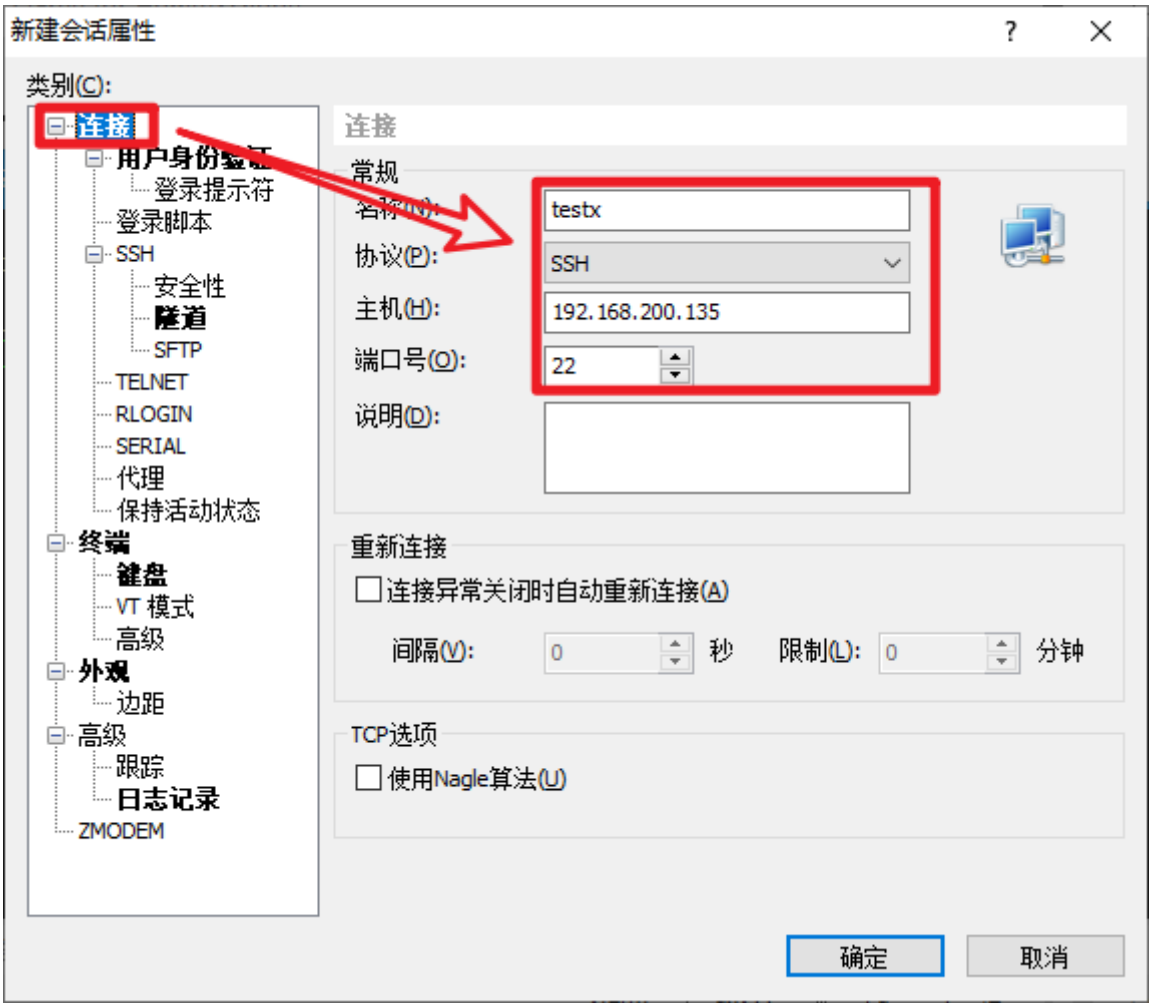


### (2) 新建会话连接

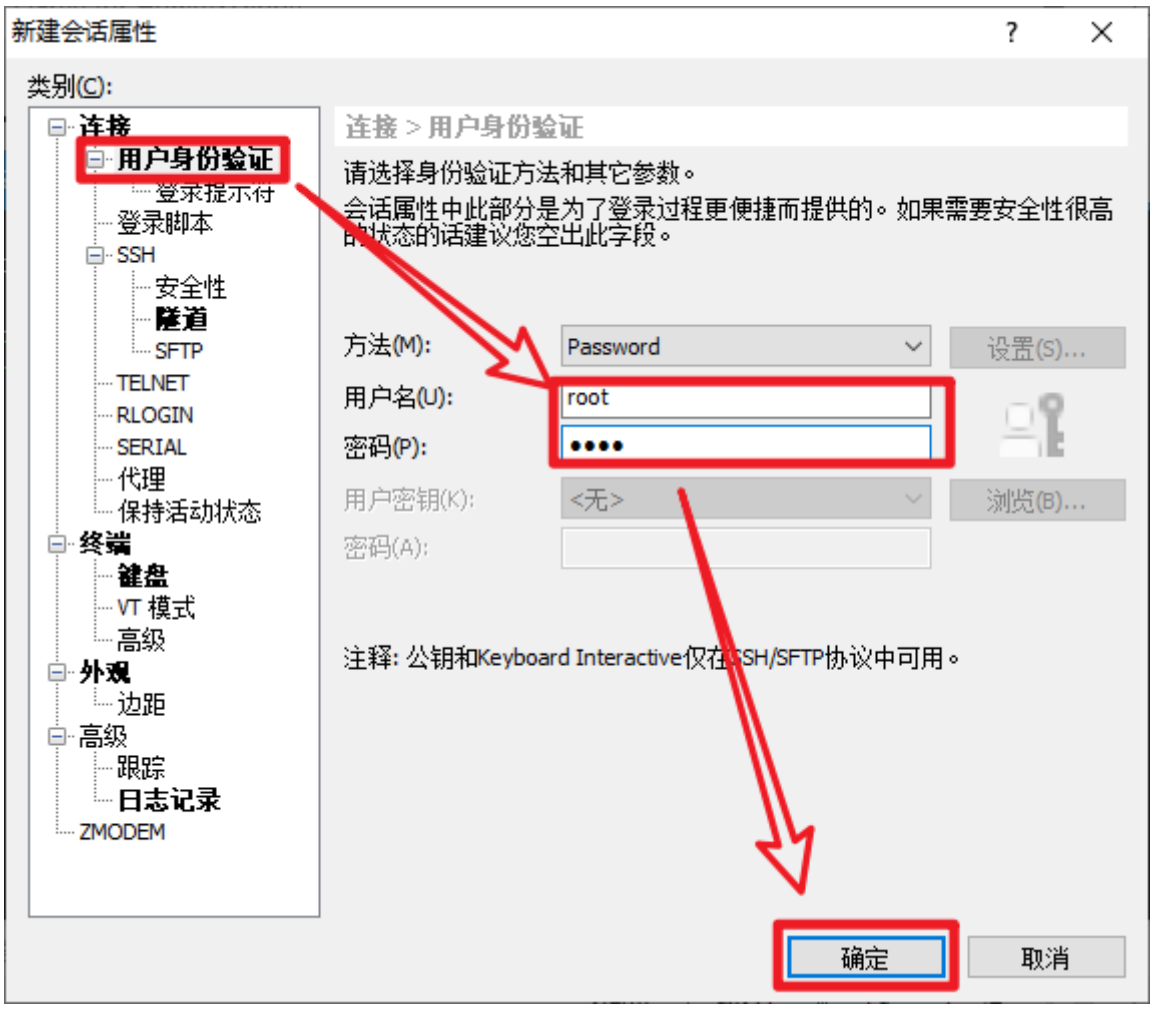



### (3) 输入机器名称（随意）、目标机器ip、端口

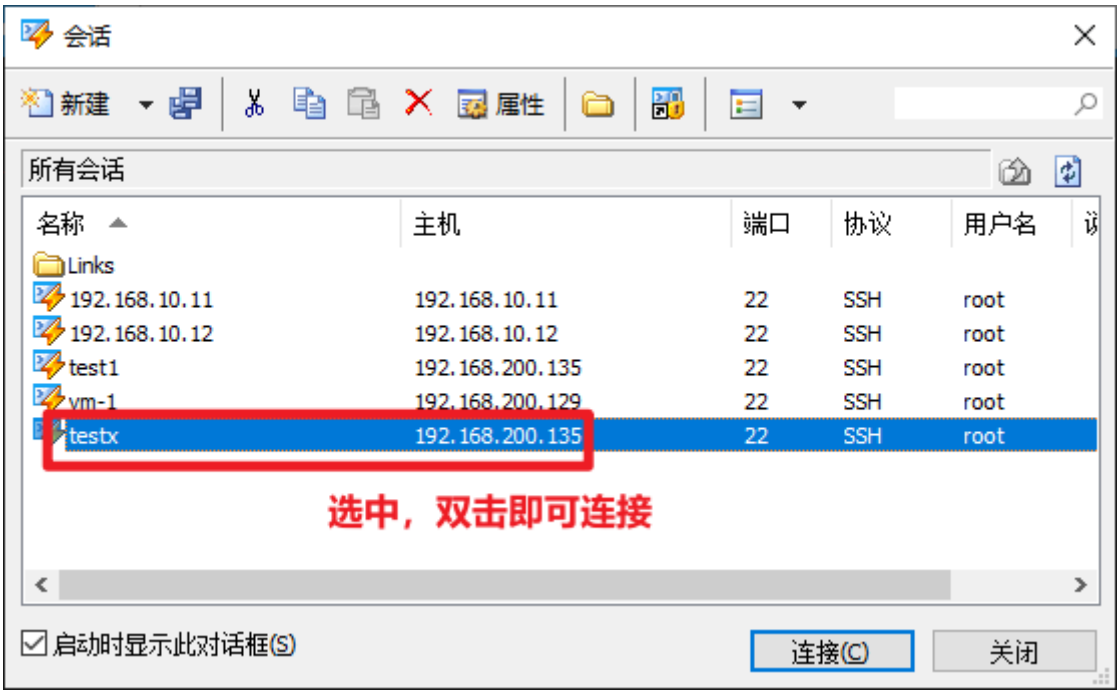
> 远程连接默认端口是22



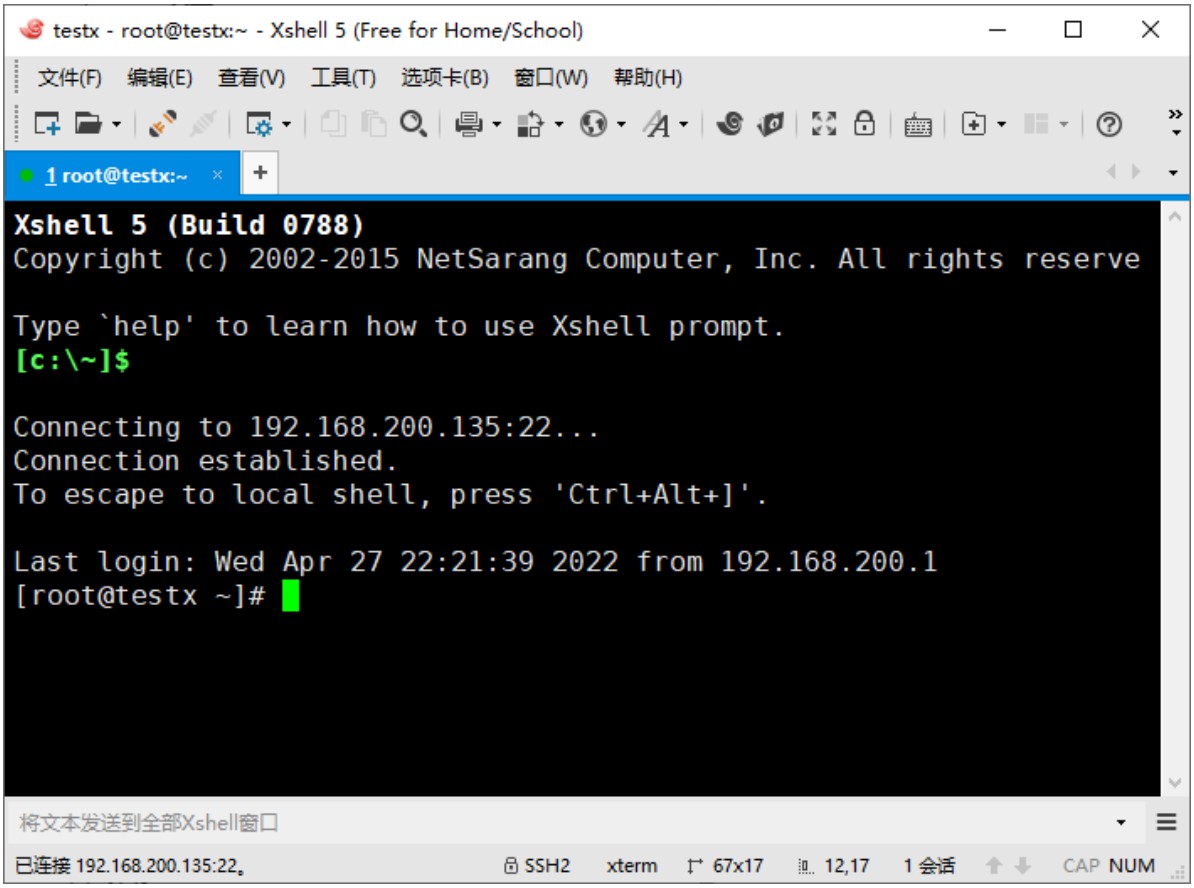
### (4) 输入目标机器的用户名和密码




### (5) 双击即可连接目标机器

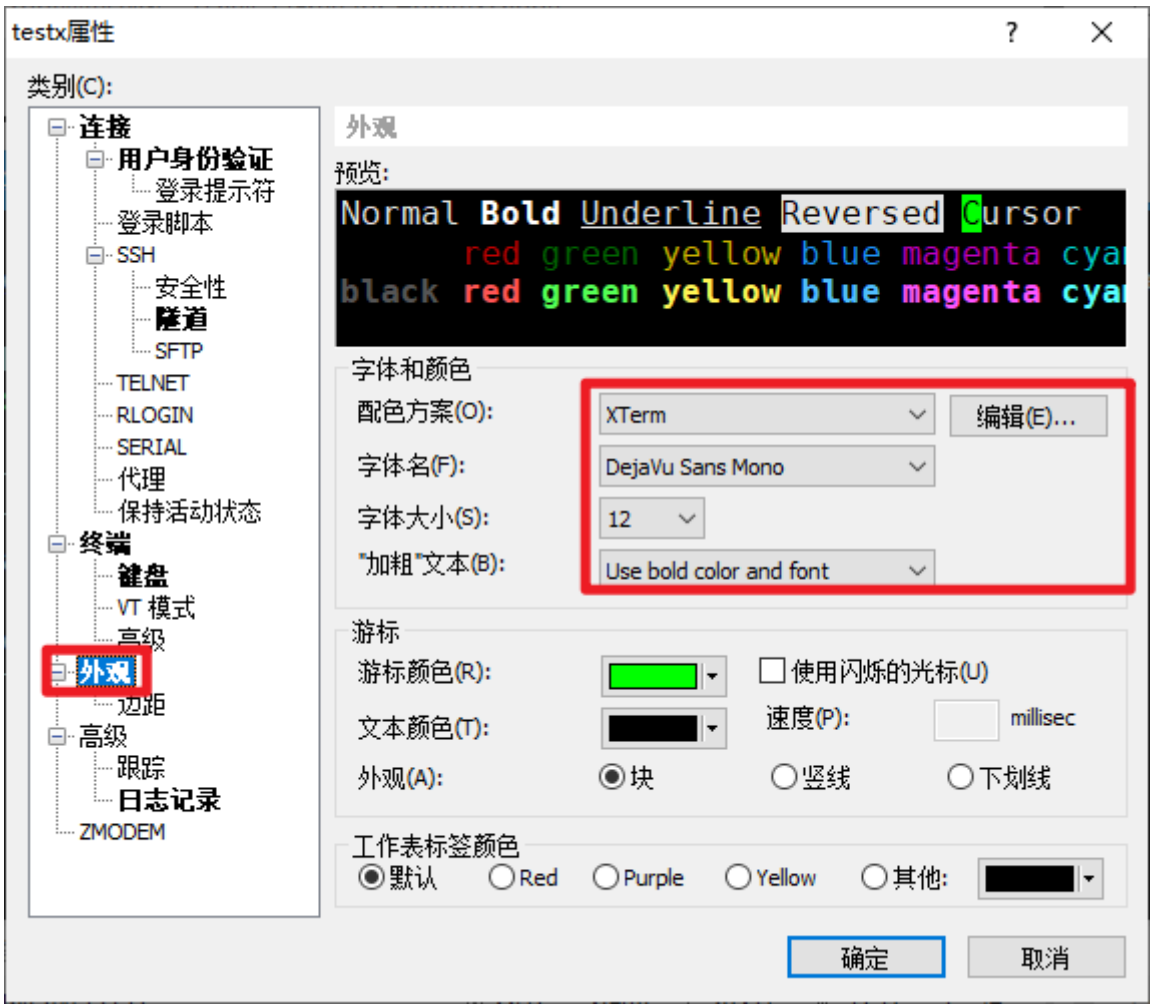


### （6）连接成功




### （7）调整字体大小

> 如果感觉字体小的，可以调整一下字体，其他功能大家慢慢摸索，或者百度，资料很多。




- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

# 8、系统管理

## 8.1、Linux中的进程和服务

计算机中，一个正在执行的程序或命令，被叫做“进程”（process）。

启动之后一直存在、常驻内存的进程，一般被称作“服务”（service）。

## 8.2、service 服务管理（CentOS 6 版本-了解）

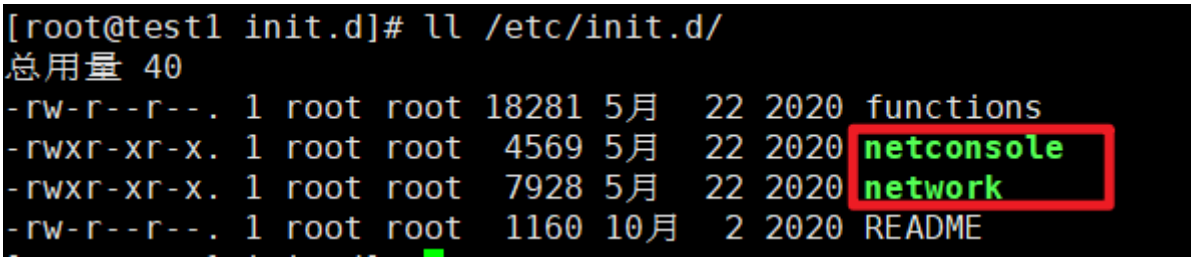
### 1）基本语法

```
service 服务名 start|stop|restart|status
```

- centos6及以前的版本使用的是service命令做服务管理，而从7开始使用systemctl来做服务管理了
- start：启动服务
- stop：停止服务
- restart：重启服务
- status：查看服务状态

### 2）经验技巧

service能够启动的服务名称位于`/etc/init.d/`目录，如下图，发现只有2个服务（下图中绿色的部分表示可执行文件）




### 3）案例实操

#### （1）查看网络服务状态

```
[root@test1 init.d]# service network status
已配置设备:
lo ens33
当前活跃设备:
lo ens33 virbr0
```

#### （2）停止网络服务

```
[root@test1 init.d]# service network stop
```

#### （3）启动网络服务

```
[root@test1 ~]# service network start
Starting network (via systemctl):                          [  确定  ]
```

#### （4）重启网络服务

```
[root@test1 ~]# service network restart
Restarting network (via systemctl):                        [  确定  ]
```

## 8.3、chkconfig 设置后台服务的自启配置（CentOS 6 版本）

### 1） 基本语法

| 命令 | 解释 |
| --- | --- |
| chkconfig | 查看所有服务器自启配置 |
| chkconfig 服务名 off | 关掉指定服务的自动启动 |
| chkconfig 服务名 on | 开启指定服务的自动启动 |
| chkconfig 服务名 --list | 查案服务开机启动状态 |

### 2） 案例实操

#### （1）开启/关闭 network(网络)服务的自动启动

```
[root@test1 ~]# chkconfig network on
[root@test1 ~]# chkconfig network off
```

#### （2）开启/关闭 network 服务指定级别的自动启动

```
[root@test1 ~]#chkconfig --level 指定级别 network on
[root@test1 ~]#chkconfig --level 指定级别 network off
```

## 8.4、systemctl（CentOS 7 版本 重点掌握）

### 1）基本语法

```
systemctl start|stop|restart|status 服务名
```

> systemctl：system control的意思，系统控制

### 2）经验技巧

systemctl能够操作的服务名称位于`/usr/lib/systemd/system`目录，如下图

```
[root@test1 system]# ll -al /usr/lib/systemd/system
总用量 1544
drwxr-xr-x. 27 root root 20480 4月  23 00:13 .
drwxr-xr-x. 13 root root  4096 4月  23 00:12 ..
-rw-r--r--.  1 root root   275 10月  2 2020 abrt-ccpp.service
-rw-r--r--.  1 root root   380 10月  2 2020 abrtd.service
-rw-r--r--.  1 root root   361 10月  2 2020 abrt-oops.service
-rw-r--r--.  1 root root   266 10月  2 2020 abrt-pstoreoops.service
-rw-r--r--.  1 root root   262 10月  2 2020 abrt-vmcore.service
-rw-r--r--.  1 root root   311 10月  2 2020 abrt-xorg.service
-rw-r--r--.  1 root root   729 4月   1 2020 accounts-daemon.service
-rw-r--r--.  1 root root   569 8月   6 2019 alsa-restore.service
-rw-r--r--.  1 root root   465 8月   6 2019 alsa-state.service
-rw-r--r--.  1 root root   682 10月  2 2020 anaconda-direct.service
```

### 3）案例实操

#### （1）查看防火墙服务的状态

```
systemctrl status firewalld
```

```
[root@test1 ~]# systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; vendor preset: enabled)
   Active: active (running) since 三 2022-04-27 21:30:45 CST; 1 day 14h ago
     Docs: man:firewalld(1)
 Main PID: 834 (firewalld)
    Tasks: 2
   CGroup: /system.slice/firewalld.service
           └─834 /usr/bin/python2 -Es /usr/sbin/firewalld --nofork --nopid
```

### （2）停止防火墙服务

```
[root@test1 ~]# systemctl stop firewalld
```

### （3）启动防火墙服务

```
[root@test1 ~]# systemctl start firewalld
```

### （4）重启防火墙服务

```
[root@test1 ~]# systemctl restart firewalld
```

## 8.5、systemctl设置后台服务的自启动配置

### 1）基本语法

| 命令 | 描述 |
| --- | --- |
| systemctl list-unit-files | 查看服务开机启动状态 |
| systemctl disable service_name | 关掉指定服务的自动启动 |
| systemctl enable service_name | 开启指定服务的自动启动 |

### 2）案例实操

#### （1）查看服务开机启动状态

```
[root@test1 ~]# systemctl list-unit-files
UNIT FILE                                   STATE
proc-sys-fs-binfmt_misc.automount           static
dev-hugepages.mount                         static
dev-mqueue.mount                            static
proc-fs-nfsd.mount                          static
proc-sys-fs-binfmt_misc.mount               static
run-vmblock\x2dfuse.mount                   disabled
sys-fs-fuse-connections.mount               static
sys-kernel-config.mount                     static
sys-kernel-debug.mount                      static
tmp.mount                                   disabled
var-lib-nfs-rpc_pipefs.mount                static
brandbot.path                               disabled
cups.path                                   enabled
systemd-ask-password-console.path           static
systemd-ask-password-plymouth.path          static
systemd-ask-password-wall.path              static
session-1.scope                             static
session-c1.scope                            static
abrt-ccpp.service                           enabled
abrt-oops.service                           enabled
abrt-pstoreoops.service                     disabled
```

### （2）关闭防火墙服务的自动启动

```
[root@test1 ~]# systemctl disable firewalld
```

下面来测试一下，先把防火墙自启动关闭

```
[root@test1 ~]#
[root@test1 ~]# systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; vendor preset: enabled)
   Active: active (running) since 五 2022-04-29 22:42:29 CST; 3min 20s ago
     Docs: man:firewalld(1)
 Main PID: 853 (firewalld)
    Tasks: 2
   CGroup: /system.slice/firewalld.service
           └─853 /usr/bin/python2 -Es /usr/sbin/firewalld --nofork --nopid

4月 29 22:42:28 test1 systemd[1]: Starting firewalld - dynamic firewall daemon...
4月 29 22:42:29 test1 systemd[1]: Started firewalld - dynamic firewall daemon.
4月 29 22:42:29 test1 firewalld[853]: WARNING: AllowZoneDrifting is enabled. This is considered a
Hint: Some lines were ellipsized, use -l to show in full.
[root@test1 ~]# systemctl disable firewalld
Removed symlink /etc/systemd/system/multi-user.target.wants/firewalld.service.
Removed symlink /etc/systemd/system/dbus-org.fedoraproject.FirewallD1.service.
[root@test1 ~]# systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; disabled; vendor preset: enabled)
   Active: active (running) since 五 2022-04-29 22:42:29 CST; 3min 32s ago
     Docs: man:firewalld(1)
 Main PID: 853 (firewalld)
   CGroup: /system.slice/firewalld.service
           └─853 /usr/bin/python2 -Es /usr/sbin/firewalld --nofork --nopid

4月 29 22:42:28 test1 systemd[1]: Starting firewalld - dynamic firewall daemon...
4月 29 22:42:29 test1 systemd[1]: Started firewalld - dynamic firewall daemon.
4月 29 22:42:29 test1 firewalld[853]: WARNING: AllowZoneDrifting is enabled. This is considered a
Hint: Some lines were ellipsized, use -l to show in full.
[root@test1 ~]#
```

查看防火墙服务的状态
启用状态
这表示当前服务自启动是否开启
关闭防火墙服务自动启动

执行下面命令，重启机器

```
[root@test1 ~]# shutdown -r now
```

重启之后，执行下面命令，再看一下防火墙当前状态以及自启动状态，都是关闭的

```
[root@test1 ~]# systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; disabled; vendor preset: enabled)
   Active: inactive (dead)
     Docs: man:firewalld(1)
```

自启动被关闭了
目前防火墙的状态，被关闭状态

### （3）开启防火墙服务的自动启动

```
[root@test1 ~]# systemctl enable firewalld
```

## 8.6、系统运行级别

### 1）Linux运行级别【CentOS 6中】




查看默认级别：vi /etc/inittab

Linux系统有7种运行级别（runlevel）：常用的级别3和5

| 运行级别 | 说明 |
| --- | --- |
| 0 | 系统停机状态，系统默认运行级别不能设置为0，否则不能正常启动 |
| 1 | 单用户工作状态，root权限，用于系统维护，禁止远程登录 |
| 2 | 多户状态（没有NFS），不支持网络 |
| 3 | 完全的多用户状态（有NFS），登陆后进入控制台命令模式 |
| 4 | 系统未使用，保留 |
| 5 | X11控制台，登陆后进入图形GUI模式 |
| 6 | 系统正常关闭并重启，默认运行级别不能设置为6，否则不能正常启动 |

## 2）CentOS7 运行级别简化为

### multi-user.target：多用户有网，无图形界面

等价于原运行级别3

### graphical.target：多用户有网，有图形界面

等价于原运行级别5

## 3）systemctl get-default：查看当前运行级别

```
[root@test1 ~]# systemctl get-default
graphical.target
```

## 4）systemctl set-default：修改当前运行级别

```
systemctl set-default TARGET.target
```

（这里 TARGET 取 multi-user 或者 graphical）

如：

```
[root@test1 ~]# systemctl set-default multi-user.target
Removed symlink /etc/systemd/system/default.target.
Created symlink from /etc/systemd/system/default.target to
/usr/lib/systemd/system/multi-user.target.
```

# 8.7、关闭防火墙

## 1）临时关闭防火墙

### （1）查看防火墙状态

```
system status firewalld
```

### （2）临时关闭防火墙

```
system stop firewalld
```

## 2）开机启动时关闭防火墙

### （1）设置开机自动启动防火墙

```
systemctl enable firewalld.service
```

> 服务名称后面的.service可以省略不写

### （2）设置开机不启动防火墙

```
systemctl disable firewalld.service
```

# 8.8、关机重启命令

在 linux 领域内大多用在服务器上，很少遇到关机的操作。毕竟服务器上跑一个服务是永无止境的，除非特殊情况下，不得已才会关机。

## 1）基本语法

### sync：将数据由内存同步到硬盘中

linux中为提高效率，写磁盘的时候会先将数据写入缓冲器，缓冲区满了之后才会刷到磁盘，sync命令可以将缓冲区的数据立即写入磁盘。

### half：停机，关闭系统，但不断电

### poweroff：关机，断电

### reboot：重启，等同于shutdown -r now

### shutdown [选项] 时间

| 选项 | 功能 |
| --- | --- |
| -H | 相当于--half，停机 |
| -r | -r=reboot 重启 |

| 参数 | 功能 |
| --- | --- |
| now | 立刻关机 |
| 时间 | 等待多久后关机（单位是分钟） |

## 2）经验技巧

Linux 系统中为了提高磁盘的读写效率，对磁盘采取了 “预读迟写”操作方式。当用户保存文件时，Linux 核心并不一定立即将保存数据写入物理磁盘中， 而是将数据保存在缓冲区中，等缓冲区满时再写入磁盘，这种方式可以极大的提高磁盘写入数据的效率。但是，也带来了安全隐患，如果数据还未写入磁盘时，系统掉电或者其他严重问题出现，则将导致数据丢失。使用 sync 指令可以立即将缓冲区的数据写入磁盘。

## 3）案例实操

### （1）sync：将数据由内存同步到硬盘中

```
[root@test1 ~]# sync
```

### （2）reboot：重启

```
[root@test1 ~]# reboot
```

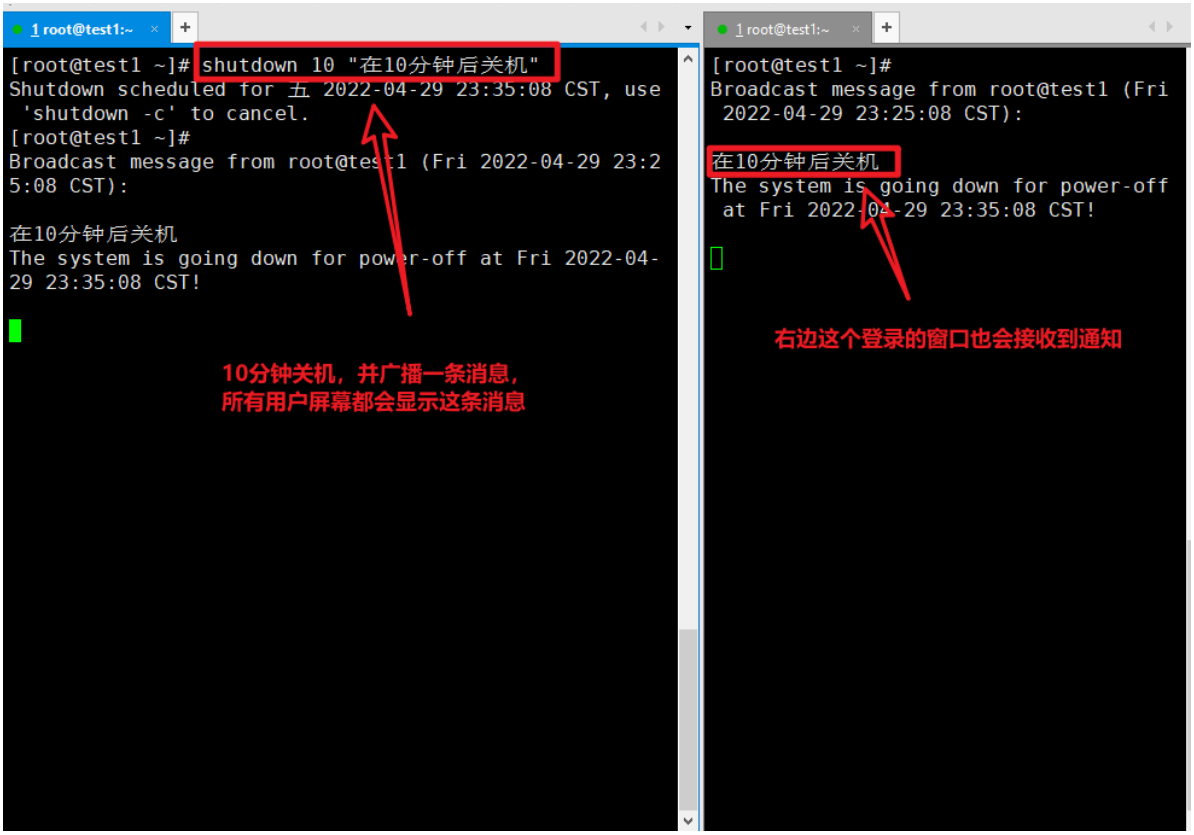
### （3）halt：停机（不断电）

```
[root@test1 ~]# halt
```

### （4）10分钟后关机，并广播消息

> 广播的消息所有登录的用户屏幕上都会显示。

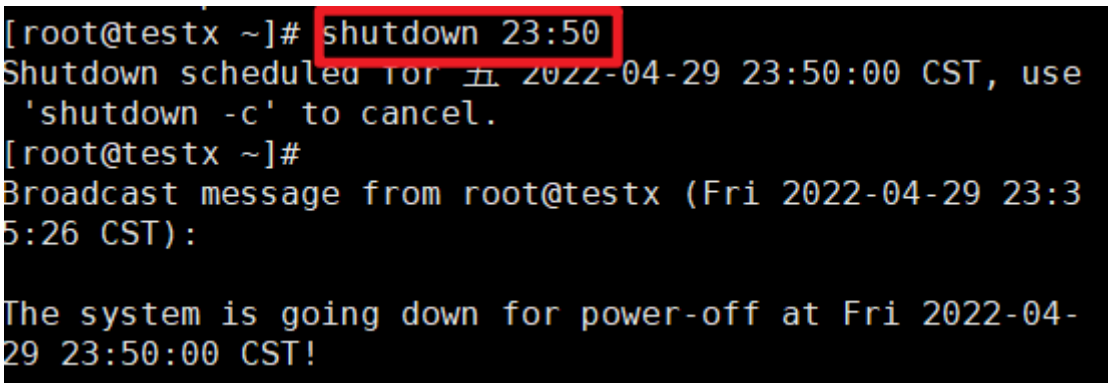
```
[root@test1 ~]# shutdown 10 "在10分钟后关机"
```



### (5) shutdown 时间点：指定时间点关机

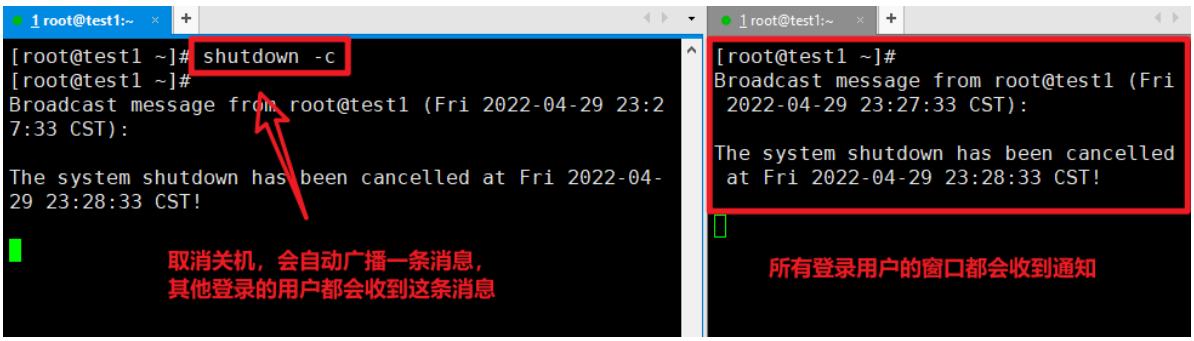
> 下面设置23点50分关机

```
[root@testx ~]# shutdown 23:50
```




### (6) shutdown -c：取消关机，并广播消息

```
[root@test1 ~]# shutdown -c
```




### (7) shutdown -h now：立马关机（等同于reboot）

```
[root@test1 ~]# shutdown -h now
```

### (8) shutdow -r now：立即重启（等同于reboot）

```
[root@test1 ~]# shutdown -r now
```


- 领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】
- 更多学习资料


# 9、帮助命令

## 9.1、man：获得命令帮助信息

#### 1) 基本语法

```
man [命令或配置文件]
```

> 获得命令帮助信息

#### 2) 显示说明

| 信息 | 功能 |
| --- | --- |
| NAME | 命令的名称和单行描述 |
| SYNOPSIS | 怎样使用命令 |
| DESCRIPTION | 命令功能的深入讨论 |
| EXAMPLES | 怎样使用命令的例子 |
| SEE ALSO | 相关主题（通常是手册页） |

**3）案例实操**

**（1）查看ls命令的帮助信息**

```
[root@testx ~]# man cd
```

**（2）使用经验**

man命令比较适合用来查看shell外部命令的帮助文档，内部命令是直接内嵌在shell中的，系统加载启动之后会随着shell一起加载，常驻系统内存中；其他的命令成为外部命令，外部命令有些比较大，用的时候才会去加载执行。

**（3）type：查看命令的类型**

如下，可以看出cd是shell内嵌的命令，说明cd是内部命令，而ls是外部命令。

```
[root@testx ~]# type cd
cd 是 shell 内嵌
[root@testx ~]# type ls
ls 是 `ls --color=auto' 的别名
```

如果用`man cd`查看内部命令cd的帮助信息，结果会显示成下面这样，看起来就比较懵了。

```
BASH(1)                  General Commands Manual                 BASH(1)

NAME
       bash - GNU Bourne-Again SHell (GNU 命令解释程序 " Bourne二世" )

概述(SYNOPSIS)
       bash [options] [file]

版权所有(COPYRIGHT)
       Bash  is  Copyright  (C)  1989-2002 by the Free Software Foundation,
       Inc.

描述(DESCRIPTION)
       Bash                                是一个与                        sh
       兼容的命令解释程序，可以执行从标准输入或者文件中读取的命令。     Bash
       也整合了 Korn 和 C Shell (ksh 和 csh) 中的优秀特性。

       Bash 的目标是成为遵循 IEEE POSIX Shell and Tools specification (IEEE
       Working Group 1003.2，可移植操作系统规约： shell 和工具) 的实现。
```

这是由于man查看cd命令的帮助文档时，有3个可选的帮助文档，man选择了第1个

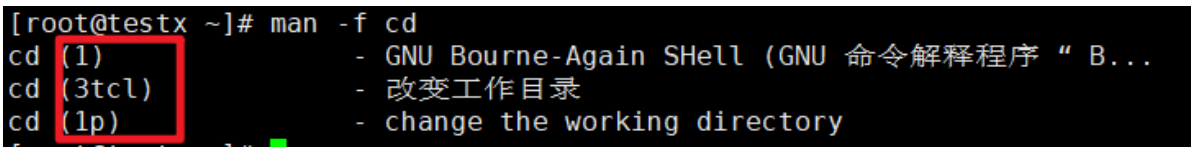
**（4）man -f 命令：列出命令的帮助文档列表**

比如我们查看cd命令的帮助文档列表，如下，列出了3个，man默认选择的是第一个，就是shell对应的那个，并不是我们希望看到的，我们希望看到的是第2个。

```
[root@testx ~]# man -f cd
cd (1)               - GNU Bourne-Again SHell (GNU 命令解释程序 “B...
cd (3tcl)            - 改变工作目录
cd (1p)              - change the working directory
```

**（5）man 文档编号 命令：用来查看某个命令指定的帮助文档**

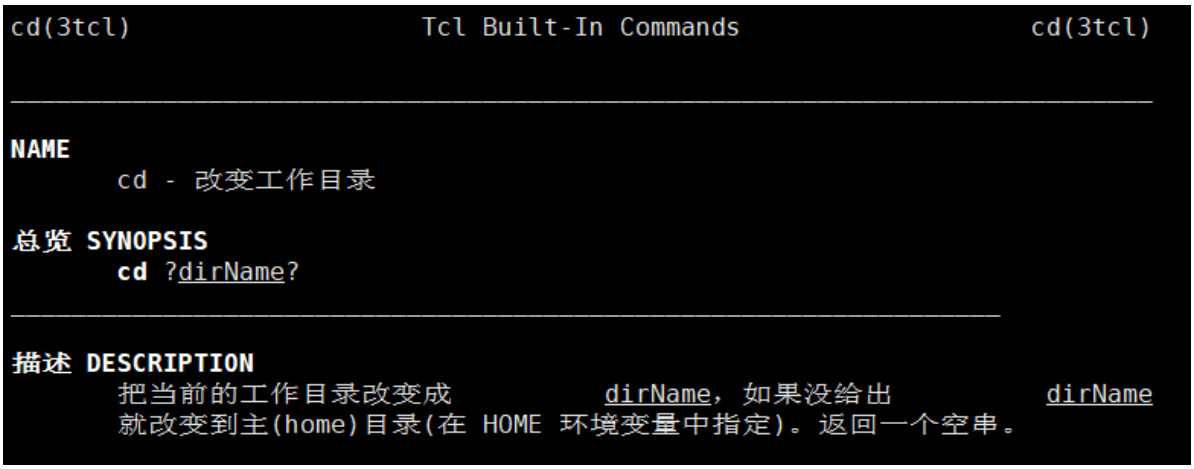
如下，man列出了cd有3个帮助文档，红色的部分是文档的编号




第2个文档是我们希望看的，对应的编号是：3tcl，man后面跟上文档编号即可查看这个文档，如下

```
[root@testx ~]# man 3tcl cd
```

效果如下




# 9.2、type：查看命令的类型【内部命令|外部命令】

一部分基础功能的系统命令是直接内嵌在 shell 中的，系统加载启动之后会随着 shell 一起加载，常驻系统内存中。这部分命令被称为“内置（built-in） 命令”； 相应的其它命令被称为“外部命令”。

**1）基本语法**

```
type 命令
```

> 查看命令的类型，如果结果中显示有“shell 内嵌”，表示是内置命令

**2）示例**

下面看cd、ls、mkdir这3个命令的类型，从下面可以看出cd是内部命令，其他两个是外部命令。

```
[root@testx ~]# type cd
cd 是 shell 内嵌
[root@testx ~]# type ls
ls 是 `ls --color=auto' 的别名
[root@testx ~]# type mkdir
mkdir 是 /usr/bin/mkdir
```

# 9.3、help：获取shell内置命令帮助信息

一部分基础功能的系统命令是直接内嵌在 shell 中的，系统加载启动之后会随着 shell 一起加载，常驻系统内存中。这部分命令被称为“内置（built-in） 命令”； 相应的其它命令被称为“外部命令”。

**1）基本语法**

```
help 命令
```

> 获得shell内置命令的帮助信息

**2）示例**

**（1）查看cd命令的帮助信息**

```
[root@testx ~]# help cd
```

# 9.4、命令 --help：查看外部命令的帮助信息

**1）语法**

```
命令 --help
```

**2）示例**

下面我们来查看外部命令ls的帮助文档

```
[root@testx ~]# ls --help
```

```
[root@testx ~]# ls --help
用法：ls [选项]... [文件]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
  -a, --all                  不隐藏任何以. 开始的项目
  -A, --almost-all           列出除. 及.. 以外的任何项目
      --author               与-l 同时使用时列出每个文件的作者
  -b, --escape               以八进制溢出序列表示不可打印的字符
      --block-size=SIZE      scale sizes by SIZE before printing them; e.g.,
                               '--block-size=M' prints sizes in units of
                               1,048,576 bytes; see SIZE format below
  -B, --ignore-backups       do not list implied entries ending with ~
  -c                         with -lt: sort by, and show, ctime (time of last
                               modification of file status information);
                               with -l: show ctime and sort by name;
                               otherwise: sort by ctime, newest first
  -C                         list entries by columns
      --color[=WHEN]         colorize the output; WHEN can be 'never', 'auto'
,
                               or 'always' (the default); more info below
  -d, --directory            list directories themselves, not their contents
```

# 9.5、clear：清屏

比如目前屏幕是下面这样，可以输入clear命令，清屏

```
[root@testx ~]# ls
15     2.txt  anaconda-ks.cfg       公共  视频  文档  音乐
1.txt  a      initial-setup-ks.cfg  模板  图片  下载  桌面
[root@testx ~]# dir
15     2.txt  anaconda-ks.cfg       公共  视频  文档  音乐
1.txt  a      initial-setup-ks.cfg  模板  图片  下载  桌面
[root@testx ~]# clear
```

clear命令执行后，屏幕被清理干净了，对应的快捷键是`ctrl+l`，如下

```
[root@testx ~]#
```

但是这种清屏只是将的内容向上滚动了而已，并没有真正意义上的清屏，如果要完全清屏，需要用reset命令。

## 9.6、reset：彻底清屏

执行reset命令后，命令窗口会被重启，会彻底清屏。

reset执行前

```
[root@testx ~]# ls
15     2.txt  anaconda-ks.cfg       公共  视频  文档  音乐
1.txt  a      initial-setup-ks.cfg  模板  图片  下载  桌面
[root@testx ~]# dir
15     2.txt  anaconda-ks.cfg       公共  视频  文档  音乐
1.txt  a      initial-setup-ks.cfg  模板  图片  下载  桌面
[root@testx ~]# reset
```

执行后，如下，被彻底清屏了

```
[root@testx ~]#
```

## 9.7、常用快捷键

| 常用快捷键 | 功能 |
| --- | --- |
| ctrl+c | 停止进程 |
| ctrl+l | 清屏，等同于clear；彻底清屏是：reset |
| 善于用tab键 | 提示（更重要的是可以防止敲错） |
| 上下键 | 查找执行过的命令 |


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 10、文件目录类命令

## 10.1、pwd：显示当前工作目录的绝对路径

**1）基本语法**

```
pwd
```

**2）案例**

```
[root@testx 桌面]# pwd
/root/桌面
```

# 10.2、ls：列出目录的内容

**1）基本语法**

```
ls [选项] [目录或是文件]
```

**2）选项说明**

| 选项 | 说明 |
| --- | --- |
| -a | 全部文件，连同隐藏文件（开头为.的文件）一起列出来（常用） |
| -l | 长数据串列出，包含文件的属性与权限等数据（常用），等价于ll命令 |
| 目录参数非必须 | 命令后面的目录可以省略，省略时，查看的是当前目录 |

**3）显示说明**

每行出的信息依次是：

- 文件类型与权限
- 连接数
- 文件属主
- 文件属组
- 文件大小(byte)
- 创建或最近修改的时间
- 名字

**4）案例**

**（1）查看当前目录所有的内容信息**

```
[root@testx a]# ls -al
总用量 4
drwxr-xr-x.  4 root root   29 5月   1 11:32 .
dr-xr-x---. 15 root root 4096 4月  30 23:20 ..
drwxr-xr-x.  2 root root    6 5月   1 11:32 .1.txt
drwxr-xr-x.  3 root root   15 4月  30 17:47 b
```

**（2）查看目录/usr的文件信息**

```
[root@testx a]# ls /usr/
bin  etc  games  include  lib  lib64  libexec  local  sbin  share  src  tmp
```

# 10.3、cd：切换目录

**1）基本语法**

```
cd [参数]
```

### 2）参数说明

| 参数 | 功能 |
| --- | --- |
| cd 绝对路径 | 切换路径 |
| cd 相对路径 | 切换路径 |
| cd ~或cd | 回到自己的家目录 |
| cd - | 回到上一次所在目录，可以用来在2个目录来回跳转 |
| cd .. | 回到当前目录的上一级目录 |
| cd -P | 跳转到实际物理路径，而非快捷方式路径 |

### 3）案例

#### （1）使用绝对路径切换到/usr/local目录

```
[root@testx 桌面]# cd /usr/local/
```

#### （2）使用相对路径切换到“公共的”目录

> 如下，当前在/root目录，我们想到root目录下面的`桌面`目录，那么可以直接使用`cd 桌面`即可，这种情况下使用相对路径更方便，相对路径是相对于当前目录来说的。

```
[root@testx ~]# pwd
/root
[root@testx ~]# ls
15     2.txt  anaconda-ks.cfg        公共  视频  文档  音乐
1.txt  a      initial-setup-ks.cfg  模板  图片  下载  桌面
[root@testx ~]# cd 桌面
[root@testx 桌面]#
```

#### （3）回到当前用户自己的家目录

> 如下，当前是root用户，目前在`/usr/local`目录，执行cd直接回到了root用户自己的目录，即`/root`目录

```
[root@testx local]# pwd
/usr/local
[root@testx local]# cd
[root@testx ~]# pwd
/root
```

#### （4）cd - 回到上一次所在目录

> 下面命令，先通过cd跳到/usr/local目录，然后又通过cd跳到/root目录，执行cd - 回到上一次所在的目录，即/usr/local目录，多次执行cd -，可以在两个目录之间来回切换，还是很实用的一个命令

```
# 跳到/usr/local目录
[root@testx local]# cd /usr/local/
[root@testx local]# pwd
```

```
/usr/local
# 跳到/root目录
[root@testx local]# cd /root
# 回到上一次的目录，即/usr/local目录
[root@testx ~]# cd -
/usr/local
[root@testx local]# cd -
/root
[root@testx ~]# cd -
/usr/local
[root@testx local]# cd -
/root
[root@testx ~]# cd -
/usr/local
```

**（5）cd ..回到上一级目录**

> 下面命令，从/usr/local目录通过cd ..回到上一级目录，即/usr目录

```
[root@testx local]# cd /usr/local/
[root@testx local]# pwd
/usr/local
[root@testx local]# cd ..
[root@testx usr]# pwd
/usr
```

# 10.4、mkdir：创建一个新的目录

**1）基本语法**

```
mkdir [选项] 要创建的目录
```

**2）选项说明**

| 选项 | 说明 |
| --- | --- |
| -p | 创建多层目录 |

**3）案例**

**（1）创建一个目录test**

```
[root@testx ~]# mkdir test
```




**（2）创建多级目录**

> 递归创建多级目录需要加-p选项，否则，父目录不存在时会报错

```
[root@testx ~]# mkdir a/b/c/d
mkdir: 无法创建目录"a/b/c/d": 没有那个文件或目录
[root@testx ~]# mkdir -p a/b/c/d
```

# 10.5、rmdir：删除一个空目录

**1）基本语法**

```
rmdir 要删除的空目录
```

**2）案例**

**（1）删除一个空目录**

```
[root@testx ~]# rmdir c
```

**（2）删除非空目录会失败**

```
[root@testx ~]# rmdir a
rmdir: 删除 "a" 失败: 目录非空
```

# 10.6、touch：创建空文件

**1）基本语法**

```
touch 文件名称
```

**2）案例**

> 当前目录创建1.txt文件

```
[root@testx ~]# touch 1.txt
```

# 10.7、cp：复制文件或目录

**1）基本语法**

> 复制source文件到dest

```
cp [选项] source dest
```

**2）选项说明**

| 选项 | 说明 |
| --- | --- |
| -r | 递归复制整个文件夹 |

**3）参数说明**

| 参数 | 说明 |
| --- | --- |
| source | 源文件 |
| dest | 目标文件 |

#### 4）经验技巧

强制覆盖不提示的方法：`\cp`，因为cp命令是`cp -i`的别名，`-i`选项的作用是当出现文件覆盖的时候需要提示用户选择是否覆盖，这也是为了安全起见，而`\命令`可以按照命令原始的方式运行。

#### 5）案例

##### （1）复制文件

```
# 当前目录创建1.txt文件
[root@testx ~]# touch 1.txt
# 将1.txt文件复制到/tmp/目录
[root@testx ~]# cp 1.txt /tmp/
```

##### （2）递归复制整个文件夹

```
# 创建目录a
[root@testx ~]# mkdir a
# 在a目录中创建2个txt文件
[root@testx ~]# touch a/1.txt
[root@testx ~]# touch a/2.txt
# 查看a目录内容
[root@testx ~]# ls a
1.txt  2.txt
# 递归复制a目录所有文件到b目录
[root@testx ~]# cp -r a b
# 查看b目录内容
[root@testx ~]# ls b
1.txt  2.txt
```

##### （3）\cp：复制，覆盖不提示

```
# 当前目录创建1.txt文件
[root@testx ~]# touch 1.txt
# 将1.txt复制到/tmp目录
[root@testx ~]# cp 1.txt /tmp/
# 将1.txt复制到/tmp目录，由于已存在，会提示是否覆盖？
[root@testx ~]# cp 1.txt /tmp/
cp：是否覆盖"/tmp/1.txt"？ y
# 使用\cp复制，存在的情况下，会直接覆盖
[root@testx ~]# \cp 1.txt /tmp/
```

## 10.8、rm：删除文件或目录

#### 1）基本语法

> 删除文件或目录

```
rm [选项] 文件1 文件2 [文件N...]
```

### 2）选项说明

| 选项 | 说明 |
| --- | --- |
| -r | 递归删除目录中所有内容 |
| -f | 强制执行删除操作，不会提示用户是否确认删除 |
| -v | 显示命令的详细执行过程 |

### 3）案例

#### （1）删某个文件

```
# 创建2个txt文件
[root@testx ~]# touch 1.txt 2.txt
# 查看目录内容，2个文件已创建好了
[root@testx ~]# ls
1.txt  2.txt  公共  模板  视频  图片  文档  下载  音乐  桌面
# 使用rm删除1.txt，会提示用户是否确认删除，输入y即可删除
[root@testx ~]# rm 1.txt
rm：是否删除普通空文件 "1.txt"？y
# 删除2.txt，输入n，表示不同意删除
[root@testx ~]# rm 2.txt
rm：是否删除普通空文件 "2.txt"？n
# 使用rm -f可以强制删除，不会提示确认信息
[root@testx ~]# rm -f 2.txt
```

#### （2）递归删除目录所有内容

```
# 递归创建a/b/c/d目录
[root@testx ~]# mkdir -p a/b/c/d
# 递归强制删除a目录及内部所有内容
[root@testx ~]# rm -rf a
```

#### （3）删除多个文件，且输出提示信息

```
# 创建3个目录
[root@testx ~]# mkdir a1 a2 a3
# 递归&强制&输出删除信息，删除3个目录
[root@testx ~]# rm -rfv a1 a2 a3
已删除目录："a1"
已删除目录："a2"
已删除目录："a3"
```

#### （4）*通配符删除

> **rm -rf /***：此命令慎用，否则永无回头之日。

```
# 创建3个目录：a1，a2，a3
[root@testx ~]# mkdir a1 a2 a3
# ls查看目录信息，3个目录创建好了
[root@testx ~]# ls
a1  a2  a3  公共  模板  视频  图片  文档  下载  音乐  桌面
# 强制&递归删除以文件名称以a开头的所有文件
[root@testx ~]# rm -rf a*
# ls查看目录信息，3个a开头的文件都被删掉了
[root@testx ~]# ls
公共  模板  视频  图片  文档  下载  音乐  桌面
```

# 10.9、mv：移动文件与目录或重命名

**1）基本语法**

```
mv oldNameFile newNameFile（重命名文件）
mv /temp/movefile /targetFolder (移动文件)
```

**2）案例**

**（1）重命名**

> 将1.txt重命名为2.txt

```
[root@testx a]# mv 1.txt 2.txt
```

**（2）移动文件**

```
# 当前目录创建1.txt文件
[root@testx a]# touch 1.txt
# 当前目录创建a目录
[root@testx a]# mkdir a
# 查看当前目录内容
[root@testx a]# ls
1.txt  a
# 将1.txt文件移到a目录中
[root@testx a]# mv 1.txt a
# 查看目录a的内容，可以看到有1.txt了
[root@testx a]# ls a
1.txt
```

**（3）mv a b**

- 若a和b都是目录，b不存在时，相当于把a目录命令为b目录
- 若a和b都是目录，b存在的时，相当于把a目录移动到b目录中

# 10.10、cat：查看文件内容

**1）基本语法**

```
cat [选项] 要查看的文件
```

**2）选项说明**

| 选项 | 说明 |
| --- | --- |
| -n | 显示所有行的行号，也包含空行 |

**3）经验技巧**

一般查看比较小的文件，一屏幕能显示全的。

**4）案例**

```
[root@testx ~]# cat 1.txt
个人博客：http://www.itsoku.com
上面有很多java资料和精品文章
欢迎大家访问
[root@testx ~]# cat -n 1.txt
     1  个人博客：http://www.itsoku.com
     2  上面有很多java资料和精品文章
     3  欢迎大家访问
```

# 10.11、more：文件内容分屏查看器

适合查看内容超过一个屏幕的文件，支持上下翻页查看文件内容。

more命令是一个基于VI编辑器的文本过滤器，它以全屏幕的方式按页显示文本文件的内容，more命令中内置了若快捷键，详见操作说明。

**1）基本语法**

```
more 要查看的文件
```

**2）操作说明**

| 操作 | 说明 |
| --- | --- |
| 空白键(space) | 向下翻一页 |
| b | 向上翻一页 |
| Enter | 向下翻【一行】 |
| q | 代表立刻离开more，不再显示该文件内容 |
| = | 输出当前行的行号 |
| :f | 输出文件名和当前行的行号 |

**3）示例**

> 使用more查看2.txt文件内容

```
[root@testx ~]# more 2.txt
```

# 10.12、less：分屏显示文件内容

less指令用来分屏查看文件内容，它的功能与more指令类似，但是比more指令更加强大，支持各种显示终端，less指令在显示文件内容时，并不是一次将整个文件加载之后才显示，而是根据显示需要去加载的内容，对于显示大型文件具有较高的效率。

**1）基本语法**

```
less 需要查看的文件
```

**2）操作说明**

| 操作 | 说明 |
| --- | --- |
| 空白键（space） | 向下翻动一页 |
| [pagedown] 或 f | 上下翻动一页 |
| [pageup] 或 b | 向上翻动一页 |
| /字符串 | 向下搜寻【字符串】的功能；n：向下查找；N：向上查找 |
| ?字符串 | 向上搜索【字符串】的功能，n：向上查找，N：向下查找 |
| q | 离开less这个程序 |

**3）经验技巧**

用SecureCRT时候[pagedown]和[pageup]可能会出现无法识别的问题

**4）案例**

> 使用less查看2.txt的内容，这个文件比较大

```
[root@testx ~]# less 2.txt
```

# 10.13、echo：输出内容到控制台

**1）基本语法**

```
echo [选项] [输出内容]
```

选项：

-e：支持反斜线的字符转换

| 控制字符 | 作用 |
| --- | --- |
| `\\` | 输出\本身 |
| \n | 换行符 |
| \t | 制表符，也就是Tab键 |

**2）案例**

```
[root@testx ~]# echo "hello\tworld"
hello\tworld
[root@testx ~]# echo -e "hello\tworld"
hello    world
[root@testx ~]# echo -e "hello\nworld"
hello
world
```

# 10.14、head：显示文件头部内容

head用于显示文件的开头部分内容，默认情况下head指令显示文件的前10行内容。

**1）基本语法**

```
head 文件 （功能描述：查看文件头10行内容）
head -n 5 文件 （功能描述：查看文件头5行内容，5可以是任意行数）
```

**2）选项说明**

| 选项 | 功能 |
| --- | --- |
| -n 行数 | 指定显示头部内容的行数 |

**3）案例**

> 使用head命令查看1.txt前2行内容

```
[root@testx ~]# cat 1.txt
个人博客：http://www.itsoku.com
上面有很多java资料和精品文章
欢迎大家访问
[root@testx ~]# head -n 2 1.txt
个人博客：http://www.itsoku.com
上面有很多java资料和精品文章
```

# 10.15、tail：输出文件尾部内容

tail用于输出文件中尾部的内容，默认情况下tail指令显示文件后10行内容。

还可以用来实时监控文件的变更。

**1）基本语法**

| 语法 | 说明 |
| --- | --- |
| tail 文件 | 查看文件尾部10行内容 |
| tail -n 5 文件 | 查看文件尾部5行内容，5可以是任意行数 |
| tail -f 文件 | 实时追踪该文档的所有更新 |

**2）案例**

（1）查看1.txt文件末尾1行内容

```
[root@testx ~]# tail -n 1 1.txt
```

（2）实时追踪1.txt的内容，当有更新的时候，当前屏幕会自动输出所有的更新，通常用来查看日志文件变化，非常有用

```
[root@testx ~]# tail -f 1.txt
```

# 10.16、>输出重定向 和 >>追加

**1）基本语法**

| 语法 | 说明 |
| --- | --- |
| ls -l > 目标文件 | 将ls -l命令的结果写入到目标文件中（会覆盖目标文件内容） |
| ls -al >> 目标文件 | 将ls -al命令的结果追加到目标文件中 |
| cat 文件1 > 文件2 | 将文件1的内容覆盖到文件2中 |
| echo "内容" >> 目标文件 | 将一段文字追加到目标文件 |

**2）案例**

**（1）将`ll命令结果`结果写入到1.txt文件**

```
[root@testx ~]# ll
总用量 0
drwxr-xr-x. 2 root root 6 4月  23 00:21 公共
drwxr-xr-x. 2 root root 6 4月  23 00:21 模板
drwxr-xr-x. 2 root root 6 4月  23 00:21 视频
drwxr-xr-x. 2 root root 6 4月  23 00:21 图片
drwxr-xr-x. 2 root root 6 4月  23 00:21 文档
drwxr-xr-x. 2 root root 6 4月  23 00:21 下载
drwxr-xr-x. 2 root root 6 4月  23 00:21 音乐
drwxr-xr-x. 2 root root 6 4月  23 00:21 桌面
[root@testx ~]# ll > 1.txt
[root@testx ~]# cat 1.txt
总用量 0
-rw-r--r--. 1 root root 0 5月   2 14:10 1.txt
drwxr-xr-x. 2 root root 6 4月  23 00:21 公共
drwxr-xr-x. 2 root root 6 4月  23 00:21 模板
drwxr-xr-x. 2 root root 6 4月  23 00:21 视频
drwxr-xr-x. 2 root root 6 4月  23 00:21 图片
drwxr-xr-x. 2 root root 6 4月  23 00:21 文档
drwxr-xr-x. 2 root root 6 4月  23 00:21 下载
drwxr-xr-x. 2 root root 6 4月  23 00:21 音乐
drwxr-xr-x. 2 root root 6 4月  23 00:21 桌面
```

**（2）将`ls命令结果`追加到1.txt文件**

```
[root@testx ~]# ls >> 1.txt
```

**（3）采用echo将hello单词追加到1.txt中**

```
[root@testx ~]# echo hello >> 1.txt
```

# 10.17、ln：创建软连接

软连接也称为符号链接，类似于windows里的快捷方式，有自己的数据块，主要存放了连接其他文件的路径。

**1）基本语法**

> 给源文件创建一个软连接

```
ln -s [源文件或目录] [软连接名称]
```

**2）经验**

删除软连接：rm -rf 软连接名称，而不是 rm -rf 软连接名称/

如果使用rm -rf 软连接名/ 删除，会把软连接对应的真实目录下内容删除。

查询：通过ll就可以查看，列表属性第1位是1，尾部会有位置指向。

**3）案例**

**（1）创建软连接**

> 在/tmp目录创建一个快捷方式desktop，指向/root/桌面

```
[root@testx tmp]# cd /tmp/
[root@testx tmp]# ln -s /root/桌面 ./desktop
[root@testx tmp]# ll desktop
lrwxrwxrwx. 1 root root 12 5月   2 14:53 desktop -> /root/桌面
[root@testx tmp]# cd desktop/
[root@testx desktop]# pwd
/tmp/desktop
# pwd -P：查看当前快捷方式对应的物理路径
[root@testx desktop]# pwd -P
/root/桌面
```

**（2）cd -P：进入软连接实际物理路径**

```
[root@testx tmp]# ln -s /root/桌面 ./desktop
[root@testx tmp]# cd -P desktop/
[root@testx 桌面]# pwd
/root/桌面
```

# 10.18、history：查看已执行过的历史命令

**1）基本语法**

```
history
```

**2）案例**

**（1）查看已执行过的命令**

> history可以查看已执行过的历史命令，返回的结果：命令编号 命令

```
[root@testx ~]# history
    1  pwd
    2  exit
    3  pwd
    4  ll
    5  mkdir 123
    6  ll
    7  rm 123
    8  rm -rf 123
```

**(2) `!命令编号`，可以执行对应编号的命令**

> 上面编号为5对应的命令是`mkdir 123`，那么我们可以使用`!5`再次把`mkdir 123`命令执行一遍

```
[root@testx ~]# !5
mkdir 123
```

**(3) history n：查看最近的n行历史命令**

```
[root@testx desktop]# history
    1  history
    2  pwd
    3  echo "ok"
    4  ls
    5  history
[root@testx desktop]# history 3
    4  ls
    5  history
    6  history 3
```

**(4) history -c：清除命令历史记录**

> 如果我们不想让别人看到命令的历史记录，可以使用history -c来清除命令记录。

```
[root@testx desktop]# history
    1  history
    2  pwd
    3  echo "ok"
    4  ls
    5  history
    6  history 3
    7  history
[root@testx desktop]# history -c
[root@testx desktop]# history
    1  history
```

# 10.19、tree：树形显示目录所有文件

tree命令在系统中默认是不存在的，所以使用前需要先安装，然后才可以使用。

**1）安装tree软件包**

执行下面命令进行安装，过程中提示输入的地方输入y就可以

```
[root@testx b]# yum install tree
已加载插件：fastestmirror, langpacks
Loading mirror speeds from cached hostfile
 * base: mirrors.njupt.edu.cn
 * extras: ftp.sjtu.edu.cn
 * updates: mirrors.njupt.edu.cn
正在解决依赖关系
--> 正在检查事务
---> 软件包 tree.x86_64.0.1.6.0-10.el7 将被 安装
--> 解决依赖关系完成

依赖关系解决

================================================================================
================================================================
 Package                          架构                                  版本
                          源                                  大小
================================================================================
================================================================
正在安装:
 tree                             x86_64                             1.6.0-10.el7
                           base                                46 k

事务概要
================================================================================
================================================================
安装  1 软件包

总下载量：46 k
安装大小：87 k
Is this ok [y/d/N]: y
Downloading packages:
tree-1.6.0-10.el7.x86_64.rpm
                                          |  46 kB  00:00:00
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  正在安装     : tree-1.6.0-10.el7.x86_64
                                                                1/1
  验证中       : tree-1.6.0-10.el7.x86_64
                                                                1/1

已安装:
  tree.x86_64 0:1.6.0-10.el7


完毕!
```

**2）语法**

```
tree [选项] [目录]
```

> 树形显示目录中的文件。
>
> 目录可以省略，默认为当前目录

### 3）选项

| 选项 | 功能 |
| --- | --- |
| -h | 以人容易理解的方式显示文件大小（GBytes、MBytes、KBytes） |
| -L n | n用来指定显示的树的深度 |

### 4）tree命令的使用

```
# 默认递归按照树形显示所有文件信息
[root@testx b]# tree
.
├── 1.tar.gz
├── log.tar.gz
└── target
    └── logs
        ├── 1.log
        └── 2.log

2 directories, 4 files

# -h选项：会同时显示文件大小（以人容易理解的方式展示文件占用磁盘的大小）
[root@testx b]# tree -h
.
├── [197K]  1.tar.gz
├── [ 98K]  log.tar.gz
└── [  18]  target
    └── [  32]  logs
        ├── [197K]  1.log
        └── [197K]  2.log

2 directories, 4 files

# -L n：n用来指定树的深度，比如下面只展示一层
[root@testx b]# tree -L 2
.
├── 1.tar.gz
├── log.tar.gz
└── target
    └── logs

2 directories, 2 files

# 多个选项可以同时使用
[root@testx b]# tree -Lh 2
.
├── [197K]  1.tar.gz
├── [ 98K]  log.tar.gz
└── [  18]  target
    └── [  32]  logs

2 directories, 2 files
```


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

# 11、时间日期类命令

## 11.1、date：日期命令

**1）基本语法**

```
date [OPTION]...[+FORMAT]
```

**2）选项说明**

| 选项 | 功能 |
| --- | --- |
| -d <时间字符串> | 显示指定的“时间字符串”表示的时间，而非当前时间 |
| -s <日期时间> | 设置系统日期时间 |

**3）参数说明**

| 参数 | 功能 |
| --- | --- |
| <+日期时间格式> | 指定显示时使用的日期时间格式 |

## 11.2、date：显示当前时间

**1）基本语法**

| 命令 | 说明 |
| --- | --- |
| date | 显示当前时间 |
| date +%Y | 显示当前年份 |
| date +%m | 显示当前月份 |
| date +%d | 显示当前是本月哪一天 |
| date "+%Y-%m-%d %H:%M:%S" | 显示年月日时分秒 |

**2）案例实操**

> 当前时间是`2022-05-02 19:53:42`，注意下面6个命令的输出。

```
[root@testx desktop]# date
2022年 05月 02日 星期一 19:53:08 CST
[root@testx desktop]# date +%Y
2022
[root@testx desktop]# date +%y
22
[root@testx desktop]# date +%m
05
[root@testx desktop]# date +%d
02
[root@testx desktop]# date "+%Y-%m-%d %H:%M:%S"
2022-05-02 19:53:42
```

# 11.3、date：显示非当前时间

1）基本语法

| 命令 | 说明 |
| --- | --- |
| date -d '1 days ago' | 显示前一天时间 |
| date -d '-1 days ago' | 显示明天时间 |

**2）案例**

**（1）今天、昨天、明天**

```
[root@testx desktop]# date
2022年 05月 02日 星期一 20:25:10 CST
[root@testx desktop]# date -d '1 days ago'
2022年 05月 01日 星期日 20:25:21 CST
[root@testx desktop]# date -d '-1 days ago'
2022年 05月 03日 星期二 20:25:24 CST
```

**（2）当前时间、一小时前、一小时后**

> days改成hours便可

```
[root@testx desktop]# date
2022年 05月 02日 星期一 20:26:28 CST
[root@testx desktop]# date -d '1 hours ago'
2022年 05月 02日 星期一 19:26:36 CST
[root@testx desktop]# date -d '-1 hours ago'
2022年 05月 02日 星期一 21:26:39 CST
```

# 11.4、date -s：设置系统时间

**1）基本语法**

date -s 字符串时间

**2）案例**

> 设置系统当前时间

```
[root@testx desktop]# date -s '2021-12-12 20:50:10'
```

# 11.5、cal：查看日历

### 1）基本语法

```
cal [选项]
```

> 不加选项，显示月日历

### 2）选项说明

| 选项 | 说明 |
| --- | --- |
| 具体某一年 | 显示这一年的日历 |

### 3）案例

#### （1）查看当前月的日历

```
[root@testx desktop]# cal
      五月 2022
日 一 二 三 四 五 六
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31
```

#### （2）查看2018年的日历

```
[root@testx desktop]# cal 2018
                               2018

        一月                   二月                   三月
日 一 二 三 四 五 六   日 一 二 三 四 五 六   日 一 二 三 四 五 六
    1  2  3  4  5  6                1  2  3                1  2  3
 7  8  9 10 11 12 13    4  5  6  7  8  9 10    4  5  6  7  8  9 10
14 15 16 17 18 19 20   11 12 13 14 15 16 17   11 12 13 14 15 16 17
21 22 23 24 25 26 27   18 19 20 21 22 23 24   18 19 20 21 22 23 24
28 29 30 31            25 26 27 28            25 26 27 28 29 30 31

        四月                   五月                   六月
日 一 二 三 四 五 六   日 一 二 三 四 五 六   日 一 二 三 四 五 六
 1  2  3  4  5  6  7          1  2  3  4  5                   1  2
 8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
29 30                  27 28 29 30 31         24 25 26 27 28 29 30

        七月                   八月                   九月
日 一 二 三 四 五 六   日 一 二 三 四 五 六   日 一 二 三 四 五 六
 1  2  3  4  5  6  7             1  2  3  4                      1
 8  9 10 11 12 13 14    5  6  7  8  9 10 11    2  3  4  5  6  7  8
15 16 17 18 19 20 21   12 13 14 15 16 17 18    9 10 11 12 13 14 15
```

```
22 23 24 25 26 27 28   19 20 21 22 23 24 25   16 17 18 19 20 21 22
29 30 31               26 27 28 29 30 31      23 24 25 26 27 28 29
                                              30
        十月                    十一月                  十二月
日 一 二 三 四 五 六   日 一 二 三 四 五 六   日 一 二 三 四 五 六
    1  2  3  4  5  6                1  2  3                     1
 7  8  9 10 11 12 13    4  5  6  7  8  9 10    2  3  4  5  6  7  8
14 15 16 17 18 19 20   11 12 13 14 15 16 17    9 10 11 12 13 14 15
21 22 23 24 25 26 27   18 19 20 21 22 23 24   16 17 18 19 20 21 22
28 29 30 31            25 26 27 28 29 30      23 24 25 26 27 28 29
                                              30 31
```

- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

# 12、用户管理命令

## 12.1、useradd：添加新用户

**1）基本语法**

| 语法 | 说明 |
| --- | --- |
| useradd 用户名 | 添加新用户 |
| useradd -g 组名 用户名 | 添加新用户到某个组 |

**2）案例**

**（1）添加一个新用户：tom**

> 添加用户时，默认会在/home目录生成一个和用户名同名的用户的家目录

```
[root@testx ~]# useradd tom
[root@testx ~]# ll /home
总用量 0
drwx------. 3 tom tom 78 5月   3 16:31 tom
```

## 12.2、passwd：设置用户密码

**1）基本语法**

| 语法 | 说明 |
| --- | --- |
| passwd 用户名 | 为指定的用户设置密码 |

**2）案例**

**（1）给用户tom设置密码**

> 会提示输入密码，输入过程屏幕上不会显示。

```
[root@testx ~]# passwd tom
```

## 12.3、id：查看用户是否存在

### 1）基本语法

```
id 用户名
```

### 2）案例

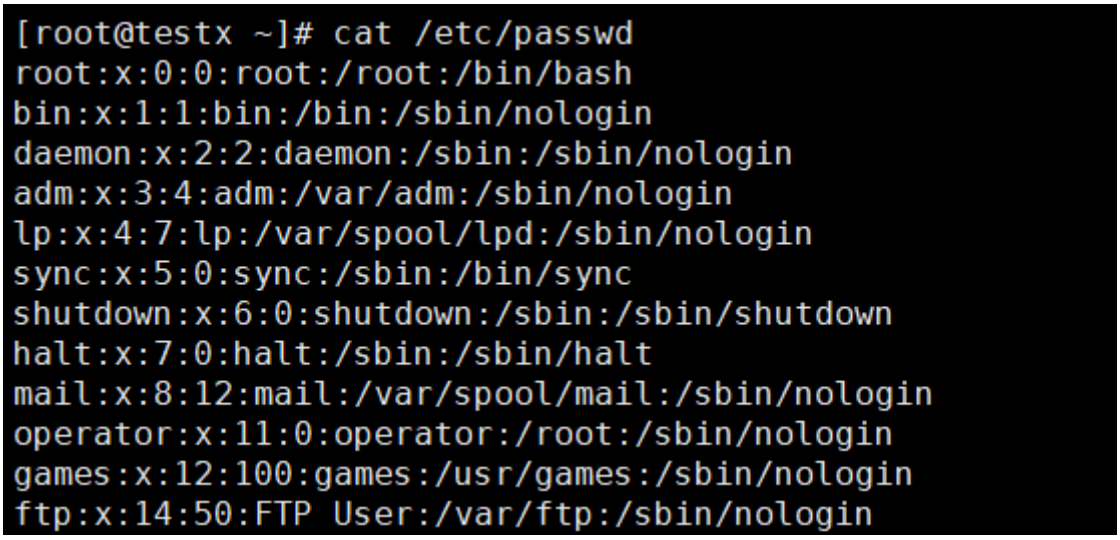
#### （1）查看tom、jack是否存在

```
[root@testx ~]# id tom
uid=1001(tom) gid=1001(tom) 组=1001(tom)
[root@testx ~]# id jack
id: jack: no such user
```

## 12.4、cat /etc/passwd：查看创建了哪些用户

> 所有创建的用户在`/etc/passwd`文件中都会有记录

```
[root@testx ~]# cat /etc/passwd
```




## 12.5、su：切换用户

su：swith user的意思，表示切换用户

### 1）基本语法

| 语法 | 说明 |
| --- | --- |
| su 用户名 | 切换到目标用户，只能获得目标用户的执行权限，不能获得其环境变量 |
| su - 用户名 | 切换到目标用户并获得目标用户的环境变量及执行权限 |

### 2）案例

> 当前是root用户，这root用户下，去执行`su`和`su -`的切换用户的效果，`echo $PATH`用来输出环境变量PATH的信息，注意看环境变量的值有什么不同

```
[root@testx ~]# su tom
[tom@testx root]$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/root/bin
[tom@testx root]$ exit
exit
[root@testx ~]# su - tom
上一次登录：二 5月  3 16:43:17 CST 2022pts/0 上
[tom@testx ~]$ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/tom/.local/bin:/hom
e/tom/bin
```

# 12.6、userdel：删除用户

**1）基本语法**

| 语法 | 说明 |
| --- | --- |
| userdel 用户名 | 删除用户，但会保留用户的主目录（即家目录） |
| userdel -r 用户名 | 删除用户及其主目录（即家目录） |

**2）选项说明**

| 选项 | 说明 |
| --- | --- |
| -r | 删除用户的同事，删除与用户相关的所有文件 |

**3）案例**

**（1）删除用户但保留用户主目录**

```
# 创建用户user1
[root@testx ~]# useradd user1
# /home中产生了一个user1目录，是user1的家目录
[root@testx ~]# ls /home
user1
# 使用id查看user1是否存在
[root@testx ~]# id user1
uid=1000(user1) gid=1000(user1) 组=1000(user1)
# 删除user1，但是不会删掉其家目录/home/user1
[root@testx ~]# userdel user1
# 使用id查看user1，不存在了
[root@testx ~]# id user1
id: user1: no such user
# user1的家目录/home/user1还在
[root@testx ~]# ls /home
user1
```

**（2）删除用户及其主目录，都会删除**

```
# 创建用户user2
[root@testx ~]# useradd user2
# user2家目录：/home/user2
[root@testx ~]# ls /home
user1  user2
```

```
# 查看user2用户信息
[root@testx ~]# id user2
uid=1000(user2) gid=1000(user2) 组=1000(user2)
# 删除用户user2及其家目录/home/user2
[root@testx ~]# userdel -r user2
# 查看user2信息，不存在了
[root@testx ~]# id user2
id: user2: no such user
# user2的家目录/home/user2也被删掉了
[root@testx ~]# ls /home
user1
```

# 12.7、who：查看登录用户信息

**1）基本语法**

| 语法 | 说明 |
| --- | --- |
| whoami | 显示当前用户名称 |
| who am i | 显示登录用户的用户名以及登录时间 |

**2）案例**

**（1）显示自身用户名称**

```
# 当前root用户
[root@testx ~]# whoami
root
# 切换到user1
[root@testx ~]# su user1
# 当前是user1
[user1@testx root]$ whoami
user1
```

**（2）显示登录用户的用户名以及登录时间**

```
# 我们是以root用户登录的
[root@testx ~]# who am i
root     pts/0        2022-05-03 12:53 (192.168.200.1)
# 切换到user1
[root@testx ~]# su user1
# 显示登录用户，即root用户
[user1@testx root]$ who am i
root     pts/0        2022-05-03 12:53 (192.168.200.1)
# 显示当前用户，即user1
[user1@testx root]$ whoami
user1
```

# 12.8、sudo：设置普通用户具有root权限

*sudo*是linux系统管理指令，是允许系统管理员让普通用户执行一些或者全部的root命令的一个工具，用法：`sudo 管理员命令`

**1）添加ready用户，并设置其密码**

```
[root@testx ~]# useradd tom
[root@testx ~]# passwd tom
```

**2）修改配置文件**

```
[root@testx ~]# vi /etc/sudoers
```

修改/etc/sudoers文件，找到下面一行，在root下面添加一行，如下图所示

```
## Allow root to run any commands anywhere
root    ALL=(ALL)   ALL
tom ALL=(ALL)   ALL
```

或者配置成采用sudo命令是，不需要输入密码

```
## Allow root to run any commands anywhere
root    ALL=(ALL)   ALL
tom ALL=(ALL)   NOPASSWD:ALL
```

修改完毕，现在可以用tom账号登录，在需要执行的命令前面加上sudo，即可获得root权限进行操作.

**3）案例**

> 下面以tom用户登录，操作下面命令，在/opt目录创建a目录报错“权限不够”，用`sudo mkdir /opt/a`来操作就可以了，会提示需要输入tom用户的密码。

```
[tom@testx ~]$ mkdir /opt/a
mkdir: 无法创建目录"/opt/a": 权限不够
[tom@testx ~]$ sudo mkdir /opt/a
[sudo] tom 的密码：
[tom@testx ~]$ ls /opt
a  rh
```

# 12.9、usermod：修改用户

**1）基本语法**

```
usermod [选项] 用户名
```

**2）选项说明**

```
  -c, --comment 注释            GECOS 字段的新值
  -d, --home HOME_DIR           用户的新主目录
  -e, --expiredate EXPIRE_DATE  设定帐户过期的日期为 EXPIRE_DATE
  -f, --inactive INACTIVE       过期 INACTIVE 天数后，设定密码为失效状态
  -g, --gid GROUP               强制使用 GROUP 为新主组
  -G, --groups GROUPS           新的附加组列表 GROUPS
  -a, --append GROUP            将用户追加至上边 -G 中提到的附加组中，
                                并不从其它组中删除此用户
  -h, --help                    显示此帮助信息并推出
  -l, --login LOGIN             新的登录名称
  -L, --lock                    锁定用户帐号
  -m, --move-home               将家目录内容移至新位置 (仅于 -d 一起使用)
```

```
  -o, --non-unique              允许使用重复的(非唯一的) UID
  -p, --password PASSWORD       将加密过的密码 (PASSWORD) 设为新密码
  -R, --root CHROOT_DIR         chroot 到的目录
  -P, --prefix PREFIX_DIR       prefix directory where are located the /etc/*
files
  -s, --shell SHELL             该用户帐号的新登录 shell
  -u, --uid UID                 用户帐号的新 UID
  -U, --unlock                  解锁用户帐号
  -v, --add-subuids FIRST-LAST  add range of subordinate uids
  -V, --del-subuids FIRST-LAST  remove range of subordinate uids
  -w, --add-subgids FIRST-LAST  add range of subordinate gids
  -W, --del-subgids FIRST-LAST  remove range of subordinate gids
  -Z, --selinux-user  SEUSER       用户账户的新 SELinux 用户映射
```

### 3）案例

#### （1）usermod -g：修改用户组

> 一个用户可以有一个主要组和多个附加组

```
# 创建用户jack，默认主要组和用户名同名
[root@testx ~]# useradd jack
# 查看用户信息，uid：用户id，gid：用户主要组id，组：用户组列表
[root@testx ~]# id jack
uid=1004(jack) gid=1006(jack) 组=1006(jack)
# 将jack的主要组指定为root
[root@testx ~]# usermod -g root jack
# 查看jack用户信息，gid变成root组了
[root@testx ~]# id jack
uid=1004(jack) gid=0(root) 组=0(root)
```

#### （2）usermodel -G：指定用户附加组列表

> 可以给用户指定多个附加组，语法：`usermod -G 附加组1[,附加组n]`

```
# 创建2个用户组gr1、gr2
[root@testx ~]# groupadd gr1
[root@testx ~]# groupadd gr2
# 查看jack的用户信息，主用户组gid是root用户组
[root@testx ~]# id jack
uid=1004(jack) gid=0(root) 组=0(root)
# 指定jack的附加用户组为gr1
[root@testx ~]# usermod -G gr1 jack
# 查看jack的用户信息，组=0(root),1007(gr1),含有2个用户组了，第一个是主要用户组，后面的都是附加组
[root@testx ~]# id jack
uid=1004(jack) gid=0(root) 组=0(root),1007(gr1)
# 指定jack的附加组为gr1和gr2
[root@testx ~]# usermod -G gr1,gr2 jack
# 现在jack有3个组了，1个主要组root，2个附加组：gr1和gr2
[root@testx ~]# id jack
uid=1004(jack) gid=0(root) 组=0(root),1007(gr1),1008(gr2)
```

#### （3）usermod -a -G 组名 ： 给用户添加附加组

> `usermod -G`用来设置用户的附加组，如果我们想给用户添加一个附加组，那么就需要用到-a命令了

```
# jack，主要用户组：root，2个附加组：gr1、gr2
[root@testx ~]# id jack
uid=1004(jack) gid=0(root) 组=0(root),1007(gr1),1008(gr2)
# 创建主gr3
[root@testx ~]# groupadd gr3
# 给jack添加一个附件用户组gr3
[root@testx ~]# usermod -a -G gr3 jack
# jack此时有3个附加组了：gr1、gr2、gr3
[root@testx ~]# id jack
uid=1004(jack) gid=0(root) 组=0(root),1007(gr1),1008(gr2),1009(gr3)
```

**（3）grpasswd：添加或删除附加用户组**

| 语法 | 说明 |
| --- | --- |
| gpasswd -a userA groupB | 给用户userA添加附加用户组groupB |
| gpaswd -d userA groupB | 将groupB从用户userA的附加组中移除 |

```
[root@testx ~]# id jack
uid=1004(jack) gid=0(root) 组=0(root),1007(gr1),1008(gr2),1009(gr3)
[root@testx ~]# gpasswd -d jack gr3
正在将用户“jack”从“gr3”组中删除
[root@testx ~]# id jack
uid=1004(jack) gid=0(root) 组=0(root),1007(gr1),1008(gr2)
[root@testx ~]# gpasswd -a jack gr3
正在将用户“jack”加入到“gr3”组中
[root@testx ~]# id jack
uid=1004(jack) gid=0(root) 组=0(root),1007(gr1),1008(gr2),1009(gr3)
```


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 13、用户组管理命令

每个用户都有一个用户组，系统可以对一个用户组中的所有用户进行集中管理。不同Linux系统对用户组的规定有所不同。

如Linux下的用户属于与它同名的用户组，这个用户组在创建用户时同时创建。

用户组的管理涉及用户组的添加、删除和修改。组的增加、删除和修改实际上就是对`/etc/group`文件的更新。

## 13.1、groupadd：创建用户组

**1）语法**

```
groupadd 组名
```

**2）案例**

**（1）添加用户组group1**

```
[root@testx ~]# groupadd group1
```

## 13.2、groupdel：删除用户组

**1）语法**

```
groupdel 组名
```

**2）案例**

**（1）删除group1组**

```
[root@testx ~]# groupdel group1
```

## 13.3、groupmod：修改用户组组名

**1）语法**

```
groupmod -n 新组名 老组名
```

**2）案例**

**（1）修改grou1的组名为group2**

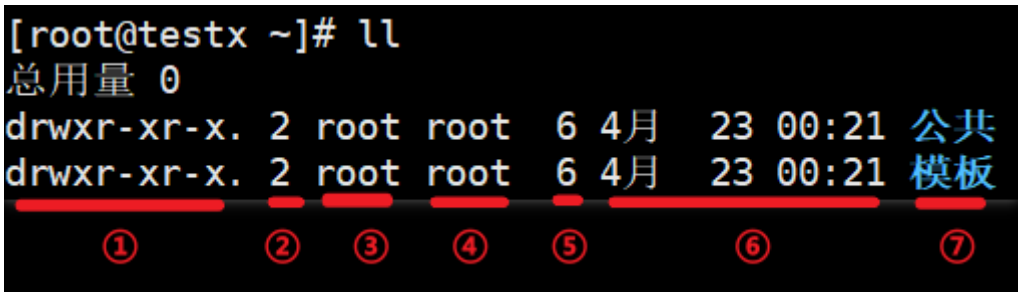
```
[root@testx ~]# groupadd group1
[root@testx ~]# groupmod -n group2 group1
```

# 14、文件权限类命令

## 14.1、文件属性

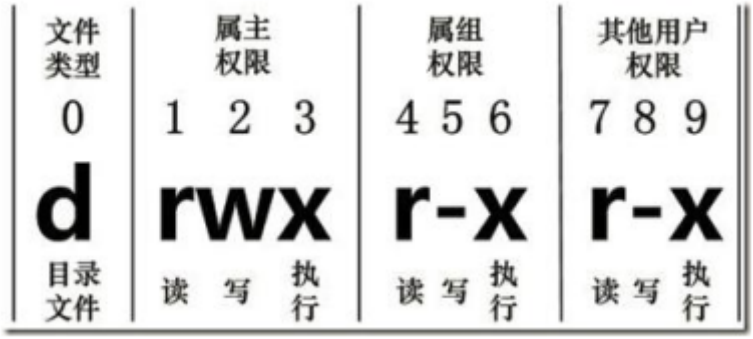
Linux系统是一种典型的多用户系统，不同的用户处于不同的地位，拥有不同的权限。为了保护系统的安全线，Linux系统对不同的用户访问同一文件（包括目录文件）的权限做了不同的规定，在Linux中我们可以使用ll或者ls -l 命令来显示一个文件的属性以及文件所属的用户和组。

**1）ll命令查看文件属性以及文件属性说明**




- ①：10个字符：文件类型与权限
- ②：如果是文件，这个数字表示硬链接的个数，如果是目录，这个数字表示目录中文件夹的个数
- ③：文件属主（文件拥有者）
- ④：文件所属组
- ⑤：文件大小
- ⑥：文件创建或者最后修改时间
- ⑦：文件名称

**2）从左到右的10个字符表示文件类型与权限**




- 0位：表示文件类型，在Linux中第一个字符代表这个文件是目录、文件或链接文件等等
  - -：代表文件
  - d：代表目录
  - l：表示连接文件
- 第1-3位：文件所有者拥有的权限（User）
- 第4-6位：该文件所属组拥有的权限（Group）
- 第7-9位：其他用户对该文件拥有的权限（Other）

**2）rwx作用文件和目录的不同解释**

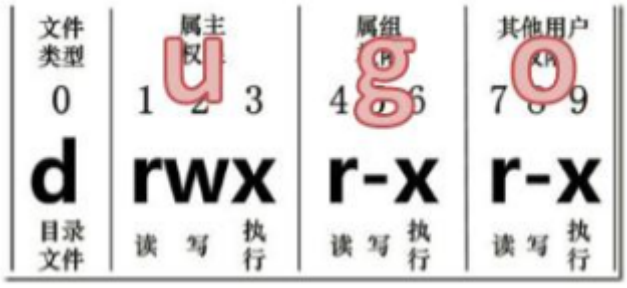
**（1）作用到文件**

- r：代表可读（read）：可以读取、查看文件的内容
- w：代表可写（write）：可以修改文件，但是不带表可以删除该文件，删除一个文件的前提条件是对该文件所在的目录有写权限，才能删除该文件
- x：带包可执行（execute）：可以被系统执行

**（2）作用到目录**

- r：代表可读（read）：可以读取，即可以通过ls查看目录内容
- w：代表可写（write）：可以修改，即可以在目录内创建+删除+重命名目录内的文件
- x：代表可执行（execute）：可以进入该目录，即可以通过cd进去

# 14.2、chmod：改变文件或目录权限




**1）第一种方式变更权限**

```
chmod [选项] 模式[,模式] 文件或目录
```

- 选项
  - R：递归处理
- 模式：`{ugoa}{+-=}{rwx}`

- u：所属者
- g：所属组
- o：其他人
- a：所有人（u、g、o的总和）

**2）第二种方式变更权限**

```
chmod [选项] [八进制模式] 文件或目录
```

- 选项
  - R：递归处理
- 八进制模式
  - r=4
  - w=2
  - x=1
  - wx=2+1=3
  - rx=4+1=5
  - rw=4+2=6
  - rwx=4+2+1=7

**3）案例实操**

（1）为1.sh文件的所属者添加执行权限

```
[root@testx ~]# chmod u+x 1.sh
```

（2）为1.sh文件的所属组添加读取和执行权限

```
chmod g+rx 1.sh
```

（3）修改1.sh文件权限：拥有者去掉执行权限，其他用户添加执行权限

```
[root@testx ~]# chmod u-x,o+x 1.sh
```

（4）设置1.sh用户权限，所有者有rwx权限，所属组以及其他用户有wx曲线

```
[root@testx ~]# chmod u=rwx,g=wx,o=wx 1.sh
```

（5）通过数字的方式设置权限，所有者、所属组、其他用户都具有可读可写可执行权限

```
[root@testx ~]# chmod 777 1.sh
```

（6）修改test文件夹及其内部所有文件的所有者、 所属组、 其他用户都具有可读可写可执行权限，需要用到-R参数，递归进行设置

```
[root@testx ~]# chmod -R 777 test/
```

# 14.3、chown：改变所有者

**1）基本语法**

> 改变文件或者目录的所有者

```
chown [选项] [所有者][:组] [文件或目录]
```

**2）选项说明**

| 选项 | 说明 |
| --- | --- |
| -R | 递归操作 |

**3）案例**

| 案例 | 说明 |
| --- | --- |
| `chown tom 1.sh` | 将1.sh的所有者改为tom |
| `chown :g1 1.sh` | 将1.sh的所属组改为g1 |
| `chrown -R tom:g1 test` | 递归将test目录及其所有子文件，所属者改为tom，所属组改为g1 |

## 14.4、chgrp：改变文件或目录所属组

**1）基本语法**

> 改变文件或者目录的所属组

```
chgrp [选项] [最终用户组] [文件或目录]
```

**2）选项说明**

| 选项 | 说明 |
| --- | --- |
| -R | 递归操作 |

**3）案例**

| 案例 | 说明 |
| --- | --- |
| `chgrp g1 1.sh` | 将1.sh的所属组改为g1 |
| `chgrp -R g1 test` | 递归将test目录及其所有子文件，所属组改为g1 |


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 15、搜索查找类命令

## 15.1、find：查找文件或者目录

find指令将从指定的目录向下递归地遍历其各个子目录，将满足条件的文件显示在终端。

**1）基本语法**

```
find [path...] [选项]
```

**2）选项说明**

| 选项 | 说明 |
| --- | --- |
| -name 文件名称 | 按文件名查找文件 |
| -user 用户名 | 查找属于指定用户名的所有文件 |
| -size [+-]n[bcwkMG] | 按照指定多文件大小查找文件<br>n：数字，表示文件大小，n后面可以跟单位<br>单位为(区分大小写)：<br>b：块（512字节）<br>c：字节<br>w：字（2字节）<br>k：千字节<br>M：兆直接<br>G：吉字节 |

**3）案例**

| 案例 | 说明 |
| --- | --- |
| `find -name 1.txt` | 在当前目录及其所有子目录中查询1.txt文件 |
| `find / -name 1.txt` | 在/目录及其所有子目录中递归查找1.txt文件 |
| `find / -name *.txt` | 在/目录及其所有子目录中递归查找.txt结尾的文件 |
| `find /opt/ /home/ -name *.txt` | 在[/opt,/home]两个目录中递归查找.txt结尾的文件 |
| `find / -size 10M` | 在/目录递归查找文件大小为10M的文件 |
| `find / -size +10M` | 在/目录递归查找大于10M的文件 |
| `find / -size +10M -size -100M` | 在/目录递归查找大于10M且小于100M的文件 |
| `find / -name *.log -size +50M -size -100M` | 在/opt目录中查找名称以.log结尾的且大于50M小于100M的文件 |

# 15.2、locate：快速定位文件路径

`locate` 命令和 `find` 命令一样都是用来在系统下查找文件或目录。但 locate命令要比find -name快得多，原因在于 `locate` 命令在查找文件时并不扫描具体目录，而是搜索一个已经创建好的数据库 `/var/lib/mlocate/mlocate.db` 。这个数据库中含有本地几乎所有文件信息（一些被排除在外的目录或刚创建的目录可能不会包含在数据库中。Linux系统会自动创建这个数据库，并且通过定时任务每天自动更新一次，因此，我们在用whereis和locate 查找文件时，有时会找到已经被删除的数据，或者刚刚建立文件，却无法查找到，原因就是因为数据库文件没有被更新。为了避免这种情况，可以在使用 `locate` 之前，先使用 `updatedb` 命令，手动更新数据库,但这也需要一定的时间，时间长短和空间大小文件数量有关。整个locate工作其实是由四部分组成的:

1. `/usr/bin/updatedb` 主要用来更新数据库，通过crontab自动完成的
2. `/usr/bin/locate` 查询文件位置

3. `/etc/updatedb.conf` updatedb的配置文件
4. `/var/lib/mlocate/mlocate.db` 存放文件索引信息的数据文件

**1）语法**

```
locate [OPTION]... [PATTERN]...
```

**2）选项**

```
  -b, --basename         match only the base name of path names
  -c, --count            只输出找到的数量
  -d, --database DBPATH  使用DBPATH指定的数据库，而不是默认数据库
/var/lib/mlocate/mlocate.db
  -e, --existing         only print entries for currently existing files
  -L, --follow           follow trailing symbolic links when checking file
existence (default)
  -h, --help             显示帮助
  -i, --ignore-case      忽略大小写
  -l, --limit, -n LIMIT  limit output (or counting) to LIMIT entries
  -m, --mmap             ignored, for backward compatibility
  -P, --nofollow, -H     don't follow trailing symbolic links when checking file
existence
  -0, --null             separate entries with NUL on output
  -S, --statistics       don't search for entries, print statistics about
eachused database
  -q, --quiet            安静模式，不会显示任何错误讯息
  -r, --regexp REGEXP    使用基本正则表达式
      --regex            使用扩展正则表达式
  -s, --stdio            ignored, for backward compatibility
  -V, --version          显示版本信息
  -w, --wholename        match whole path name (default)
```

**3）经验技巧**

由于 locate 指令基于数据库进行查询， 所以第一次运行前， 必须使用 updatedb 指令创建 locate 数据库。

**4）查找文件**

```
[root@testx ~]# updatedb
[root@testx ~]# locate 1.txt
/etc/brltty/brl-ts-pb65_pb81.txt
/etc/pki/nssdb/pkcs11.txt
/home/jack/1.txt
/home/jack/b/1.txt
/home/tom/1.txt
```


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 15.3、grep：过滤查找即“|”管道符

管道符，“|”，表示将前一个命令的处理结果输出通过管道传递给后面的命令进行处理。

**1）基本语法**

```
grep 选项 查找内容 源文件
```

**2）选项说明**

| 选项 | 功能 |
| --- | --- |
| -n | 显示匹配行及行号 |

**3）案例**

**查找1.txt中ready在哪些行**

> 下面先使用cat命令查看1.txt的内容。
>
> 然后后面有使用了2种方式检索ready位于1.txt中所在行号及内容。
>
> 第1种：`grep -n ready 1.txt`
>
> 第2种：`cat 1.txt | grep -n ready`，这里使用到了管道命令，将cat 1.txt的结果通过管道命令传递给 grep，grep将传递过来的内容作为查找目标。

```
[root@testx ~]# cat 1.txt
hello
不错哦 ready
ok
ready
[root@testx ~]# grep -n ready 1.txt
2:不错哦 ready
4:ready
[root@testx ~]# cat 1.txt | grep -n ready
2:不错哦 ready
4:ready
```

# 15.4、which：在PATH下查找命令位置

这条命令主要是用来查找系统***PATH目录下***的可执行文件，说白了就是查找那些我们已经安装好的可以直接执行的命令，比如

```
[root@testx ~]# which useradd
/usr/sbin/useradd
[root@testx ~]# which ls
alias ls='ls --color=auto'
    /usr/bin/ls
```

`which` 查找的可执行文件，必须是要在 PATH 下的可执行文件，而不能是没有加入 PATH 的可执行文件，即使他就是可执行文件，但是没有加入到系统搜索路径，他仍然无法被 `which` 发现（好吧，有点啰嗦了）。

# 15.5、whereis：查找可执行文件路径

`which`和`whereis`命令都是Linux操作系统下查找可执行文件路径的命令。

`whereis`这个命令可以用来查找二进制（命令）、源文件、man文件。与`which`不同的是这条命令可以是通过文件索引数据库而非PATH来查找的，所以查找的面比`which`要广。例如：

```
[root@testx ~]# whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz /usr/share/man/man1p/ls.1p.gz
[root@testx ~]# whereis useradd
useradd: /usr/sbin/useradd /usr/share/man/man8/useradd.8.gz
```

可以看到，`whereis`不仅找到了 ls 可执行文件的位置，还找到了其 `man` 帮助文件，可见其搜索范围比较广，不局限于PATH。


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 16、压缩和解压类命令

## 16.1、gzip/gunzip：压缩、解压

**1）基本语法**

| 命令 | 描述 |
| --- | --- |
| gzip 文件 | 压缩文件，只能将文件压缩为*.gz文件 |
| gunzip 文件.gz | 解压文件 |

**2）经验技巧**

- 只能压缩文件不能压缩目录
- 不保留原来的文件
- 同时多个文件会参数多个压缩包

**3）案例**

**（1）gzip压缩文件**

> 压缩后，源文件会消失

```
[root@testx b]# ls
spring.log
[root@testx b]# gzip spring.log
[root@testx b]# ls
spring.log.gz
```

**（2）gunzip解压文件**

> 解压后，源文件会消失

```
[root@testx b]# ls
spring.log.gz
[root@testx b]# gunzip spring.log.gz
[root@testx b]# ls
spring.log
```

## 16.2、zip/unzip：压缩、解压

**1）基本语法**

| 语法 | 描述 |
| --- | --- |
| zip [选项] xxx.zip 要压缩的文件列(可以有多个) | 压缩文件和目录的命令 |
| unzip [选项] xxx.zip | 解压文件 |

**2）选项说明**

| zip选项 | 功能 |
| --- | --- |
| -r | 压缩目录 |
| **unzip选项** | **功能** |
| -d 目录 | 指定解压后文件的存放目录 |

**3）经验技巧**

zip压缩命令在windows/linux都通用，可以压缩目录且保留源文件。

**4）案例**

**（1）将1.log和2.log压缩到package.zip中**

```
[root@testx b]# ls
1.log  2.log
[root@testx b]# zip package.zip 1.log 2.log
  adding: 1.log (deflated 75%)
  adding: 2.log (deflated 75%)
[root@testx b]# ls
1.log  2.log  package.zip
```

**（2）将log目录压缩到log.zip中**

```
# 当前目录有3个文件
[root@testx log]# ls
1.log  2.log  package.zip
# 删除2个log文件，保留等待解压的package.zip
[root@testx log]# rm -rf 1.log 2.log
# 剩下带解压的package.zip
[root@testx log]# ls
package.zip
# 对其进行解压，默认会解压在当前目录
[root@testx log]# unzip package.zip
Archive:  package.zip
  inflating: 1.log
  inflating: 2.log
# 当前目录有3个文件了
[root@testx log]# ls
1.log  2.log  package.zip
```

**（3）将package.zip文件解压到logs目录**

```
[root@testx log]# ls
package.zip
# 创建一个 logs 目录，用来存放稍后解压的文件
[root@testx log]# mkdir logs
[root@testx log]# ls
logs  package.zip
# 将package.zip解压到logs目录
[root@testx log]# unzip -d logs/ package.zip
Archive:  package.zip
  inflating: logs/1.log
  inflating: logs/2.log
[root@testx log]# ls logs/
1.log  2.log
```

**（4）将log目录压缩到log.zip中**

```
[root@testx b]# ls
log
[root@testx b]# ls log
1.log  2.log
[root@testx b]# zip -r log.zip log
  adding: log/ (stored 0%)
  adding: log/1.log (deflated 75%)
  adding: log/2.log (deflated 75%)
[root@testx b]# ls
log  log.zip
```

# 16.3、tar：打包、解压（常用）

1）基本语法

```
tar [选项] xxx.tar.gz 需要打包的文件列表
```

> 打包文件或者目录，压缩后的文件格式为tar.gz

2）选项说明

| 选项 | 说明 |
| --- | --- |
| -c | 产生.tar打包文件 |
| -v | 显示详细信息 |
| -f | 指定文件名 |
| -z | 用gzip对文件进行压缩或者解压 |
| -x | 解包.tar文件 |
| -C | 解压到指定目录 |

3）案例

**（1）tar -czvf：压缩多个文件**

> 语法：`tar -czvf 打包后的文件.tar.gz 被打包的文件列表`

```
[root@testx b]# ls
1.log  2.log
[root@testx b]# tar -czvf log.tar.gz 1.log 2.log
1.log
2.log
[root@testx b]# ls
1.log  2.log  log.tar.gz
```

### (2) tar -czvf：打包目录

> 语法：`tar -czvf 打包后的文件.tar.gz 被打包的目录`

```
[root@testx b]# ls
logs
[root@testx b]# ls logs/
1.log  2.log
# 将logs目录打包为log.tar.gz文件
[root@testx b]# tar -czvf log.tar.gz logs/
logs/
logs/1.log
logs/2.log
[root@testx b]# ls
logs  log.tar.gz
```

### (3) tar -xzvf：解压文件，默认解压到当前目录

> 语法：`tar -xzvf 被解压的文件`

```
[root@testx b]# ls
log.tar.gz
[root@testx b]# tar -xzvf log.tar.gz
logs/
logs/1.log
logs/2.log
[root@testx b]# ls
logs  log.tar.gz
[root@testx b]# ls logs/
1.log  2.log
```

### (3) tar -xzvf：解压文件到指定的目录

> 语法：`tar -xzvf 被解压的文件 -C 解压后的目标目录`

```
[root@testx b]# ls
log.tar.gz
[root@testx b]# mkdir target
[root@testx b]# tar -xzvf log.tar.gz -C target/
logs/
logs/1.log
logs/2.log
[root@testx b]# ls
log.tar.gz  target
[root@testx b]# ls target/
logs
[root@testx b]# ls target/logs/
1.log  2.log
```

- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

# 17、磁盘管理类命令

## 17.1、磁盘查看和分区类

### 17.1.1、du：查看文件和目录占用的磁盘空间

**1）基本语法**

```
du 目录/文件
```

> 显示目录下每个字母的磁盘使用情况

**2）选项说明**

| 选项 | 功能 |
| --- | --- |
| -h | 以人们交易阅读的GBytes，MBytes，KBytes等格式自行显示 |
| -a | 不仅查看子目录大小，还包括文件 |
| -c | 显示所有的文件和子目录大小后，显示总和 |
| -s | 只显示总和 |
| --max-depth=n<br>或<br>-d n | 指定统计子目录的深度为第n层 |

**3）案例实操**

**（1）查看当前用户主目录占用的磁盘空间大小**

```
[root@testx ~]# cd
[root@testx ~]# du -sh
7.1M    .
```

**（2）综合案例**

```
# 通过tree查看一下文件结构
[root@testx b]# tree
.
├── 1.tar.gz
├── log.tar.gz
└── target
    └── logs
        ├── 1.log
        └── 2.log

2 directories, 4 files

# 默认情况只会显示目录的磁盘使用请求
[root@testx b]# du -h
400K    ./target/logs
400K    ./target
700K    .

# 通过du递归查看所有文件的磁盘占用空间
[root@testx b]# du -ah
200K    ./1.tar.gz
100K    ./log.tar.gz
200K    ./target/logs/1.log
200K    ./target/logs/2.log
400K    ./target/logs
400K    ./target
700K    .

# 查看深度为1的文件磁盘占用信息
[root@testx b]# du -ah -d 1
200K    ./1.tar.gz
100K    ./log.tar.gz
400K    ./target
700K    .
```

## 17.1.2、df：查看磁盘空间使用情况

**1）基本语法**

```
df 选项
```

> 列出文件系统的整体磁盘使用量，检查文件系统磁盘占用情况。

**2）选项**

| 选项 | 功能 |
| --- | --- |
| -h | 以人们较易阅读的GBytes、MBytes、KBytes等格式显示大小 |

**3）案例**

```
# 查看磁盘使用情况
[root@testx b]# df -h
文件系统                 容量  已用  可用 已用% 挂载点
devtmpfs                 894M     0  894M    0% /dev
tmpfs                    910M     0  910M    0% /dev/shm
tmpfs                    910M   11M  900M    2% /run
tmpfs                    910M     0  910M    0% /sys/fs/cgroup
/dev/mapper/centos-root   17G  5.0G   13G   29% /
/dev/sda1               1014M  185M  830M   19% /boot
tmpfs                    182M   12K  182M    1% /run/user/42
tmpfs                    182M     0  182M    0% /run/user/0
```

## 17.1.3、lsblk：查看设备挂载情况

**1）语法**

```
lsblk [选项] [<设备> ...]
```

> 设备参数可以省略，此时会列出所有设备

**2）选项**

| 选项 | 功能 |
| --- | --- |
| -f | 查看详细的设备挂载情况，显示文件系统信息 |

**3）案例**

**（1）lsblk：查看设备挂载信息**

```
NAME            MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda               8:0    0   20G  0 disk
├─sda1            8:1    0    1G  0 part /boot
└─sda2            8:2    0   19G  0 part
  ├─centos-root 253:0    0   17G  0 lvm  /
  └─centos-swap 253:1    0    2G  0 lvm  [SWAP]
sr0              11:0    1  4.4G  0 rom
```

- NAME：块设备名
- MAJ:MIN：本栏显示主要和次要设备号
- RM：显示可移动设备。0表示非移动设备，1表示可移动设备
- SIZE：块设备的空间大小
- RO：是否只读，0表示非只读，1表示只读
- TYPE：块设备类型，比如disk磁盘，part分区，lvm逻辑卷，rom只读存储
- MOUNTPOINT：设备挂载点

**（2）lsblk -f：查看设备文件详细信息**

```
[root@testx ~]# lsblk -f
NAME            FSTYPE      LABEL           UUID
  MOUNTPOINT
sda

├─sda1          xfs                         afc5b580-2325-4927-8c90-be699ef0db3a
  /boot
└─sda2          LVM2_member                 x8BBth-iUkR-mxG0-Kzsh-Tai1-HRUN-
F0FYUy
  ├─centos-root xfs                         c7cc6061-60b3-468d-b3cd-642d164fc048
  /
  └─centos-swap swap                        bf602523-785c-4b62-879b-1cbc36071c13
  [SWAP]
sr0             iso9660     CentOS 7 x86_64 2020-11-04-11-36-43-00
  /run/media/root/CentOS 7 x86_64
```

- NAME：设备名
- FSTYPE：文件系统类型
- UUID：文件系统ID
- MOUNTPOINT：设备挂载点

**(3) lsblk 设备：查看某个设备挂载信息**

```
[root@testx ~]# lsblk /dev/sda
NAME            MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
sda               8:0    0  20G  0 disk
├─sda1            8:1    0   1G  0 part /boot
└─sda2            8:2    0  19G  0 part
  ├─centos-root 253:0    0  17G  0 lvm  /
  └─centos-swap 253:1    0   2G  0 lvm  [SWAP]
```

## 17.1.4、mount：挂载

```
mount 设备 目录
```

> 这个后面的案例中会讲。

## 17.1.5、umount：卸载

```
umount <设备> | <目录>
```

```
# 通过设备名卸载
umount -v /dev/sda1
# 通过挂载点卸载
umount -v /mnt/mymount/
```


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 17.2、磁盘管理(添加磁盘、分区、格式化、挂载)

这部分会给大家演示一下，如何在系统中添加一块硬盘，会演示介绍整个过程。

## 17.2.1、linux如何使用一块硬盘?

大家可以把硬盘想象成一个大房子，房子里面有很多房间，房间需要装修后才能入住，才能够放东西

### 1）硬盘

相当于一个大房子

### 2）分区

硬盘想要使用，也需要像房子一样划分成一块块的区域，这个就叫分区，一个分区相当于房子中的一个房间。

在linux中一块硬盘只能划分成4个大的分区，如果想划分更多的分区怎么办呢?

这里就涉及到分区的类型了，即主分区和扩展分区

- **主分区**：内部不能在创建小房间的分区，即一块硬盘最多只能有4个主分区，即4个分区的类型都是主分区。
- **扩展分区**：内部可以继续划分成小房间的分区，一块磁盘中最多只能有一个扩展分区，扩展分区内部可以划分成很多小房间，这些小房间叫做逻辑分区。

### 3）格式化

对分区进行格式化，相当于对房间进行装修，装修后然后才能放东西，即格式化之后，分区中才能够存储数据

### 4）挂载

分区经过格式化之后，可以放数据了，而linux中是通过目录来访问文件的，而真正存储数据的是磁盘中的分区，此时目录就需要和分区有个映射关系，创建映射关系这个过程就是挂载的过程，将某个区分和某个目录联系起来，当向这个目录中放文件的时候，实际上就向目录关联的分区中放东西。

## 17.2.2、硬盘说明

- linux硬盘分IDE硬盘和SCSI硬盘，目前基本上都是SCSI硬盘
- 对应IDE硬盘，驱动器标识符为"hdx~"，其中“hd”表名分区所在的设备的类型，这里是指IDE硬盘了，“x”为盘号（a为基本盘，b为基本从属盘，c为辅助主盘，d为辅助从属盘），“~”代表分区，钱4个分区用数字1到5表示，他们是主分区或者扩展分区，从5开始就是逻辑分区，例如，hda3表示为第一个IDE硬盘上第3个主分区或扩展分区，hdb2表示第2个IDE硬盘上的第2个主分区或扩展分区
- 对于SCSI硬盘则标识为“sdx~”，SCSI硬盘是用“sd”来表示分区所在的设备的类型的，其余则和IDE硬盘的表示方法一样

使用`lsblk`可以看到系统硬盘情况，下面我们来看一下

```
[root@testx ~]# lsblk
NAME            MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda               8:0    0   20G  0 disk
├─sda1            8:1    0    1G  0 part /boot
└─sda2            8:2    0   19G  0 part
  ├─centos-root 253:0    0   17G  0 lvm  /
  └─centos-swap 253:1    0    2G  0 lvm  [SWAP]
sr0              11:0    1  4.4G  0 rom  /run/media/root/CentOS 7 x86_64
```

TYPE列为disk的表示硬盘，可以看出只有一块硬盘，名称为`sda`，sd开头的是SCSI类型硬盘，这个硬盘下面有2个分区（TYPE类型为part的）

- sda1：第一个主分区
- sda2：扩展分区，下面有2个逻辑分区（TYPE列为lvm的）

    - centos-root：逻辑分区，挂载点是/，表示/目录中的数据这个分区中放在
    - centos-swap：逻辑分区，挂载点是[SWAP：交换分区]，类似于Windows的虚拟内存，就是当内存不足的时候，把一部分硬盘空间虚拟成内存使用。
- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

# 17.3、案例：演示添加硬盘的过程（含分区、格式化、挂载整个过程）

这个过程还是挺重要，当我们系统中磁盘空间不足的时候，我们需要添加硬盘，就涉及到这个过程。

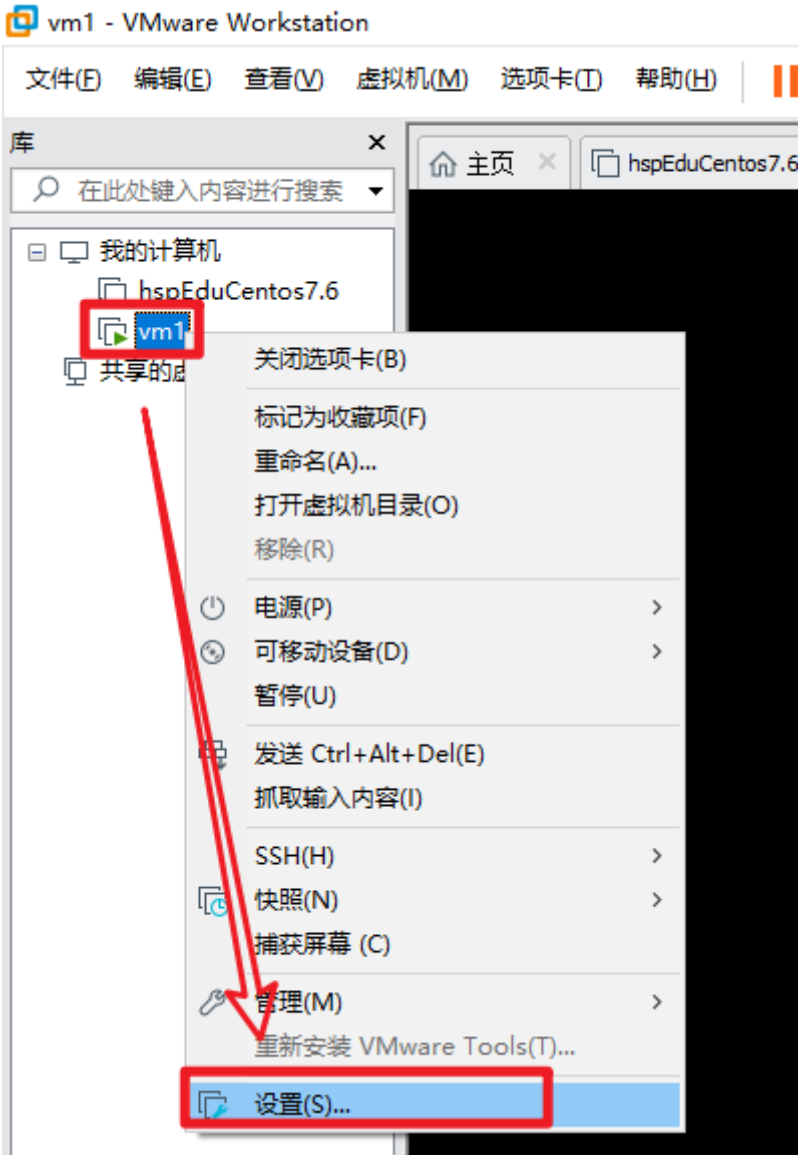
## 1）需求

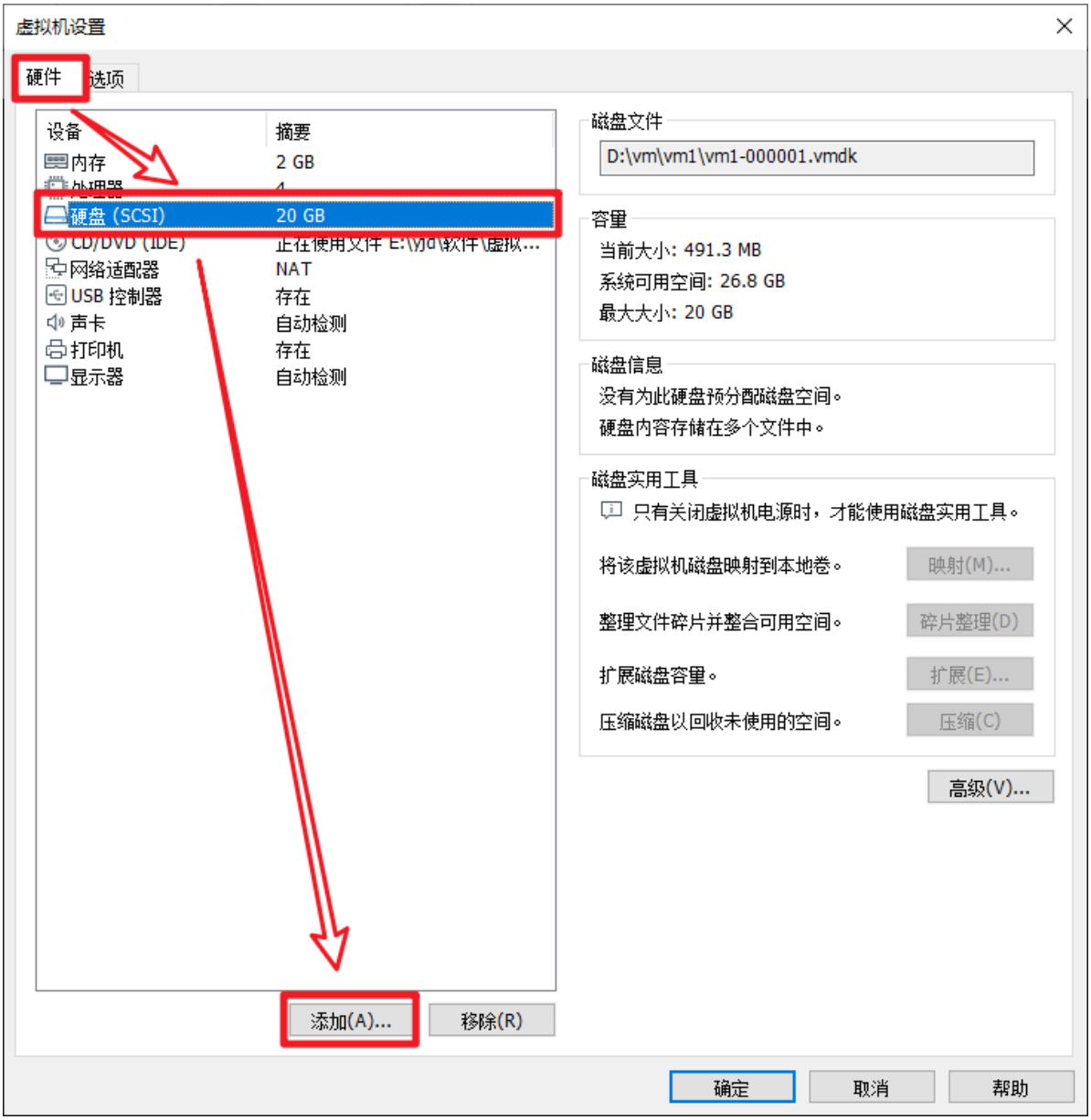
添加一块20G的硬盘，分3个主分区，1个扩展分区，扩展分区下面挂2个逻辑分区，由于我们系统中已经存在一块硬盘了：sda，那么我们重新添加一块硬盘，名称就变成：sdb了，对应的分区如下，如果大家要创建的分区和这个不一样，将过程调整一下就可以了。

| 分区名称 | 类型（P：主分区，E：扩展分区，L：逻辑分区） | 大小（G） | 文件系统类型 |
| --- | --- | --- | --- |
| sdb1 | P | 4G | xfs |
| sdb2 | P | 5G | xfs |
| sdb3 | P | 6G | xfs |
| sdb4 | E | | |
| sdb5 | L | 2G | xfs |
| sdb6 | L | 3G | xfs |

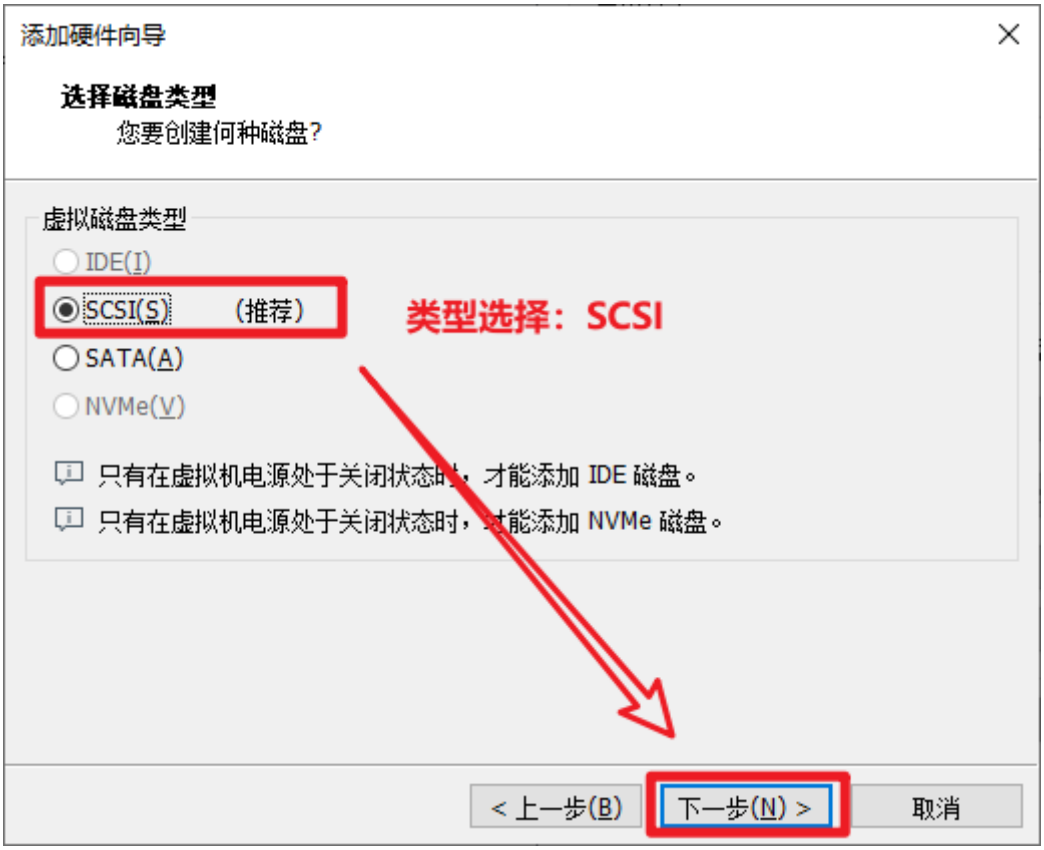
## 2）添加一块20G的硬盘

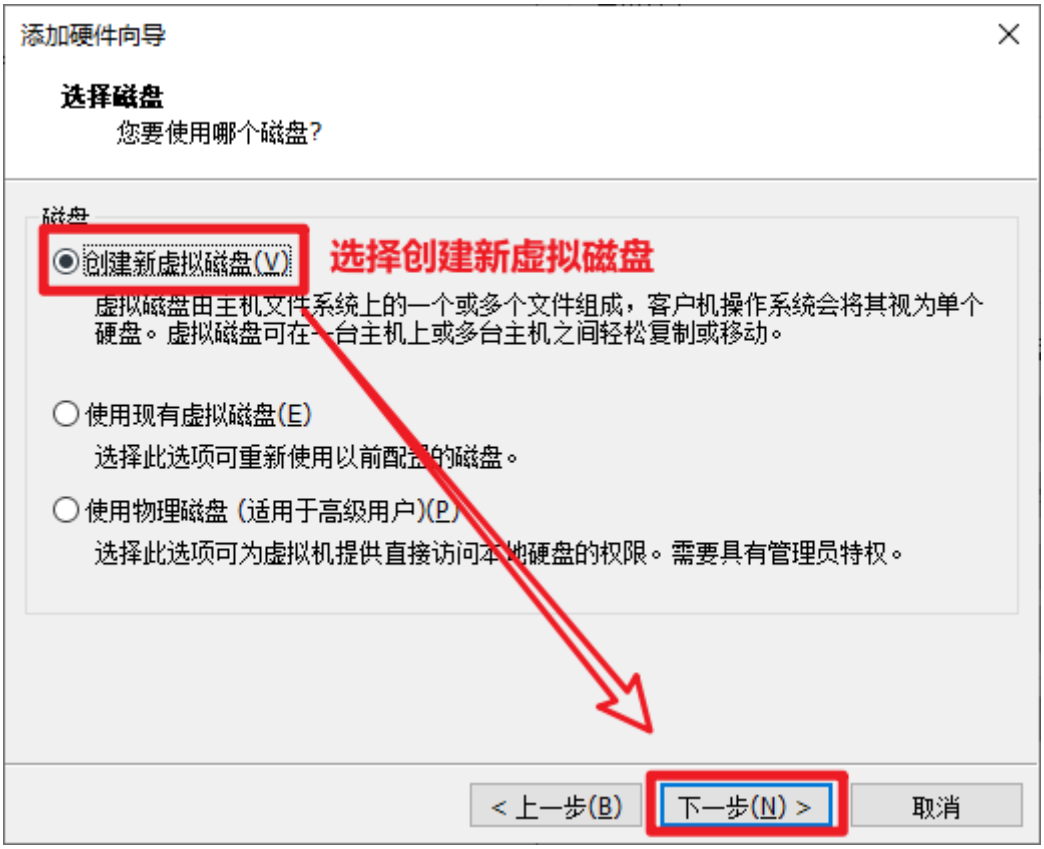
VMware左侧窗口，选中当前虚拟机->右键->设置

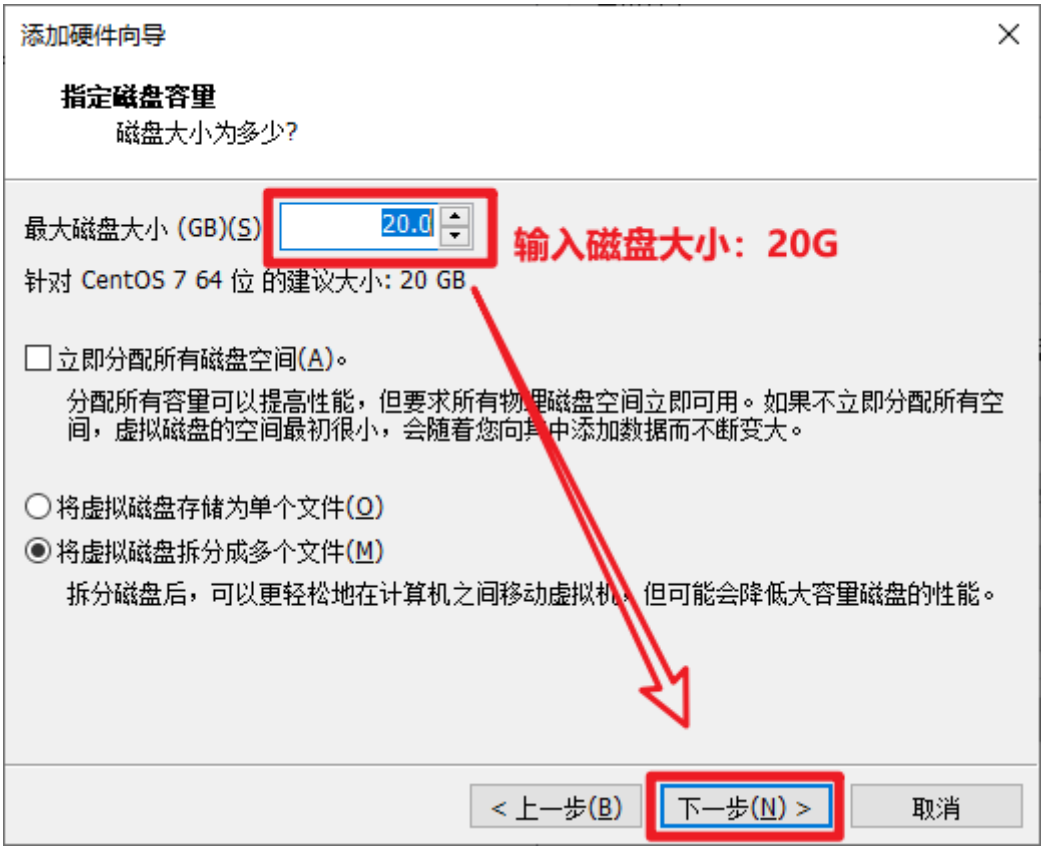

vm1 - VMware Workstation
文件(F)
编辑(E)
查看(V)
虚拟机(M)
选项卡(T)
帮助(H)
库
在此处键入内容进行搜索
我的计算机
hspEduCentos7.6
vm1
共享的虚
主页
hspEduCentos7.6
关闭选项卡(B)
标记为收藏项(F)
重命名(A)...
打开虚拟机目录(O)
移除(R)
电源(P)
可移动设备(D)
暂停(U)
发送 Ctrl+Alt+Del(E)
抓取输入内容(I)
SSH(H)
快照(N)
捕获屏幕 (C)
管理(M)
重新安装 VMware Tools(T)...
设置(S)...


虚拟机设置
硬件
选项
设备
摘要
内存
2 GB
硬盘 (SCSI)
20 GB
网络适配器
NAT
USB 控制器
存在
声卡
自动检测
打印机
存在
显示器
自动检测
磁盘文件
D:\vm\vm1\vm1-000001.vmdk
容量
当前大小: 491.3 MB
系统可用空间: 26.8 GB
最大大小: 20 GB
磁盘信息
没有为此硬盘预分配磁盘空间。
硬盘内容存储在多个文件中。
磁盘实用工具
只有关闭虚拟机电源时，才能使用磁盘实用工具。
将该虚拟机磁盘映射到本地卷。
映射(M)...
整理文件碎片并整合可用空间。
碎片整理(D)
扩展磁盘容量。
扩展(E)...
压缩磁盘以回收未使用的空间。
压缩(C)
高级(V)...
添加(A)...
移除(R)
确定
取消
帮助

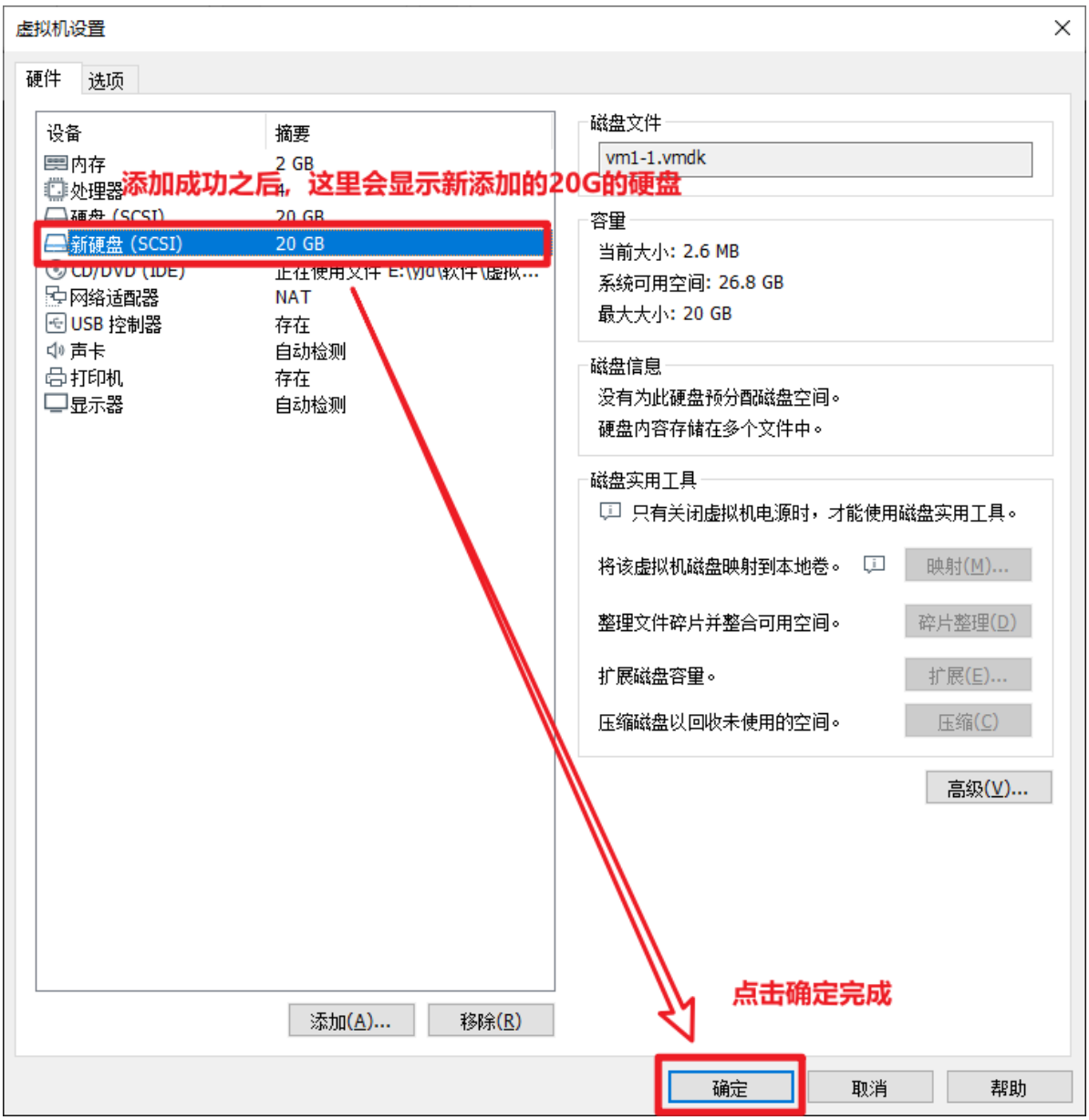

添加硬件向导
硬件类型
您要安装哪类硬件?
硬件类型(H):
硬盘
CD/DVD 驱动器
软盘驱动器
网络适配器
USB 控制器
声卡
并行端口
串行端口
打印机
通用 SCSI 设备
可信平台模块
解释
添加硬盘。
< 上一步(B)
下一步(N) >
取消


添加硬件向导
选择磁盘类型
您要创建何种磁盘?
虚拟磁盘类型
IDE(I)
SCSI(S) (推荐)
SATA(A)
NVMe(V)
类型选择：SCSI
只有在虚拟机电源处于关闭状态时，才能添加 IDE 磁盘。
只有在虚拟机电源处于关闭状态时，才能添加 NVMe 磁盘。
< 上一步(B)
下一步(N) >
取消


添加硬件向导
选择磁盘
您要使用哪个磁盘?
磁盘
创建新虚拟磁盘(V)
选择创建新虚拟磁盘
虚拟磁盘由主机文件系统上的一个或多个文件组成，客户机操作系统会将其视为单个硬盘。虚拟磁盘可在一台主机上或多台主机之间轻松复制或移动。
使用现有虚拟磁盘(E)
选择此选项可重新使用以前配置的磁盘。
使用物理磁盘 (适用于高级用户)(P)
选择此选项可为虚拟机提供直接访问本地硬盘的权限。需要具有管理员特权。
< 上一步(B)
下一步(N) >
取消


添加硬件向导
指定磁盘容量
磁盘大小为多少?
最大磁盘大小 (GB)(S)
20.0
输入磁盘大小：20G
针对 CentOS 7 64 位 的建议大小: 20 GB
立即分配所有磁盘空间(A)。
分配所有容量可以提高性能，但要求所有物理磁盘空间立即可用。如果不立即分配所有空间，虚拟磁盘的空间最初很小，会随着您向其中添加数据而不断变大。
将虚拟磁盘存储为单个文件(O)
将虚拟磁盘拆分成多个文件(M)
拆分磁盘后，可以更轻松地在计算机之间移动虚拟机，但可能会降低大容量磁盘的性能。
< 上一步(B)
下一步(N) >
取消


添加硬件向导
指定磁盘文件
您要在何处存储磁盘文件?
磁盘文件(F)
将使用多个磁盘文件创建一个 20 GB 虚拟磁盘。将根据此文件名自动命名这些磁盘文件。
vm1-1.vmdk
浏览(R)...
默认文件名称就可以
< 上一步(B)
完成
取消




## 3）重启机器，即可看到新的硬盘

需要重启机器之后，可以看到这颗硬盘，所以执行reboot，重启机器。

重启之后，用`lsblk`命令，即可看到新加的硬盘：`sdb`

```
[root@testx ~]# lsblk
NAME            MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda               8:0    0   20G  0 disk
├─sda1            8:1    0    1G  0 part /boot
└─sda2            8:2    0   19G  0 part
  ├─centos-root 253:0    0   17G  0 lvm  /
  └─centos-swap 253:1    0    2G  0 lvm  [SWAP]
sdb               8:16   0   20G  0 disk
```

这里说一下，linux中所有设备都是用文件来表示的，都在/dev目录用一个文件表示，磁盘也是一样的，我们通过下面命令来看一下

```
[root@testx ~]# ll /dev | grep -w "sd."
brw-rw----. 1 root disk      8,   0 5月  10 17:26 sda
brw-rw----. 1 root disk      8,  16 5月  10 17:26 sdb
```

- /dev/sda：对应第1块硬盘
- /dev/sdb：对应第2块硬盘

## 4）创建第1个主分区：sdb1，大小为4G

需要用到一个新的命令：`fdisk 磁盘`，来对磁盘进行分区，这里我们需要对`sdb`这块磁盘进行分区，所以需用`fdisk /dev/sdb`来进行操作。

**（1）执行`fdisk /dev/sdb`命令**

```
[root@testx ~]# fdisk /dev/sdb    1、对磁盘进行分区
欢迎使用 fdisk (util-linux 2.23.2)。

更改将停留在内存中，直到您决定将更改写入磁盘。
使用写入命令前请三思。

Device does not contain a recognized partition table
使用磁盘标识符 0x10dcd4a7 创建新的 DOS 磁盘标签。

命令(输入 m 获取帮助)：
```

**（2）输入m，查看帮助**

- n：在当前磁盘创建一个新的分去
- d：在当前磁盘删除一个分区
- p：列出当前磁盘分区信息
- w：保存配置并退出

```
命令(输入 m 获取帮助)：m    输入m，查看帮助
命令操作
   a   toggle a bootable flag
   b   edit bsd disklabel
   c   toggle the dos compatibility flag
   d   delete a partition
   g   create a new empty GPT partition table
   G   create an IRIX (SGI) partition table
   l   list known partition types
   m   print this menu
   n   add a new partition
   o   create a new empty DOS partition table
   p   print the partition table
   q   quit without saving changes
   s   create a new empty Sun disklabel
   t   change a partition's system id
   u   change display/entry units
   v   verify the partition table
   w   write table to disk and exit
   x   extra functionality (experts only)
```

**（3）输入n：创建分区，提示让我们选择创建的分区类型，我们要创建是主分区，所以需要选择p**

> p：主分区
>
> e：扩展分区

```
命令(输入 m 获取帮助)：n
Partition type:
   p   primary (0 primary, 0 extended, 4 free)
   e   extended
Select (default p): p
```

**（4）输入p：提示如下，此时会让我们选择分区编号，这里可以看出分区编号只能有4个**

```
Select (default p): p
分区号 (1-4，默认 1)：
```

**（5）输入分区编号1，如下，又会提示让我们输入扇区的位置，直接按enter键就行了，会自动取默认值**

```
分区号 (1-4，默认 1)：1
起始 扇区 (2048-41943039，默认为 2048)：
```

**（6）输入分区的大小：+4G，注意这里的格式：+size{K,M,G}这单位有大小写的，不要写错了**

```
Last 扇区，+扇区 or +size{K,M,G} (2048-41943039，默认为 41943039)：+4G
分区 1 已设置为 Linux 类型，大小设为 4 GiB
```

**（7）输入p看一下，目前的分区信息，如下**

> 可以看到列表中目前只有一个分区：/dev/sdb1

```
命令(输入 m 获取帮助)：p

磁盘 /dev/sdb：21.5 GB，21474836480 字节，41943040 个扇区
Units = 扇区 of 1 * 512 = 512 bytes
扇区大小(逻辑/物理)：512 字节 / 512 字节
I/O 大小(最小/最佳)：512 字节 / 512 字节
磁盘标签类型：dos
磁盘标识符：0x10dcd4a7

   设备 Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048     8390655     4194304   83  Linux
```

## 5）创建第2个主分区：sdb2，大小为5G

> 过程参考第4步中创建第一个分区的过程。

```
命令(输入 m 获取帮助): n
Partition type:
   p   primary (1 primary, 0 extended, 3 free)
   e   extended
Select (default p): p
分区号 (2-4, 默认 2): 2
起始 扇区 (8390656-41943039, 默认为 8390656):
将使用默认值 8390656
Last 扇区, +扇区 or +size{K,M,G} (8390656-41943039, 默认为 41943039): +5G
分区 2 已设置为 Linux 类型, 大小设为 5 GiB

命令(输入 m 获取帮助): p

磁盘 /dev/sdb: 21.5 GB, 21474836480 字节, 41943040 个扇区
Units = 扇区 of 1 * 512 = 512 bytes
扇区大小(逻辑/物理): 512 字节 / 512 字节
I/O 大小(最小/最佳): 512 字节 / 512 字节
磁盘标签类型: dos
磁盘标识符: 0x10dcd4a7

   设备 Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048     8390655     4194304   83  Linux
/dev/sdb2         8390656    18876415     5242880   83  Linux
```

## 6）创建第3个主分区：sdb3，大小为6G

```
命令(输入 m 获取帮助): n
Partition type:
   p   primary (2 primary, 0 extended, 2 free)
   e   extended
Select (default p): p
分区号 (3,4, 默认 3): 3   第3个分区
起始 扇区 (18876416-41943039, 默认为 18876416):                    大小6G
将使用默认值 18876416
Last 扇区, +扇区 or +size{K,M,G} (18876416-41943039, 默认为 41943039): +6G
分区 3 已设置为 Linux 类型, 大小设为 6 GiB

命令(输入 m 获取帮助): p

磁盘 /dev/sdb: 21.5 GB, 21474836480 字节, 41943040 个扇区
Units = 扇区 of 1 * 512 = 512 bytes
扇区大小(逻辑/物理): 512 字节 / 512 字节
I/O 大小(最小/最佳): 512 字节 / 512 字节
磁盘标签类型: dos
磁盘标识符: 0x10dcd4a7
                  此时有3个分区了
   设备 Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048     8390655     4194304   83  Linux
/dev/sdb2         8390656    18876415     5242880   83  Linux
/dev/sdb3        18876416    31459327     6291456   83  Linux
```

## 7）创建扩展分区：sdb4，大小为剩下的空间

> 过程和上面的上面创建分区的过程有点不同
>
> - 分区类型选择的是e，不能选p哦
> - 输入大小的时候，直接按enter就行了，空间大小为剩余的空间，即5G

```
命令(输入 m 获取帮助): n
Partition type:
   p   primary (3 primary, 0 extended, 1 free)
   e   extended
Select (default e): e   注意这里分区类型选：e
已选择分区 4
起始 扇区 (31459328-41943039，默认为 31459328)：这里直接按enter取默认值
将使用默认值 31459328
Last 扇区，+扇区 or +size{K,M,G} (31459328-41943039，默认为 41943039)：这里直接按enter取默认值
将使用默认值 41943039
分区 4 已设置为 Extended 类型，大小设为 5 GiB

命令(输入 m 获取帮助): p   输入p查看当前磁盘分区列表

磁盘 /dev/sdb：21.5 GB，21474836480 字节，41943040 个扇区
Units = 扇区 of 1 * 512 = 512 bytes
扇区大小(逻辑/物理)：512 字节 / 512 字节
I/O 大小(最小/最佳)：512 字节 / 512 字节
磁盘标签类型：dos
磁盘标识符：0x10dcd4a7

   设备 Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048     8390655     4194304   83  Linux
/dev/sdb2         8390656    18876415     5242880   83  Linux
/dev/sdb3        18876416    31459327     6291456   83  Linux
/dev/sdb4        31459328    41943039     5241856    5  Extended
```

/dev/sdb4为第4个分区，是个扩展分区

## 8）创建逻辑分区：sdb5，大小为2G

```
命令(输入 m 获取帮助): n   1、输入n
All primary partitions are in use
添加逻辑分区 5
起始 扇区 (31459391-41943039，默认为 31459391)：2、按enter取默认值
将使用默认值 31459391
Last 扇区，+扇区 or +size{K,M,G} (31459391-41943039，默认为 41943039)：+2G   3、输入大小+2G
分区 5 已设置为 Linux 类型，大小设为 2 GiB
```

## 9）创建逻辑分区：sdb6，大小为3G

```
命令(输入 m 获取帮助): n
All primary partitions are in use   输入n，其他地方按enter取默认值即可
添加逻辑分区 6
起始 扇区 (35657728-41943039，默认为 35657728)：
将使用默认值 35657728
Last 扇区，+扇区 or +size{K,M,G} (35657728-41943039，默认为 41943039)：
将使用默认值 41943039
分区 6 已设置为 Linux 类型，大小设为 3 GiB
```

## 10）输入p查看分区列表，6个分区都ok了

```
命令(输入 m 获取帮助)：p

磁盘 /dev/sdb：21.5 GB, 21474836480 字节，41943040 个扇区
Units = 扇区 of 1 * 512 = 512 bytes
扇区大小(逻辑/物理)：512 字节 / 512 字节
I/O 大小(最小/最佳)：512 字节 / 512 字节
磁盘标签类型：dos
磁盘标识符：0x10dcd4a7

   设备 Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048     8390655     4194304   83  Linux
/dev/sdb2         8390656    18876415     5242880   83  Linux
/dev/sdb3        18876416    31459327     6291456   83  Linux
/dev/sdb4        31459328    41943039     5241856    5  Extended
/dev/sdb5        31459391    35653695     2097152+  83  Linux
/dev/sdb6        35653759    41943039     3144640+  83  Linux
```

## 11）输入w保存并退出（别忘啦）

> 输入w：保存配置并退出，否则就白搞了

```
命令(输入 m 获取帮助)：w
The partition table has been altered!

Calling ioctl() to re-read partition table.
正在同步磁盘。
```

## 12）查看/dev/sdb磁盘分区挂载情况

```
[root@testx ~]# lsblk -f /dev/sdb
NAME   FSTYPE LABEL UUID MOUNTPOINT
sdb
├─sdb1
├─sdb2
├─sdb3
├─sdb4
├─sdb5
└─sdb6
```

## 13）格式化第一个分区sdb1

需要使用`mkfs`命令对分区进行格式化，语法

```
mkfs -t 文件类型 设备
```

这里文件类型我们选择`xfs`，文件类型这里不介绍了，大家可以自行百度。

执行下面命令对sdb1分区进行格式化

```
[root@testx ~]# mkfs -t xfs /dev/sdb1
meta-data=/dev/sdb1              isize=512    agcount=4, agsize=262144 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=0, sparse=0
data     =                       bsize=4096   blocks=1048576, imaxpct=25
         =                       sunit=0      swidth=0 blks
naming   =version 2              bsize=4096   ascii-ci=0 ftype=1
log      =internal log           bsize=4096   blocks=2560, version=2
         =                       sectsz=512   sunit=0 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
```

## 14）格式化其他几个分区（不包含扩展分区）

```
mkfs -t xfs /dev/sdb2
mkfs -t xfs /dev/sdb3
mkfs -t xfs /dev/sdb5
mkfs -t xfs /dev/sdb6
```

## 15）挂载

分区格式化好，然后需要将其和目录进行关联起来之后才可以使用，这个过程就是挂载的过程。

**挂载有2种方式**

- 方式1：使用mount命令进行挂载，重启会丢失
- 方式2：修改文件的方式，持久性的，重启之后不会丢失

## 16）使用mount命令进行挂载

mount挂载语法如下，将分区挂载到目标目录

```
mount 设备 目标目录
```

**分区和目录的映射关系如下**

| 分区（也叫挂载点或者设备） | 目录 |
| --- | --- |
| /dev/sdb1 | /home/xxx/dir1 |
| /dev/sdb2 | /home/xxx/dir2 |
| /dev/sdb3 | /home/xxx/dir3 |
| /dev/sdb5 | /home/xxx/dir5 |
| /dev/sdb6 | /home/xxx/dir6 |

执行下面命令，对5个分区进行挂载到表格中对应的目录

```
# 创建5个目标目录
[root@testx home]# mkdir -p /home/xxx/dir1 /home/xxx/dir2 /home/xxx/dir3
/home/xxx/dir5 /home/xxx/dir6
# 依次将3个主分区和2个逻辑分区挂载到对应的目录
[root@testx home]# mount /dev/sdb1 /home/xxx/dir1/
[root@testx home]# mount /dev/sdb2 /home/xxx/dir2/
[root@testx home]# mount /dev/sdb3 /home/xxx/dir3/
[root@testx home]# mount /dev/sdb5 /home/xxx/dir5/
[root@testx home]# mount /dev/sdb6 /home/xxx/dir6/
```

再来看看挂载后的情况，如下

```
# 看一下挂载情况，都ok的
[root@testx dir1]# lsblk -f /dev/sdb
NAME   FSTYPE LABEL UUID                                 MOUNTPOINT
sdb
├─sdb1 xfs          aaac1551-a072-4bd7-a8b0-a0209011f8ca /home/xxx/dir1
├─sdb2 xfs          bed7ee3e-2f6f-408c-b720-49af694bbc7a /home/xxx/dir2
├─sdb3 xfs          6e712f33-c47a-423a-9bfd-372434437549 /home/xxx/dir3
├─sdb4
├─sdb5 xfs          30f62f64-316e-477d-8b59-99fec197ac28 /home/xxx/dir5
└─sdb6 xfs          18768b27-579c-4b2c-bf9d-326eb47bb7c5 /home/xxx/dir6
```

下面来测试一下效果，向`/home/xxx/dir1`目录中扔个大文件，看看对应的分区`/dev/sdb1`磁盘使用情况是否发生变化了

```
# 丢文件之前分区/dev/sdb1的已用量：33M
[root@testx dir1]# df -h /dev/sdb1
文件系统        容量  已用  可用 已用% 挂载点
/dev/sdb1       4.0G   33M  4.0G    1% /home/xxx/dir1

# 找个大于100M的文件，丢进去
[root@testx dir1]# find / -size +100M

# 我们就将上面找出来的文件，随便拷贝一个到/home/xxx/dir1目录
[root@testx dir1]# cp /var/cache/yum/x86_64/7/updates/packages/firefox-91.8.0-
1.el7.centos.x86_64.rpm /home/xxx/dir1

# 看一下dir1中新增的文件大小，107M
[root@testx dir1]# ll -h /home/xxx/dir1/
总用量 107M
-rw-r--r--. 1 root root   12 5月  10 19:42 1.txt
-rw-r--r--. 1 root root 107M 5月  10 19:52 firefox-91.8.0-1.el7.centos.x86_64.rpm

# 此时分区/dev/sdb1已用量变成139M了
[root@testx dir1]# df -h /dev/sdb1
文件系统        容量  已用  可用 已用% 挂载点
/dev/sdb1       4.0G  139M  3.9G    4% /home/xxx/dir1
```

### 17）设置自动挂载

mount命令的挂载方式，重启后挂载关系会丢失。

linux提供了自动挂载的方式：配置文件`/etc/fstab`，用来配置设备的挂载信息，系统启动后会自动读取这个目录，执行自动挂载操作。

将下面信息加入到`/etc/fstab`中，即可实现自动挂载

```
/dev/sdb1 /home/xxx/dir1                    xfs     defaults        0 0
/dev/sdb2 /home/xxx/dir2                    xfs     defaults        0 0
/dev/sdb3 /home/xxx/dir3                    xfs     defaults        0 0
/dev/sdb5 /home/xxx/dir5                    xfs     defaults        0 0
/dev/sdb6 /home/xxx/dir6                    xfs     defaults        0 0
```


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 18、进程管理类命令（ps、kill、pstree、top、netstat）

## 18.1、ps：查看当前系统进程状态

### 1）基本语法

| 语法 | 说明 |
| --- | --- |
| ps aux | 查看系统中所有进程 |
| ps -ef | 可以查看父子进程之间的关系 |

### 2）选项说明

| 选项 | 说明 |
| --- | --- |
| a | 列出带有终端的所有用户的进程 |
| x | 列出当前用户的所有进程，包括没有终端的进程 |
| u | 面相用户友好的显示风格 |
| -e | 列出所有进程 |
| -u | 列出某个用户关联的所有进程 |
| -f | 显示完整格式的进程列表 |

## 3）ps aux：查看进程CPU、内存占用率

```
[root@testx ~]# ps aux
USER        PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root          1  0.0  0.3 194084  6244 ?        Ss   5月12   0:47
/usr/lib/systemd/systemd --switched-root --system --deserialize 22
root          2  0.0  0.0      0     0 ?        S    5月12   0:00 [kthreadd]
root          4  0.0  0.0      0     0 ?        S<   5月12   0:00 [kworker/0:0H]
```

- USER：该进程是由哪个用户产生的
- PID：进程的 ID 号
- %CPU：该进程占用 CPU 资源的百分比，占用越高，进程越耗费资源
- %MEM：该进程占用物理内存的百分比，占用越高，进程越耗费资源
- VSZ：该进程占用虚拟内存的大小，单位 KB
- RSS：该进程占用实际物理内存的大小，单位 KB
- TTY：该进程是在哪个终端中运行的。对于 CentOS 来说，tty1 是图形化终端，tty2-tty6 是本地的字符界面终端。pts/0-255 代表虚拟终端。
- STAT：进程状态，常见的状态有：
  - R：运行状态
  - S：睡眠状态
  - T：暂停状态
  - Z：僵尸状态
  - s：包含子进程
  - l：多线程
  - +：前台显示
- START：该进程的启动时间
- TIME：该进程占用 CPU 的运算时间，注意不是系统时间
- COMMAND：产生此进程的命令名

## 4）ps -ef：显示父子进程信息

如果想查看进程的 CPU 占用率和内存占用率，可以使用 `ps aux`

如果想查看进程的父进程 ID 可以使用 `ps ef`

```
[root@testx ~]# ps -ef
UID         PID   PPID  C STIME TTY          TIME CMD
root          1      0  0 5月12 ?       00:00:47 /usr/lib/systemd/systemd --
switched-root --system --deserialize 22
root          2      0  0 5月12 ?       00:00:00 [kthreadd]
root          4      2  0 5月12 ?       00:00:00 [kworker/0:0H]
root          6      2  0 5月12 ?       00:00:01 [ksoftirqd/0]
```

- UID：用户 ID
- PID：进程 ID
- PPID：父进程 ID
- C：CPU 用于计算执行优先级的因子。数值越大，表明进程是 CPU 密集型运算，执行优先级会降低；数值越小，表明进程是 I/O 密集型运算，执行优先级会提高
- STIME：进程启动的时间
- TTY：完整的终端名称

- TIME： CPU 时间
- CMD： 启动进程所用的命令和参数

### 5） ps -Lf pid：查看某个进程的所有内核线程

```
[root@testx ~]# ps -Lf 103268
UID          PID    PPID     LWP  C NLWP STIME TTY      STAT   TIME CMD
root      103268 101406 103268  0   21 14:32 pts/0    Sl+    0:00 java
ThreadStatus
root      103268 101406 103269  0   21 14:32 pts/0    Sl+    0:00 java
ThreadStatus
root      103268 101406 103270  0   21 14:32 pts/0    Sl+    0:00 java
ThreadStatus
root      103268 101406 103271  0   21 14:32 pts/0    Sl+    0:00 java
ThreadStatus
root      103268 101406 103272  0   21 14:32 pts/0    Sl+    0:00 java
ThreadStatus
root      103268 101406 103273  0   21 14:32 pts/0    Sl+    0:00 java
ThreadStatus
root      103268 101406 103275  0   21 14:32 pts/0    Sl+    0:00 java
ThreadStatus
```

- UID： 用户 ID
- PID： 进程 ID
- PPID： 父进程 ID
- C： CPU 用于计算执行优先级的因子。 数值越大， 表明进程是 CPU 密集型运算，执行优先级会降低； 数值越小， 表明进程是 I/O 密集型运算， 执行优先级会提高
- STIME： 进程启动的时间
- TTY： 完整的终端名称
- TIME： CPU 时间
- CMD： 启动进程所用的命令和参数
- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**

## 18.2、kill：终止进程

### 1）基本语法

| 语法 | 说明 |
| --- | --- |
| kill [选项] 进程号 | 通过进程号杀死进程 |
| killall 进程名称 | 通过进程名称杀死进程，也支持通配符，这在系统因负载过大而变得很慢时很有用 |

### 2）选项

| 选项 | 说明 |
| --- | --- |
| -9 | 表示强迫进程立即执行 |

### 3）案例实操

#### （1）关闭火狐浏览器

```
# 通过ps查询出firefox的进程，发现进程id是1855
[root@testx java]# ps -ef | grep firefox
root       1855  67619 22 21:22 ?        00:00:15 /usr/lib64/firefox/firefox
# 结束1855这个进程
[root@testx java]# kill -9 1855
```

#### （2）通过进程名称杀死进程

```
[root@testx firefox]# killall firefox
```

## 18.3、pstree：查看树进程

pstree命令以树状图的方式展现进程之间的派生关系，显示效果比较直观。

### 1）基本语法

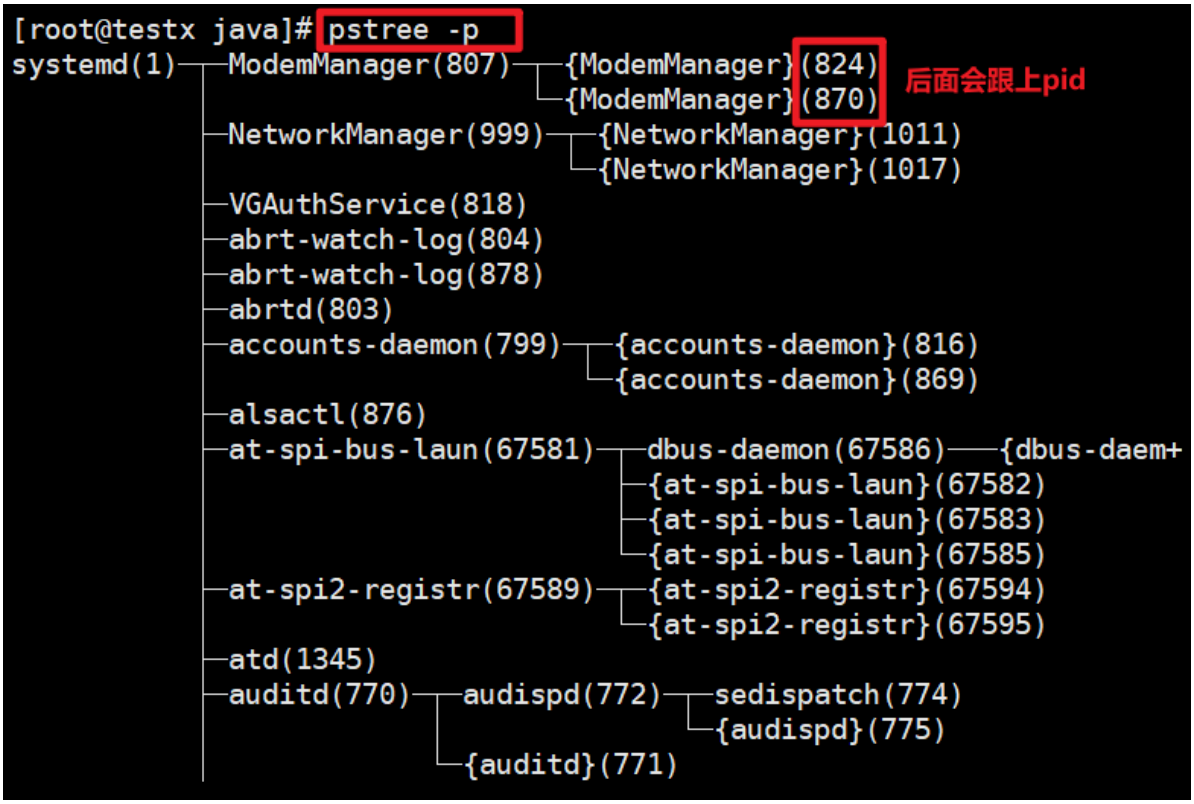
```
pstree [选项]
```

### 2）选项说明

| 选项 | 说明 |
| --- | --- |
| -p | 显示进程的PID |
| -u | 显示进程的所属用户 |

### 3）案例实操

#### （1）显示进程pid




### （2）查看一个特定的进程

如果您希望pstree仅显示特定进程的父级和子级信息，请使用-s选项。

```
pstree -s [PID]
```

如下，6826是一个java的进程，还可以看到这个进程下面的所有线程列表




## 18.4、top：实时监控系统状态

## 1）语法

```
top [选项]
```

## 2）选项说明

| 选项 | 说明 |
| --- | --- |
| -d 秒数 | 指定top命令每隔几秒刷新一下结果，默认是3秒在top命令的交互模式当中可以执行命令 |
| -i | 使top不显示任何闲置或者僵死进程 |
| -p | 通过指定监控进程ID来监控某个进程的状态 |
| -c | 显示整个命令行而不只是显示命令名 |

## 3）操作说明

在top命令结果中可以执行下面操作参与交互

| 操作 | 功能 |
| --- | --- |
| P | 以CPU使用率排序，默认就是此项 |
| M | 以内存的使用率排序 |
| N | 以PID排序 |
| q | 退出top |

## 4）查询结果字段解释

```
top - 22:56:30 up 1 day, 16:32,  4 users,  load average: 0.00, 0.01, 0.05
Tasks: 267 total,   1 running, 266 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  1863032 total,    87068 free,   812768 used,   963196 buff/cache
KiB Swap:  2097148 total,  2091004 free,     6144 used.   856636 avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND

   811 root      20   0   21684   1264    952 S   0.3  0.1   0:18.61 irqbalance

   832 root      20   0  201428   1088    588 S   0.3  0.1   0:00.58 gssproxy

 67907 root      20   0  608612  13344   6708 S   0.3  0.7   1:01.76 vmtoolsd

     1 root      20   0  194084   6260   3188 S   0.0  0.3   1:03.54 systemd

     2 root      20   0       0      0      0 S   0.0  0.0   0:00.28 kthreadd

     4 root       0 -20       0      0      0 S   0.0  0.0   0:00.00
kworker/0:0H
     6 root      20   0       0      0      0 S   0.0  0.0   0:02.08 ksoftirqd/0
```

```
      7 root      rt   0       0      0      0 S   0.0  0.0   0:00.24 migration/0
      8 root      20   0       0      0      0 S   0.0  0.0   0:00.00 rcu_bh
      9 root      20   0       0      0      0 S   0.0  0.0   0:32.09 rcu_sched
     10 root       0 -20       0      0      0 S   0.0  0.0   0:00.00 lru-add-
drain
     11 root      rt   0       0      0      0 S   0.0  0.0   0:01.23 watchdog/0
```

### （1）第1行信息为任务队列信息

| 内容 | 说明 |
| --- | --- |
| 12:26:46 | 系统当前时间 |
| up 1 day, 13:32 | 系统的运行时间，本金以运行1天13小时32分 |
| 2 users | 当前登录了2个用户 |
| load average: 0.00, 0.00, 0.00 | 系统在之前 1 分钟， 5 分钟， 15 分钟的平均负 载。 一般认为小于 1 时，负载较小。 如果大于 1， 系统已经超出负荷 |

### （2）第2行为进程信息

| 内容 | 说明 |
| --- | --- |
| Tasks: 95 total | 系统中的进程总数 |
| 1 running | 正在运行的进程数 |
| 94 sleeping | 睡眠的进程数 |
| 0 stopped | 正在停止的进程数 |
| 0 zombie | 僵尸进程。 如果不是 0， 需要手工检查僵尸进程 |

### （3）第3行为CPU信息

| 内容 | 说明 |
| --- | --- |
| us | 用户空间占用的 CPU 百分比 |
| sy | 内核空间占用的 CPU 百分比 |
| ni | 改变过优先级的用户进程占用的 CPU 百分比 |
| id | 空闲 CPU 百分比 |
| wa | 等待输入/输出的进程的占用 CPU 百分比 |
| hi | 硬中断请求服务占用的 CPU 百分比 |
| si | 软中断请求服务占用的 CPU 百分比 |
| st | st（ Steal time） 虚拟时间百分比。 就是当有虚拟 机时， 虚拟 CPU 等待实际 CPU 的时间百分比。 |

### （4）第4行为物理内存信息

| 内容 | 说明 |
| --- | --- |
| total | 物理内存的总量， 单位 KB |
| used | 已经使用的物理内存数量 |
| free | 空闲的物理内存重量 |
| buffers | 作为缓冲的内存数量 |

### （5）第5行为交换分区信息

| 内容 | 说明 |
| --- | --- |
| Swap: 524280k total | 交换分区（虚拟内存） 的总大小 |
| 0k used | 已经使用的交换分区的大小 |
| 524280k free | 空闲交换分区的大小 |
| 409280k cached | 作为缓存的交互分区的大小 |

### （6）进程信息区

统计信息区域的下方显示了各个进程的详细信息，首先来认识一下各列的含义。

- PID 进程id
- PPID 父进程id
- USER Real user name
- UID 进程所有者的用户id
- USER 进程所有者的用户名
- GROUP 进程所有者的组名
- TTY 启动进程的终端名。不是从终端启动的进程则显示为 ?

- PR 优先级
- NI nice值。负值表示高优先级，正值表示低优先级
- P 最后使用的CPU，仅在多CPU环境下有意义
- %CPU 上次更新到现在的CPU时间占用百分比
- TIME 进程使用的CPU时间总计，单位秒
- TIME+ 进程使用的CPU时间总计，单位1/100秒
- %MEM 进程使用的物理内存百分比
- VIRT 进程使用的虚拟内存总量，单位kb。VIRT=SWAP+RES
- SWAP 进程使用的虚拟内存中，被换出的大小，单位kb
- RES 进程使用的、未被换出的物理内存大小，单位kb，RES=CODE+DATA
- CODE 可执行代码占用的物理内存大小，单位kb
- DATA 可执行代码以外的部分(数据段+栈)占用的物理内存大小，单位kb
- SHR 共享内存大小，单位kb
- nFLT 页面错误次数
- nDRT 最后一次写入到现在，被修改过的页面数。
- S 进程状态。
    - D=不可中断的睡眠状态
    - R=运行
    - S=睡眠
    - T=跟踪/停止
    - Z=僵尸进程
- COMMAND 命令名/命令行

## 5）2个案例

**（1）显示进程1的状态信息**

```
top -p 进程id
```

```
[root@testx java]# top -p 1
top - 23:25:29 up 1 day, 17:01,  4 users,  load average: 0.00, 0.01, 0.05
Tasks:   1 total,   0 running,   1 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  1863032 total,    93352 free,   798432 used,   971248 buff/cache
KiB Swap:  2097148 total,  2091004 free,     6144 used.   870968 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
    1 root      20   0  194084   6260   3188 S   0.3  0.3   1:04.51 systemd
```

**（2）2500 毫秒刷新一次 TOP 内容，总共 5 次，输出内容存放到 performace.txt 文件中**

> 注：要将内容输出到文件中，必须使用 - b，表示批处理选项

```
top -b -d 2.5 -n 5 > performace.txt
```

**（3）只显示活动中的进程**

```
[root@testx java]# top -i
top - 23:28:06 up 1 day, 17:04,  4 users,  load average: 0.00, 0.01, 0.05
Tasks: 266 total,   1 running, 265 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.2 us,  0.2 sy,  0.4 ni, 99.2 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  1863032 total,    92560 free,   799176 used,   971296 buff/cache
KiB Swap:  2097148 total,  2091004 free,     6144 used.   870192 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
67933 root      39  19  737012  13668   5724 S   2.3  0.7   0:43.54 tracker-extract
67982 root      20   0  592436  14136   5348 S   0.7  0.8   0:09.34 tracker-store
    1 root      20   0  194084   6260   3188 S   0.3  0.3   1:04.63 systemd
67944 root      39  19  649468   8624   5304 S   0.3  0.5   0:02.43 tracker-miner-f
68052 root      20   0  318084   3240   2580 S   0.3  0.2   0:03.02 gvfsd-metadata
```

# 18.4、netstat：显示网络状态和端口占用信息

## 1） 基本语法

| 命令 | 说明 |
| --- | --- |
| netstat -anp \| grep 进程号 | 查看该进程网络信息 |
| netstat –nlp \| grep 端口号 | 查看网络端口号占用情况 |

## 2）选项说明

| 选项 | 功能 |
| --- | --- |
| -a | 显示所有正在监听（listen） 和未监听的套接字（socket） |
| -n | 拒绝显示别名， 能显示数字的全部转化成数字 |
| -l | 仅列出在监听的服务状态 |
| -p | 表示显示哪个进程在调用 |

## 3）连接状态详解

- **LISTEN**： 侦听来自远方的TCP端口的连接请求
- **SYN-SENT**： 再发送连接请求后等待匹配的连接请求
- **SYN-RECEIVED**：再收到和发送一个连接请求后等待对方对连接请求的确认
- **ESTABLISHED**： 代表一个打开的连接
- **FIN-WAIT-1**： 等待远程TCP连接中断请求，或先前的连接中断请求的确认
- **FIN-WAIT-2**： 从远程TCP等待连接中断请求
- **CLOSE-WAIT**： 等待从本地用户发来的连接中断请求
- **CLOSING**： 等待远程TCP对连接中断的确认
- **LAST-ACK**： 等待原来的发向远程TCP的连接中断请求的确认
- **TIME-WAIT**： 等待足够的时间以确保远程TCP接收到连接中断请求的确认
- **CLOSED**： 没有任何连接状态

### 4） 案例实操

#### （1） 通过进程号查看sshd进程的网络信息

```
[root@testx java]# netstat -anp | grep sshd
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
 69266/sshd
tcp        0      0 192.168.200.135:22      192.168.200.1:64660     ESTABLISHED
5266/sshd: root@pts
tcp        0     52 192.168.200.135:22      192.168.200.1:51188     ESTABLISHED
117205/sshd: root@p
tcp        0      0 192.168.200.135:22      192.168.200.1:64607     ESTABLISHED
5104/sshd: root@pts
tcp6       0      0 :::22                   :::*                    LISTEN
 69266/sshd
unix  3      [ ]         STREAM     CONNECTED     271445   69266/sshd
unix  2      [ ]         DGRAM                    478469   117205/sshd: root@p
unix  2      [ ]         DGRAM                    549896   5104/sshd: root@pts
unix  2      [ ]         DGRAM                    542430   5266/sshd: root@pts
```

#### （2） 查看22端口号是否被占用

```
[root@testx java]# netstat -nltp | grep 22
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
 69266/sshd
tcp6       0      0 :::22                   :::*                    LISTEN
 69266/sshd
```


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 19、系统定时任务（crontab）

## 19.1、需开启crond服务

Linux **crontab** 是用来定期执行程序的命令。

当安装完成操作系统之后，默认便会启动此任务调度命令。

**crond** 命令每分钟会定期检查是否有要执行的工作，如果有要执行的工作便会自动执行该工作。

若crond服务没有启动，则执行下面命令启动下。

```
[root@testx java]# systemctl restart crond
```

## 19.2、crontab：定时任务设置

## 1）基础语法

```
crontab [选项]
```

crontab 是用来让使用者在固定时间或固定间隔执行程序之用，换句话说，也就是类似使用者的时程表。

-u user 是指设定指定 user 的时程表，这个前提是你必须要有其权限(比如说是 root)才能够指定他人的时程表。如果不使用 -u user 的话，就是表示设定自己的时程表。

## 2）选项说明

| 选项 | 功能 |
| --- | --- |
| -e | 编辑 crontab 定时任务 |
| -l | 查询 crontab 定时任务 |
| -r | 删除当前用户所有的crontab任务 |

## 3）添加任务

### （1）编辑任务

> 执行`crontab -e`命令进入定时任务编辑界面，操作过程按照vim编辑文件的过程，每行为一个定时任务，保存之后，定时任务设置成功。

```
[root@testx java]# crontab -e

* * * * * date >> /root/2.txt
* * * * * date >> /root/4.txt
```

### （2）任务格式

```
* * * * * 执行的任务
```

| 项目 | 含义 | 范围 |
| --- | --- | --- |
| 第一个“*” | 一小时当中的第几分钟 | 0-59 |
| 第二个“*” | 一天当中的第几小时 | 0-23 |
| 第三个“*” | 一个月当中的第几天 | 1-31 |
| 第四个“*” | 一年当中的第几月 | 1-12 |
| 第五个“*” | 一周当中的星期几 | 0-7 （0和7都代表星期日） |

### （3）特殊符号说明

| 特殊符号 | 含义 |
| --- | --- |
| * | 代表任何时间。 比如第一个“ *” 就代表一小时中每分钟 都执行一次的意思。 |
| , | 代表不连续的时间。 比如“ 0 8,12,16 * * * 命令”， 就代表 在每天的 8 点 0 分， 12 点 0 分， 16 点 0 分都执行一次命令 |
| - | 代表连续的时间范围。 比如“0 5 * * 1-6 命令”， 代表在 周一到周六的凌晨 5 点 0 分执行命令 |
| */n | 代表每隔多久执行一次。 比如“*/10 * * * * 命令”， 代 表每隔 10 分钟就执行一遍命令 |

### （4） 特定时间执行命令

| 时间 | 含义 |
| --- | --- |
| 45 22 * * * 命令 | 每天 22 点 45 分执行命令 |
| 0 17 * * 1 命令 | 每周 1 的 17 点 0 分执行命令 |
| 0 5 1,15 * * 命令 | 每月 1 号和 15 号的凌晨 5 点 0 分执行命令 |
| 40 4 * * 1-5 命令 | 每周一到周五的凌晨 4 点 40 分执行命令 |
| */10 4 * * * 命令 | 每天的凌晨 4 点， 每隔 10 分钟执行一次命令 |
| 0 0 1,15 * 1 命令 | 每月 1 号和 15 号， 每周 1 的 0 点 0 分都会执行命令。 注 意： 星期几和几号最好不要同时出现， 因为他们定义的都 是天。 非常容易让管理员混乱。 |

## 19.3、案例

### （1）当前用户添加一个任务

> 每隔一分钟将当前时间输出到/root/date.txt文件中

```
* * * * * date >> /root/date.txt
```

### （2）参看当前用户任务列表

```
[root@testx java]# crontab -l
* * * * * date >> /root/2.txt
* * * * * date >> /root/4.txt
```

### （3）删除当前用户所有任务

```
# 列出当前用户所有定时任务，有2个
[root@testx ~]# crontab -l
* * * * * date >> /root/2.txt
* * * * * date >> /root/4.txt
# crontab -r 会删除当前用户所有定时任务
[root@testx ~]# crontab -r
# 列出当前用户所有任务，没有了
[root@testx ~]# crontab -l
no crontab for root
```

### （4）如果想删除某个任务怎么办呢?

可以通过`crontab -e`编辑定时任务列表实现增删改。


- **领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】**
- **更多学习资料**


# 20、软件包管理

## 20.1、RPM

### 20.1.1、RPM 概述

RPM（RedHat Package Manager）， RedHat软件包管理工具， 类似windows里面的setup.exe，是Linux这系列操作系统里面的打包安装工具， 它虽然是RedHat的标志， 但理念是通用的。

RPM包的名称格式

Apache-1.3.23-11.i386.rpm

- “apache” 软件名称
- “1.3.23-11”软件的版本号， 主版本和此版本
- “i386”是软件所运行的硬件平台， Intel 32位处理器的统称
- “rpm”文件扩展名， 代表RPM包

### 20.1.2、RPM 查询命令（rpm -qa）

#### 1） 基本语法

| 语法 | 说明 |
| --- | --- |
| rpm -q 程序包名称 | 查询指定程序包是否安装 |
| rpm -qa | 显示当前系统中所有rpm程序包列表 |
| rpm -qi 程序包名称 | 查看指定程序包的名称、版本、许可协议、用途描述等详细信息 |
| rpm -ql 程序包名称 | 显示指定的程序包在当前系统中安装的所有目录、文件列表 |
| rpm -qf 文件名或目录 | 查看指定的文件或目录是由哪个程序包所安装的 |

### 2）经验技巧

由于软件包比较多，一般都会采取过滤，rpm -qa|grep rpm软件包

### 3） 案例实操：查询firefox软件安装情况

```
[root@testx ~]# rpm -qa|grep firefox
firefox-91.9.0-1.el7.centos.x86_64
```

## 20.1.3、RPM 卸载命令（rpm -e）

### 1） 基本语法

- rpm -e RPM软件包
- rpm -e --nodeps 软件包

### 2）选项说明

| 选项 | 功能 |
| --- | --- |
| -e | 卸载软件包 |
| --nodeps | 卸载软件时，不检查依赖，这样的话，那些使用该软件包的软件在此之后可能就不能正常工作了。 |

### 3） 案例实操：卸载firefox软件

```
[root@testx ~]# rpm -e firefox
```

## 20.1.2 RPM 查询命令（rpm -ivh）

### 1） 基本语法

```
rpm -ivh RPM包全名
```

### 2）选项说明

| 选项 | 功能 |
| --- | --- |
| -i | install， 安装 |
| -v | --verbose， 显示详细信息 |
| -h | --hash， 进度条 |
| --nodeps | 安装前不检查依赖 |

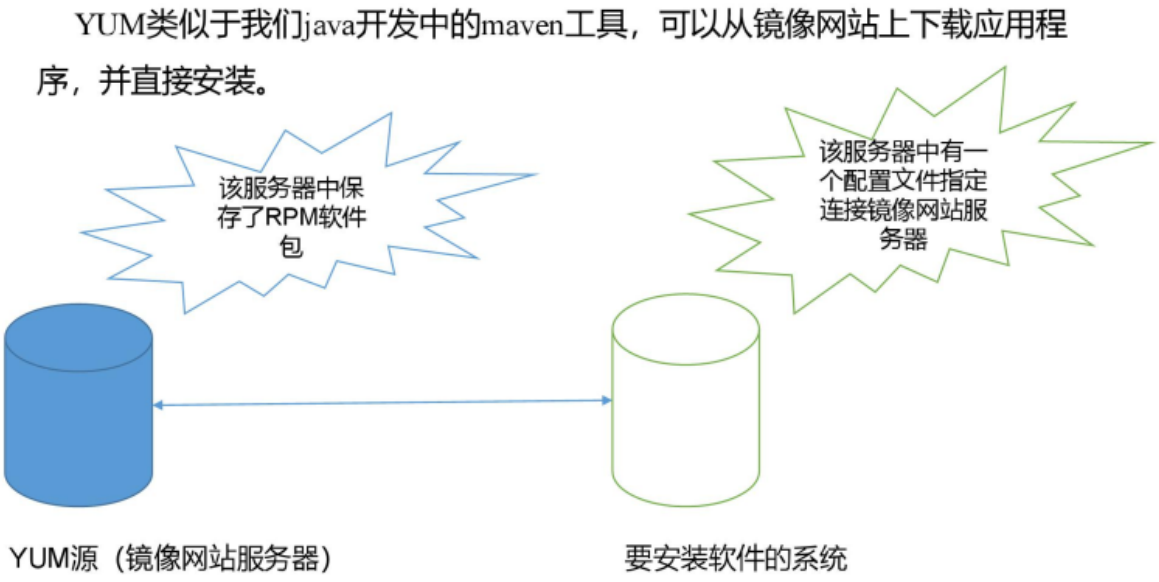
### 3） 案例实操：安装firefox软件

```
[root@testx Packages]# rpm -ivh firefox-68.10.0-1.el7.centos.x86_64.rpm
准备中...                          ################################# [100%]
正在升级/安装...
   1:firefox-68.10.0-1.el7.centos     ################################# [100%]
```

# 20.2 YUM 仓库配置

## 20.2.1 YUM 概述

YUM （全称为 Yellow dog Updater, Modified） 是一个在 Fedora 和 RedHat 以及 CentOS 中的 Shell 前端软件包管理器。 基于 RPM 包管理， 能够从指定的服务器自动下载 RPM 包 并且安装， 可以自动处理依赖性关系， 并且一次安装所有依赖的软件包， 无须繁琐地一次 次下载、 安装， 如图所示




## 20.2.2 YUM 的常用命令

### 1） 基本语法

```
yum [选项] [参数]
```

### 2） 选项说明

| 选项 | 功能 |
| --- | --- |
| -y | 使用yum安装或者卸载软件的过程中，会有提示yes还是no，跟上-y选项后，对所有提问都回答“yes” |

### 3） 参数说明

| 参数 | 功能 |
| --- | --- |
| install | 安装 rpm 软件包 |
| update | 更新 rpm 软件包 |
| check-update | 检查是否有可用的更新 rpm 软件包 |
| remove | 删除指定的 rpm 软件包 |
| list | 显示所有已经安装和可以安装的程序包 |
| clean | 清理 yum 过期的缓存 |
| deplist | 显示 yum 软件包的所有依赖关系 |
| info | 显示关于软件包或组的详细信息 |

### 4）常用命令

| 命令 | 说明 |
| --- | --- |
| yum search 软件包 | 查找某个软件包 |
| yum list | 列出所有可安装的软件包（含已安装、可安装的软件包列表） |
| `yum list java\*` | 列出java开头的软件包 |
| yum list updates | 列出所有可安装的软件包 |
| yum list installed | 列出所有以安装的软件包 |
| yum info 软件包 | 列出某个软件包的信息 |
| `yum list java\*` | 列出所有以java开头的软件包 |
| yum remove 软件包 | 卸载指定软件 |
| yum install 软件包 | 安装指定软件包 |
| yum update 软件包 | 升级指定软件包 |

### 5）案例实操实操（java1.8 查找、安装、卸载）

#### （1） 查找java1.8

```
[root@testx ~]# yum list | grep "^java-1.8\\|^jdk-1.8"
java-1.8.0-openjdk.i686                     1:1.8.0.332.b09-1.el7_9    updates
java-1.8.0-openjdk.x86_64                   1:1.8.0.332.b09-1.el7_9    updates
```

**（2）咱们就安装查找出来的第2个jdk：java-1.8.0-openjdk.x86_64**

```
[root@testx ~]# yum -y install java-1.8.0-openjdk.x86_64
```

**（3）卸载java-1.8**

先使用`yum list installed java-1.8*`查询已安装的java8包列表




然后执行下面命令，把这两个包干掉

```
[root@testx ~]# yum -y remove java-1.8.0-openjdk.x86_64 java-1.8.0-openjdk-
headless.x86_64
```

## 20.2.3、修改网络 YUM 源

默认的系统 YUM 源， 需要连接国外 apache 网站， 网速比较慢， 可以修改关联的网络YUM 源为国内镜像的网站， 比如网易 163,aliyun 等。

### 1）安装 wget

> wget 用来从指定的 URL 下载文件

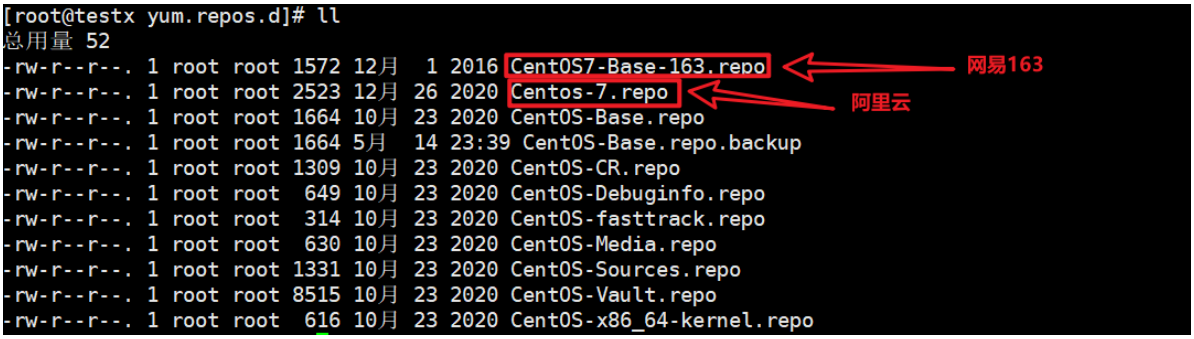
```
[root@testx ~]# yum -y install wget
```

### 2）在/etc/yum.repos.d/目录下， 备份默认的 repos 文件

```
[root@testx yum.repos.d]# pwd
/etc/yum.repos.d
[root@testx yum.repos.d]# cp CentOS-Base.repo CentOS-Base.repo.backup
```

### 3）下载网易163或者是aliyun的repos文件，任选其一

```
# 阿里云
[root@testx yum.repos.d]# wget http://mirrors.aliyun.com/repo/Centos-7.repo
# 网易163
[root@testx yum.repos.d]# wget http://mirrors.163.com/.help/CentOS7-Base-
163.repo
```




### 4）使用下载好的repos文件替换默认的repos文件

例如用 `Centos-7.repo` 替换 `CentOS-Base.repo`

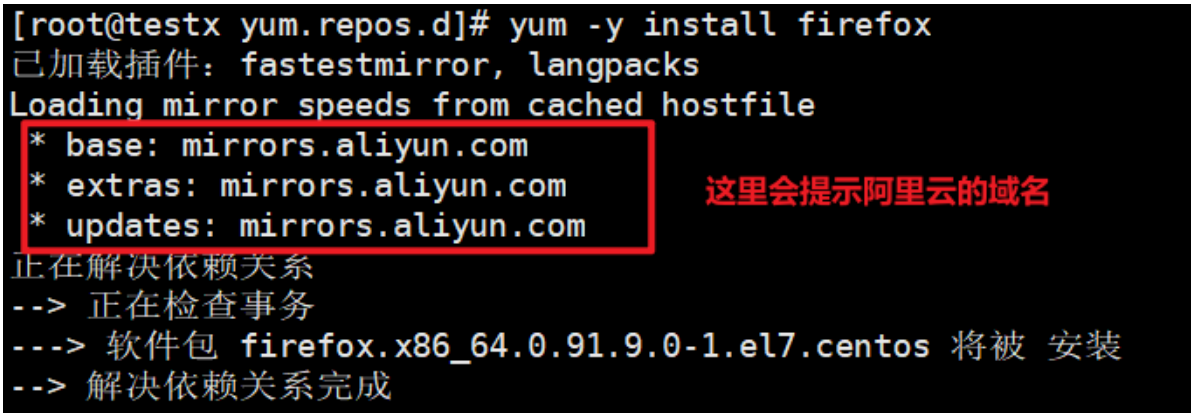
```
[root@testx yum.repos.d]# mv -f Centos-7.repo CentOS-Base.repo
```

### 5）清理旧缓存数据， 缓存新数据

```
[root@testx yum.repos.d]# yum clean all
[root@testx yum.repos.d]# yum makecache
```

### 6）测试，安装的时候会提示阿里云的域名

```
# 先卸掉firefox
[root@testx yum.repos.d]# yum -y remove firefox
# 安装firefox
[root@testx yum.repos.d]# yum -y install firefox
```





- 领取月薪4万的Java全栈学习线路图+视频【含50个核心技术栈】
- 更多学习资料